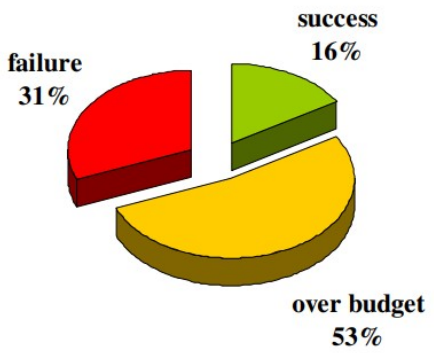
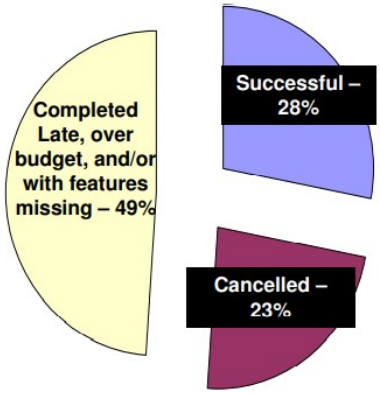
Software Engineering
Unit 1

**Q1. What is Software Crisis?**

Software industry is in Crisis!



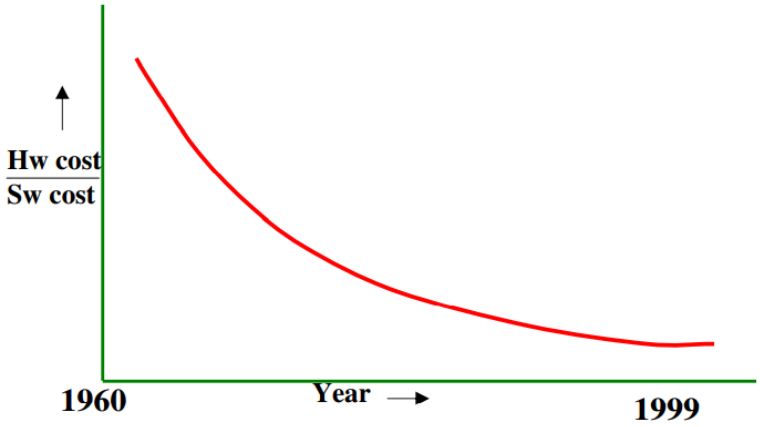

This is the SORRY state of Software Engineering Today!



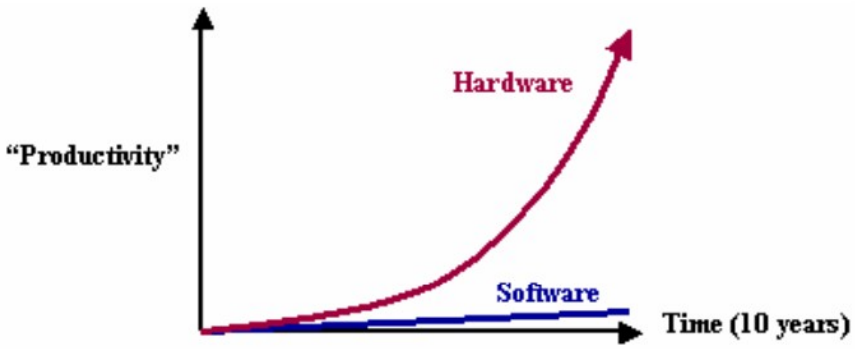

- Data on 28,000 projects completed in 2000

As per the IBM report, “31% of the project get cancelled before they are completed, 53% overrun their cost estimates by an average of 189% and for every 100 projects, there are 94 restarts”.

**Q2. Relative Cost of Hardware and Software.**




– Moore’s law: processor speed/memory capacity doubles every two years



**Q3. What are factors contributing to software crisis?**

- Larger problems,
- Lack of adequate training in software engineering,
- Increasing skill shortage,
- Low productivity improvements.

**Q4. Some Software Failures.**

## Ariane 5

**It took the European Space Agency 10 years and $7 billion** to produce Ariane 5, a giant rocket capable of hurling a pair of three-ton satellites into orbit with each launch and intended to give Europe overwhelming supremacy in the commercial space business.

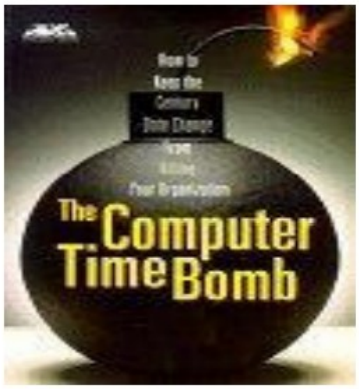
The rocket was destroyed after 39 seconds of its launch, at an altitude of two and a half miles along with its payload of four expensive and uninsured scientific satellites.

When the guidance system's own computer tried to convert one piece of data the sideways velocity of the rocket from a 64 bit format to a 16 bit format; the number was too big, and an overflow error resulted after 36.7 seconds. When the guidance system shutdown, it passed control to an identical, redundant unit, which was there to provide backup in case of just such a failure. Unfortunately, the second unit, which had failed in the identical manner a few milliseconds before.

## Y2K problem:


It was simply the ignorance about the adequacy or otherwise of using only last two digits of the year.

The 4-digit date format, like 1964, was shortened to 2-digit format, like 64.




## *The Patriot Missile*

- First time used in Gulf war
- Used as a defense from Iraqi Scud missiles
- Failed several times including one that killed 28 US soldiers in Dhahran, Saudi Arabia

**Reasons:**

A small timing error in the system's clock accumulated to the point that after 14 hours, the tracking system was no longer accurate. In the Dhahran attack, the system had been operating for more than 100 hours.

**Q5. “No Silver Bullet”**

The hardware cost continues to decline drastically.

However, there are desperate cries for a silver bullet something to make software costs drop as rapidly as computer hardware costs do.

But as we look to the horizon of a decade, we see no silver bullet. There is no single development, either in technology or in management technique, that by itself promises even one order of magnitude improvement in productivity, in reliability and in simplicity.

The hard part of building software is the specification, design and testing of this conceptual construct, not the labour of representing it and testing the correctness of representation.

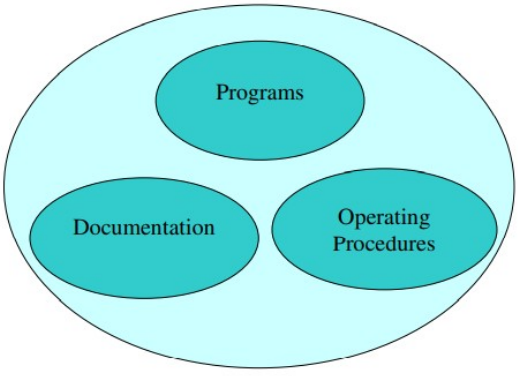
We still make syntax errors, to be sure, but they are trivial as compared to the conceptual errors (logic errors) in most systems. That is why, building software is always hard and there is inherently no silver bullet.

While there is no royal road, there is a path forward.

Is reusability (and open source) the new silver bullet?

The blame for software bugs belongs to: • Software companies • Software developers • Legal system • Universities

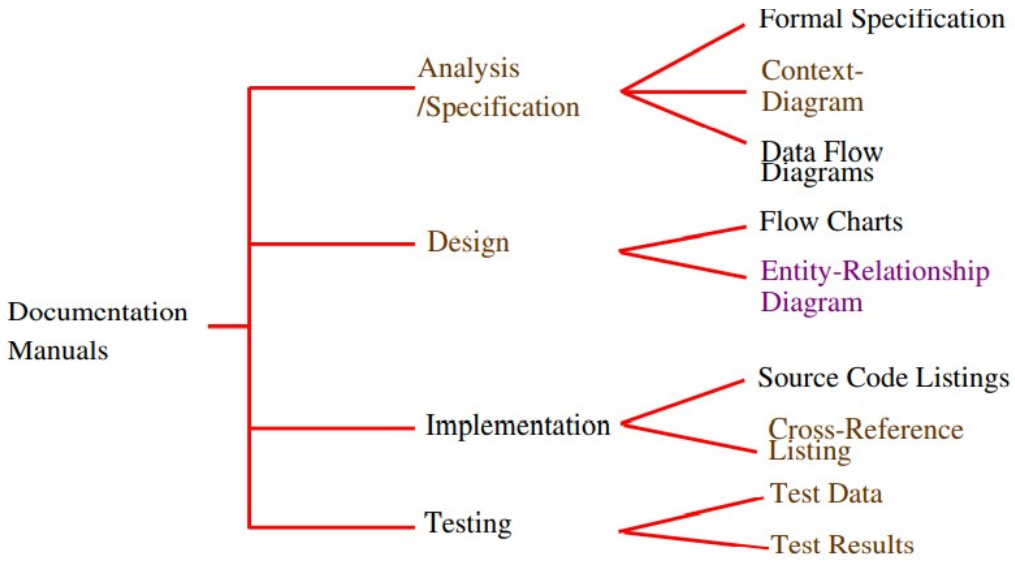
**Q6. What is software?**




Software=Program+Documentation+Operating Procedures

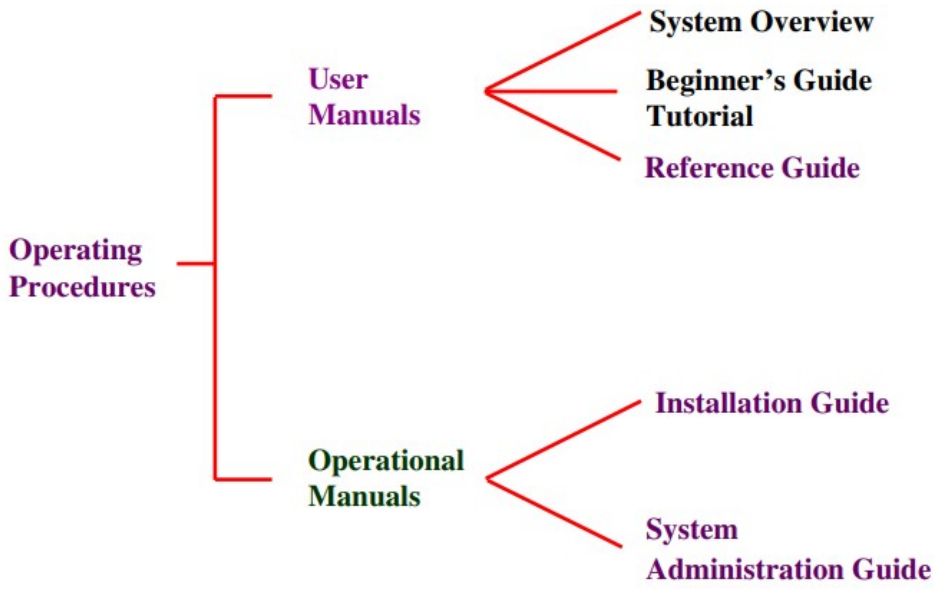
Components of software

**Q6. Documentation of software.**




List of documentation manuals

**Q7. Operating Procedures.**




List of operating procedure manuals.

**Q8. Why Software Product developed.**

• Software products may be developed for a particular customer or may be developed for a general market Software Product
• Software products may be
–Generic – developed to be sold to a range of different customers
–Bespoke (custom) - developed for a single customer according to their specification

**Q9. What is software Engineering?**

**Software engineering** is an engineering discipline which is concerned with all aspects of software production

**Software engineers** should

- adopt a systematic and organised approach to their work
- use appropriate tools and techniques depending on
  - the problem to be solved,
  - the development constraints and
- use the resources available

At the first conference on software engineering in 1968, Fritz Bauer defined software engineering as "The establishment and use of sound engineering principles in order to obtain economically developed software that is reliable and works efficiently on real machines".

Stephen Schach defined the same as "A discipline whose aim is the production of quality software, software that is delivered on time, within budget, and that satisfies its requirements".

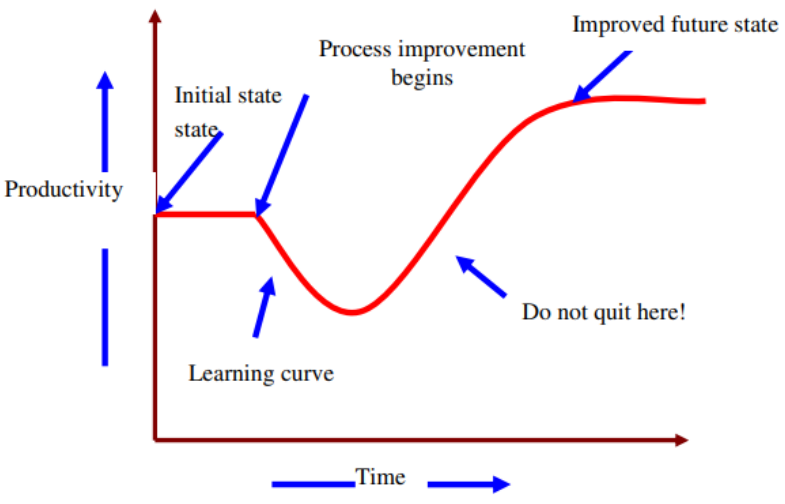
**Q10. What is Software Process?**

The software process is the way in which we produce software.

A **Software Process** refers to a structured set of activities, methods, and practices used to develop and maintain software systems. It encompasses all phases of software development, from initial conception through to delivery and ongoing maintenance.

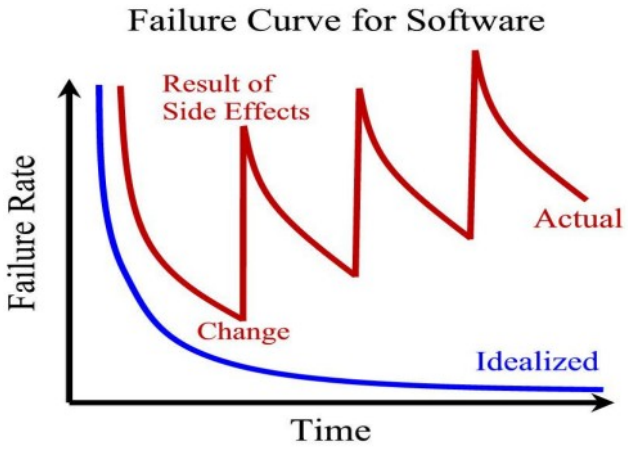
**Q11. Why is it difficult to improve software process?**

• Not enough time • Lack of knowledge • Wrong motivations • Insufficient commitment

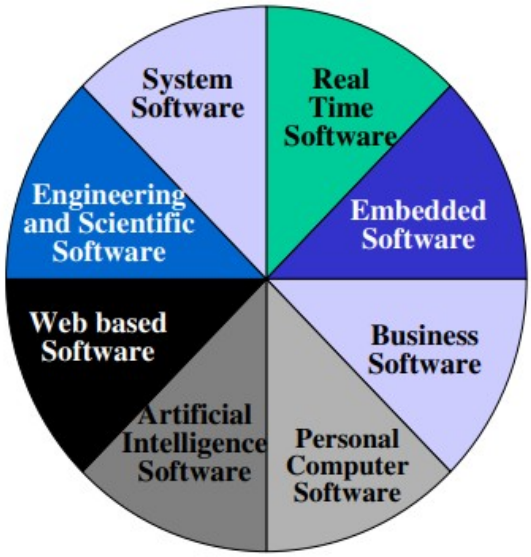


**Q12. What are characteristics of a Software?**

- Software does not wear out
- Software is not manufactured
- Reusability of component
- Software is flexible




**Q13. Changing nature of Software.**




Trend has emerged to provide source code to the customer and organizations.

Software where source codes are available are known as open source software.

Examples Open source software: LINUX, MySQL, PHP, Open office, Apache webserver etc.

**Q14. Deliverables and Milestones.**

Different deliverables are generated during software development. The examples are source code, user manuals, operating procedure manuals etc.

The milestones are the events that are used to ascertain the status of the project. Finalization of specification is a milestone. Completion of design documentation is another milestone. The milestones are essential for project planning and management.

**Q15. Product and Process**

Product: What is delivered to the customer, is called a product. It may include source code, specification document, manuals, documentation etc. Basically, it is nothing but a set of deliverables only.

Process: Process is the way in which we produce software. It is the collection of activities that leads to (a part of) a product. An efficient process is required to produce good quality products. If the process is weak, the end product will undoubtedly suffer, but an obsessive over reliance on process is also dangerous.

## Q16. Productivity and Effort.

Productivity is defined as the rate of output, or production per unit of effort, i.e. the output achieved with regard to the time taken but irrespective of the cost incurred.

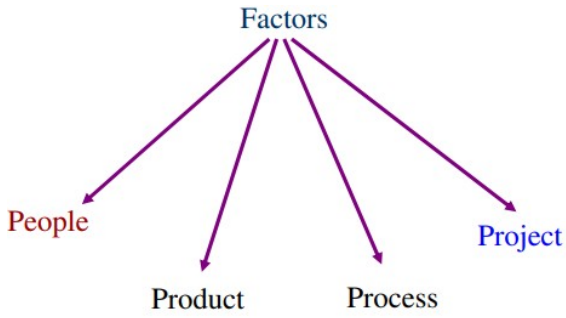
Hence most appropriate unit of effort is Person Months (PMs), meaning thereby number of persons involved for specified months. So, productivity may be measured as LOC/PM (lines of code produced/person month)

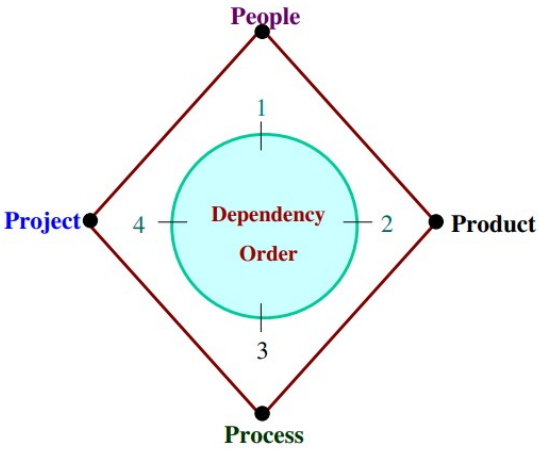
## Q17. Software Components.

“An independently deliverable piece of functionality providing access to its services through interfaces”.

“A component represents a modular, deployable, and replaceable part of a system that encapsulates implementation and exposes a set of interfaces”.

## Q18. Role of Management in Software Development.




## Q18. Software Life Cycle Models

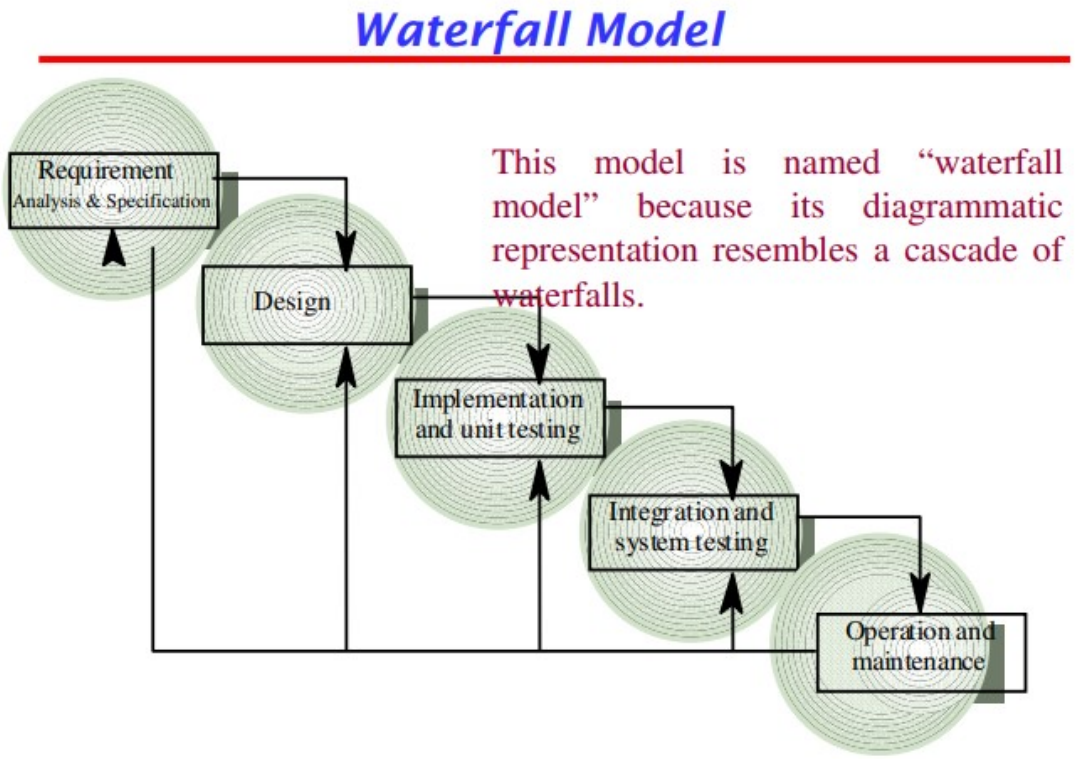
The goal of Software Engineering is to provide models and processes that lead to the production of well-documented maintainable software in a manner that is predictable.

“The period of time that starts when a software product is conceived and ends when the product is no longer available for use. The software life cycle typically includes a requirement phase, design phase, implementation phase, test phase, installation and check out phase, operation and maintenance phase, and sometimes retirement phase”.

## Q19. Waterfall Model

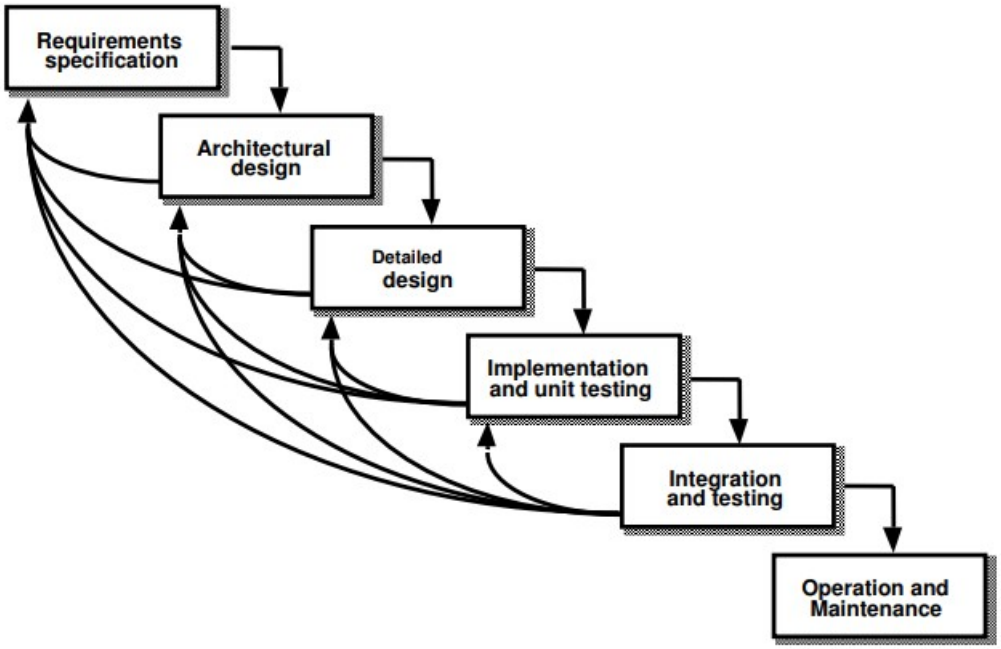
This model is easy to understand and reinforces the notion of “define before design” and “design before code”. The model expects complete & accurate requirements early in the process, which is unrealistic.

Problems of waterfall model
i. It is difficult to define all requirements at the beginning of a project
ii. This model is not suitable for accommodating any change
iii. A working version of the system is not seen until late in the project’s life
iv. It does no t scale up well to large projects.
v. Real projects are rarely sequential




## Q20. Iterative Enhancement Model

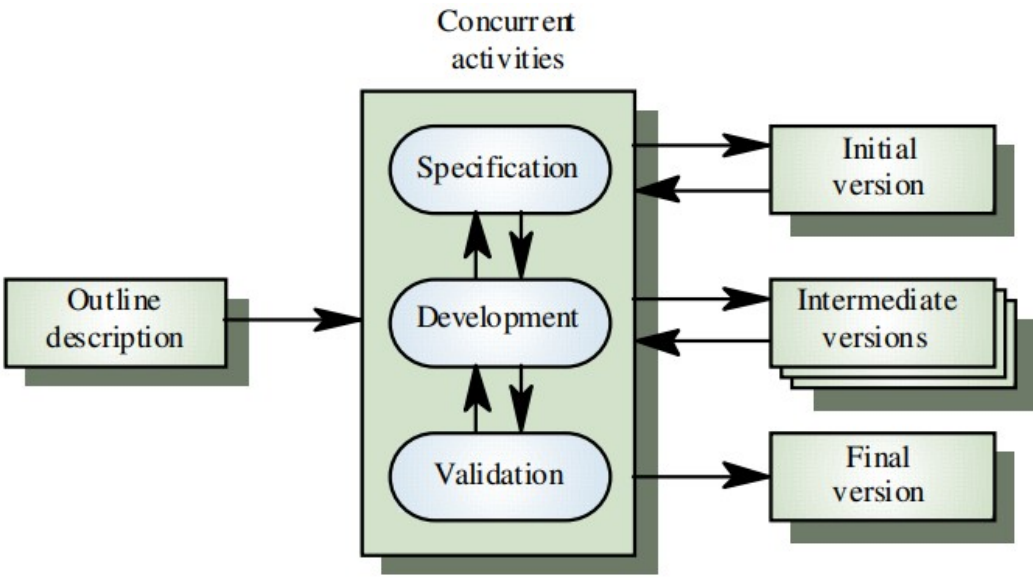
This model has the same phases as the waterfall model, but with fewer restrictions. Generally the phases occur in the same order as in the waterfall model, but they may be conducted in several cycles. Useable product is released at the end of the each cycle, with each release providing additional functionality




## Q 21. Evolutionary Process Model

Evolutionary process model resembles iterative enhancement model. The same phases as defined for the waterfall model occur here in a cyclical fashion. This model differs from iterative enhancement model in the sense that this does not require an useable product at the end of each cycle. In evolutionary development, requirements are implemented by category rather than by priority.

This model is useful for projects using new technology that is not well understood. This is also used for complex projects where all functionality must be delivered at one time, but the requirements are unstable or not well understood at the beginning

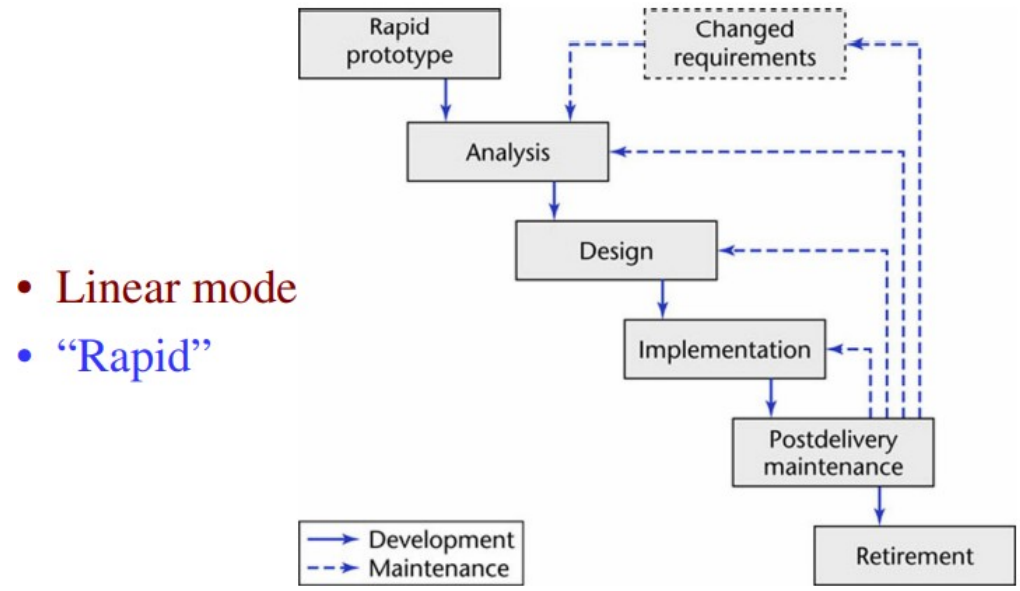


## Q22. Prototyping Model

The prototype may be a usable program but is not suitable as the final software product.

The code for the prototype is thrown away. However experience gathered helps in developing the actual system.

The development of a prototype might involve extra cost, but overall cost might turnout to be lower than that of an equivalent system developed using the waterfall model.



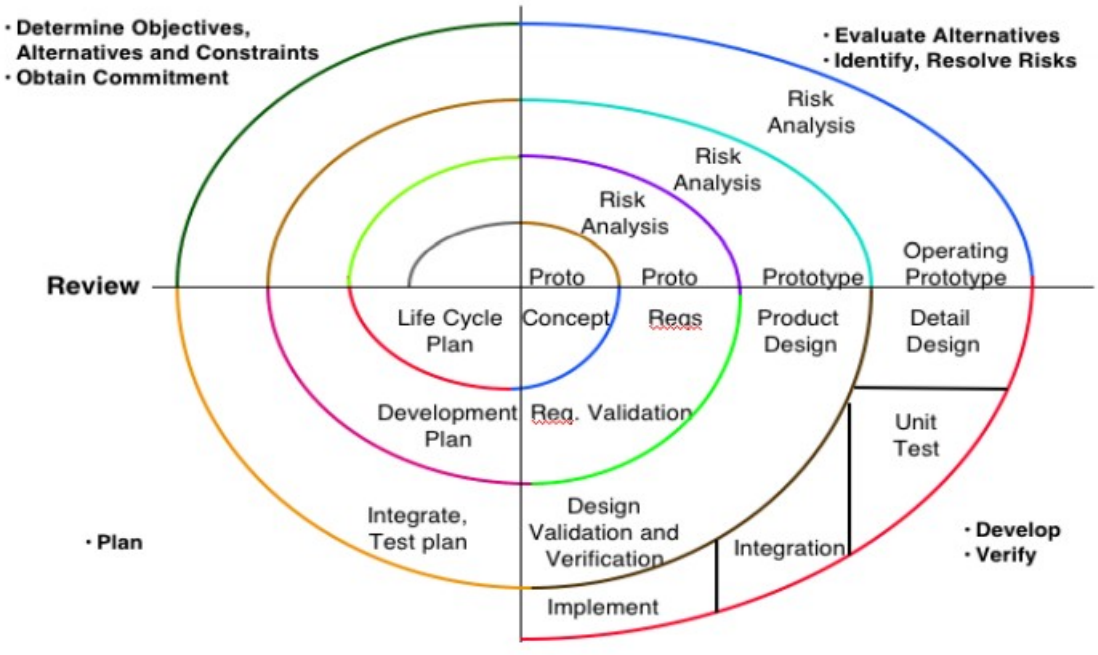

## Q23. Spiral Model

Models do not deal with uncertainly which is inherent to software projects.

Important software projects have failed because project risks were neglected & nobody was prepared when something unforeseen happened.

Barry Boehm recognized this and tired to incorporate the "project risk" factor into a life cycle model.

The result is the spiral model, which was presented in 1986



The radial dimension of the model represents the cumulative costs. Each path around the spiral is indicative of increased costs. The angular dimension represents the progress made in completing each cycle. Each loop of the spiral from X-axis clockwise through 360 represents one phase. One phase is split roughly into four sectors of major activities.

- Planning: Determination of objectives, alternatives & constraints.
- Risk Analysis: Analyze alternatives and attempts to identify and resolve the risks involved.
- Development: Product development and testing product.
- Assessment: Customer evaluation

An important feature of the spiral model is that each phase is completed with a review by the people concerned with the project (designers and programmers)

The advantage of this model is the wide range of options to accommodate the good features of other life cycle models.

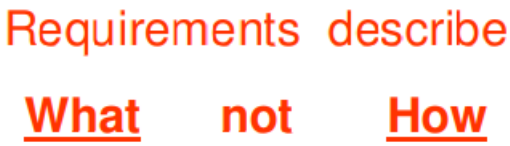
It becomes equivalent to another life cycle model in appropriate situations.

The spiral model has some difficulties that need to be resolved before it can be a universally applied life cycle model. These difficulties include lack of explicit process guidance in determining objectives, constraints, alternatives; relying on risk assessment expertise; and provides more flexibility than required for many applications.

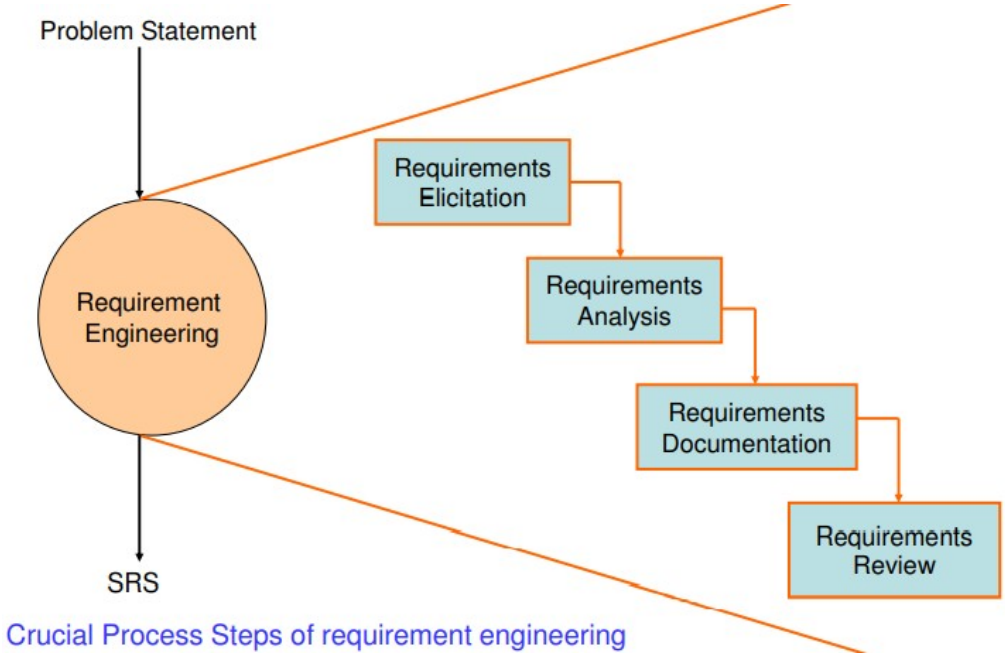
## Q24. Software Requirements Specification (SRS)

The **Software Requirements Specification (SRS)** is a comprehensive document that outlines the complete software requirements for a project. It serves as a formal agreement between stakeholders and the development team, detailing what the software should do and the constraints under which it must operate.

For example, it will describe the software's ability to handle user input, process data, and deliver appropriate results. It will also address user needs such as speed, security, and user-friendliness, but won't specify how these will be technically achieved. The document ensures clarity between stakeholders and developers, keeping the focus on "what" the system does, not "how."



## Q25. Requirement Engineering



Crucial Process Steps of requirement engineering

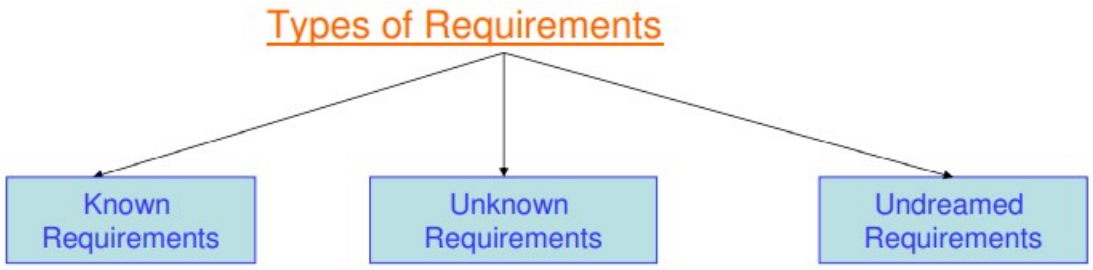
Requirement Engineering is the disciplined application of proven principles, methods, tools, and notations to describe a proposed system's intended behaviour and its associated constraints.

SRS may act as a contract between developer and customer.

State of practice Requirements are difficult to uncover
- Requirements change
- Over reliance on CASE Tools
- Tight project Schedule
- Communication barriers
- Market driven software development
- Lack of resources

A University wish to develop a software system for the student result management of its M.Tech. Programme. A problem statement is to be prepared for the software development company. The problem statement may give an overview of the existing system and broad expectations from the new software system.

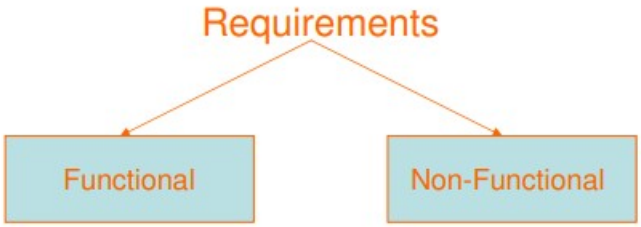
**Q27. Types of Requirements**




Stakeholder: Anyone who should have some direct or indirect influence on the system requirements.

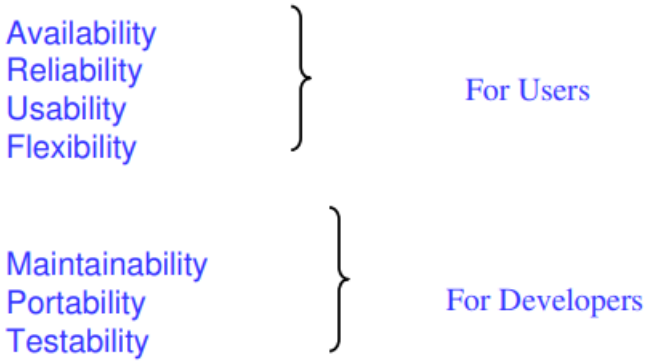
--- User

--- Affected persons




Functional requirements describe what the software has to do. They are often called product features.

Non Functional requirements are mostly quality requirements. That stipulate how well the software does, what it has to do.




User and system requirements

• User requirement are written for the users and include functional and non functional requirement.

• System requirement are derived from user requirement.

• The user system requirements are the parts of software requirement and specification (SRS) document.

Interface Specification

• Important for the customers.

TYPES OF INTERFACES

• Procedural interfaces (also called Application Programming Interfaces (APIs)). • Data structures • Representation of data

## Q28. Feasibility Study

A feasibility study checks if a project is practical and worth pursuing. It looks at key areas to decide if the project can be done successfully within time, budget, and resources.

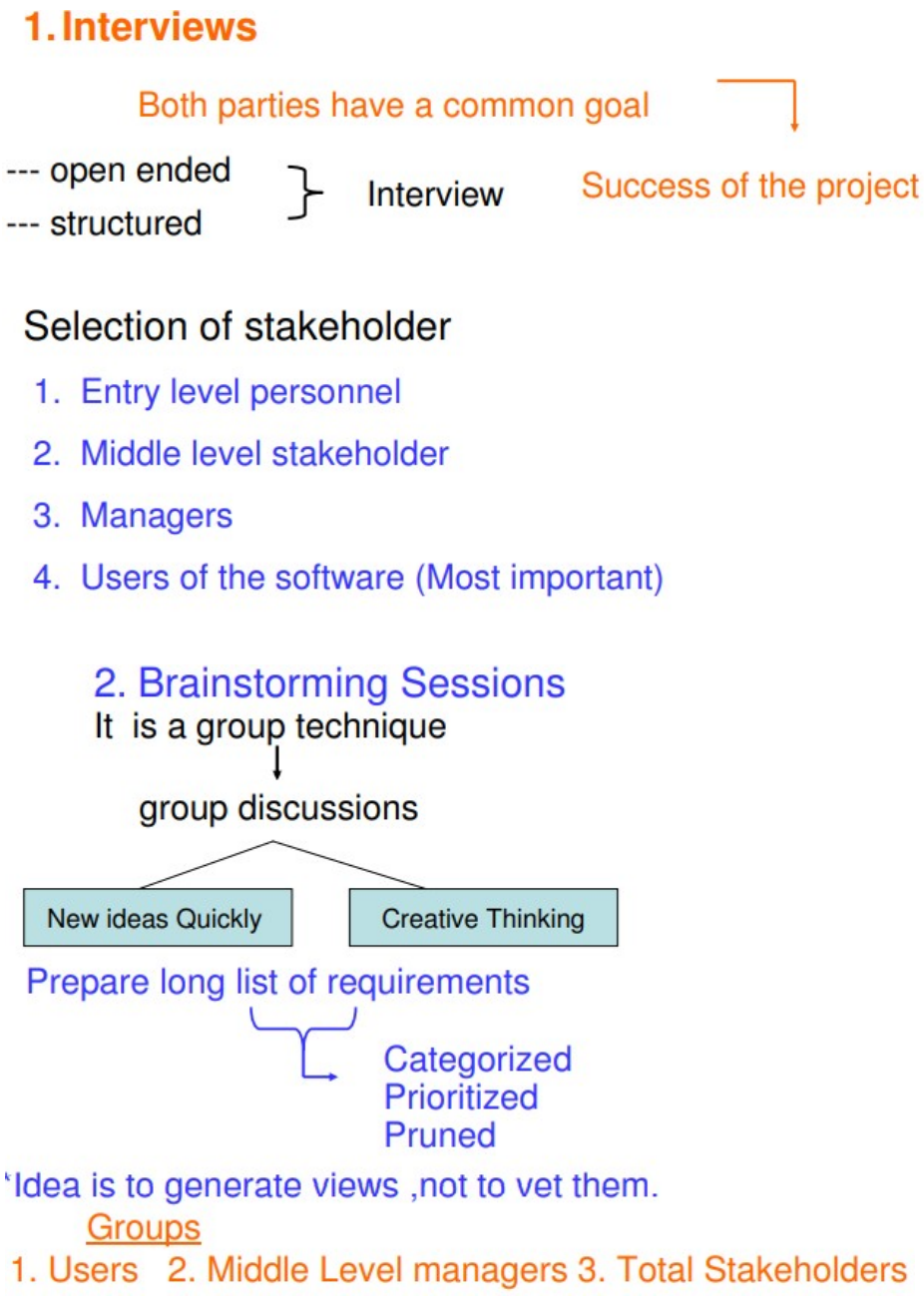
Key parts include:

1. **Technical Feasibility** – Can the needed technology and tools be developed or used?
2. **Economic Feasibility** – Is the project cost-effective, with benefits greater than costs?
3. **Operational Feasibility** – Will the system work well with current processes, and will users accept it?
4. **Legal Feasibility** – Does the project follow laws and regulations?
5. **Schedule Feasibility** – Can the project be completed on time?

This helps in deciding whether to move forward with a project.

Q29. Requirements Elicitation

Requirements elicitation is the process of gathering, discovering, and understanding the needs and expectations of stakeholders for a system or product. Techniques for requirements elicitation:



3. Facilitated Application specification Techniques (FAST)

-- Similar to brainstorming sessions. -- Team oriented approach -- Creation of joint team of customers and developers.

Guidelines
1. Arrange a meeting at a neutral site.
2. Establish rules for participation.
3. Informal agenda to encourage free flow of ideas.
4. Appoint a facilitator.
5. Prepare definition mechanism board, worksheets, wall stickier.
6. Participants should not criticize or debate.

FAST session Preparations Each attendee is asked to make a list of objects that are:

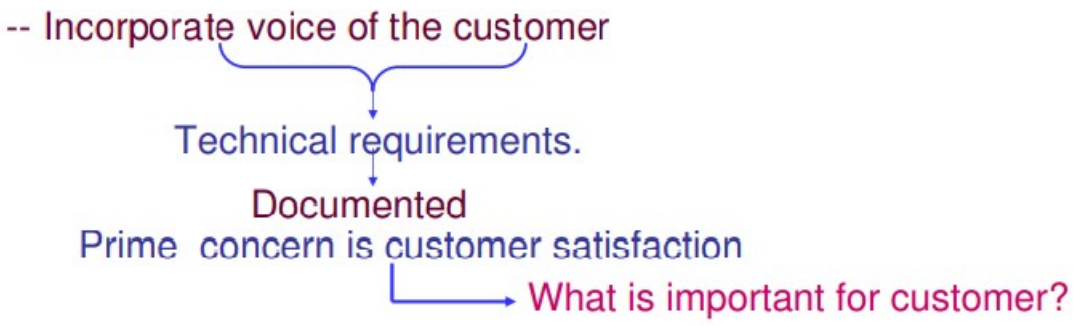
1. Part of environment that surrounds the system.
2. Produced by the system.
3. Used by the system.

A. List of constraints
B. Functions
C. Performance criteria

Activities of FAST session

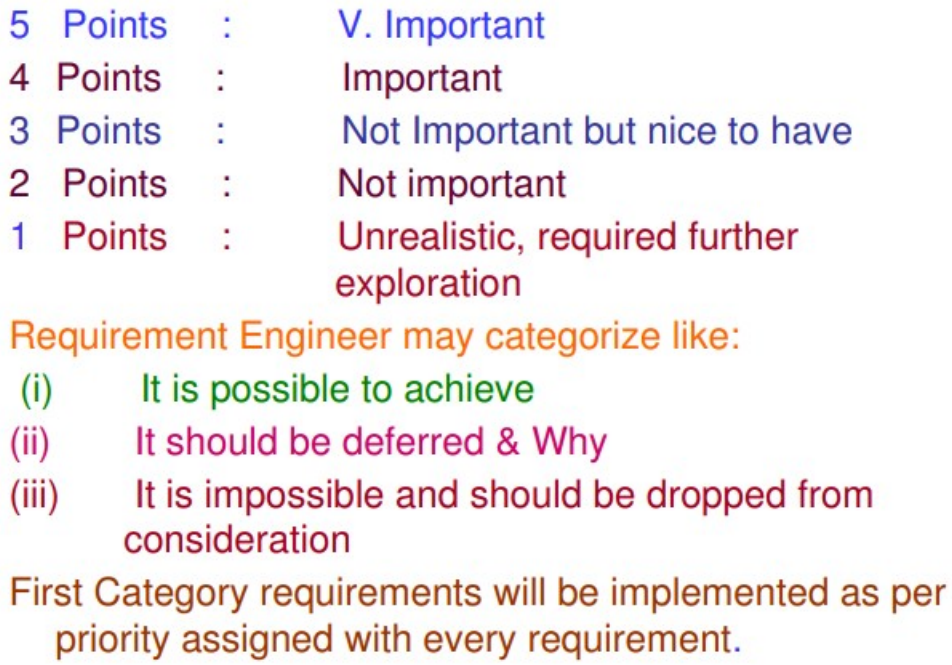
1. Every participant presents his/her list
2. Combine list for each topic
3. Discussion
4. Consensus list
5. Sub teams for mini specifications
6. Presentations of mini-specifications
7. Validation criteria
8. A sub team to draft specifications

4. Quality Function Deployment




-- Normal requirements
-- Expected requirements
-- Exciting requirements

Steps
1. Identify stakeholders
2. List out requirements
3. Degree of importance to each requirement.



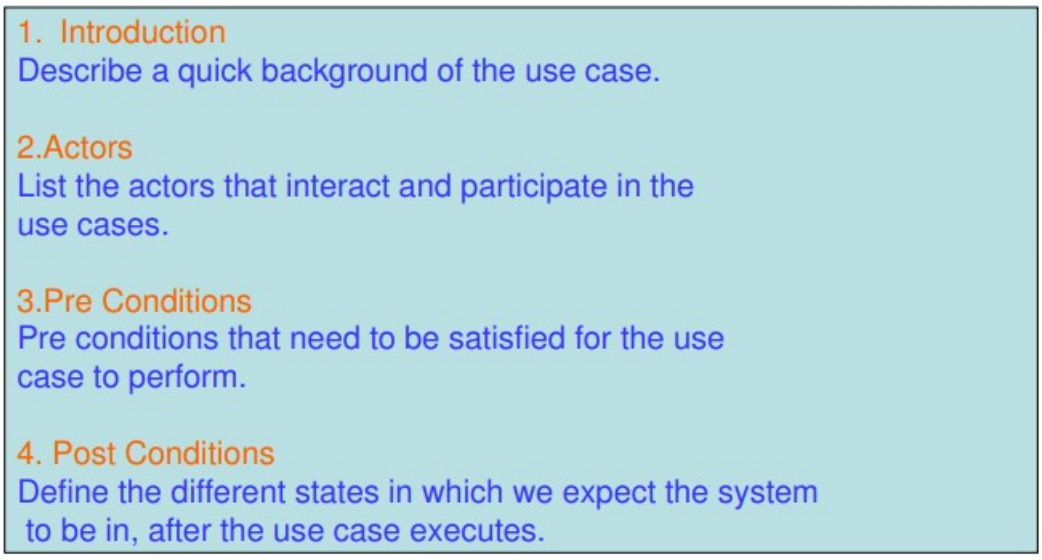

5. The Use Case Approach

The **Use Case Approach**, introduced by Ivar Jacobson, is a method for capturing and modeling functional requirements in software development. It focuses on describing how users (or "actors") interact with a system to achieve specific goals. A use case represents a specific scenario or sequence of steps that the system performs in response to an actor's request. This method is useful in understanding system behavior from the user's perspective.

Use Case captures who (actor) does what (interaction) with the system, for what purpose (goal), without dealing with system internals.

*defines all behavior required of the system, bounding the scope of the system.

Jacobson & others proposed a template for writing Use cases as shown below



5. Flow of events

5.1 Basic Flow
List the primary events that will occur when this use case is executed.

5.2 Alternative Flows
Any Subsidiary events that can occur in the use case should be separately listed. List each such event as an alternative flow.
A use case can have many alternative flows as required.

6.Special Requirements

Business rules should be listed for basic & information flows as special requirements in the use case narration .These rules will also be used for writing test cases. Both success and failures scenarios should be described.

7.Use Case relationships

For Complex systems it is recommended to document the relationships between use cases. Listing the relationships between use cases also provides a mechanism for traceability

Use Case Guidelines

1. Identify all users
2. Create a user profile for each category of users including all roles of the users play that are relevant to the system.
3. Create a use case for each goal, following the use case template maintain the same level of abstraction throughout the use case. Steps in higher level use cases may be treated as goals for lower level (i.e. more detailed), sub-use cases.
4. Structure the use case
5. Review and validate with users.

## Use case Diagrams

-- represents what happens when actor interacts with a system.
-- captures functional aspect of the system.

Actor

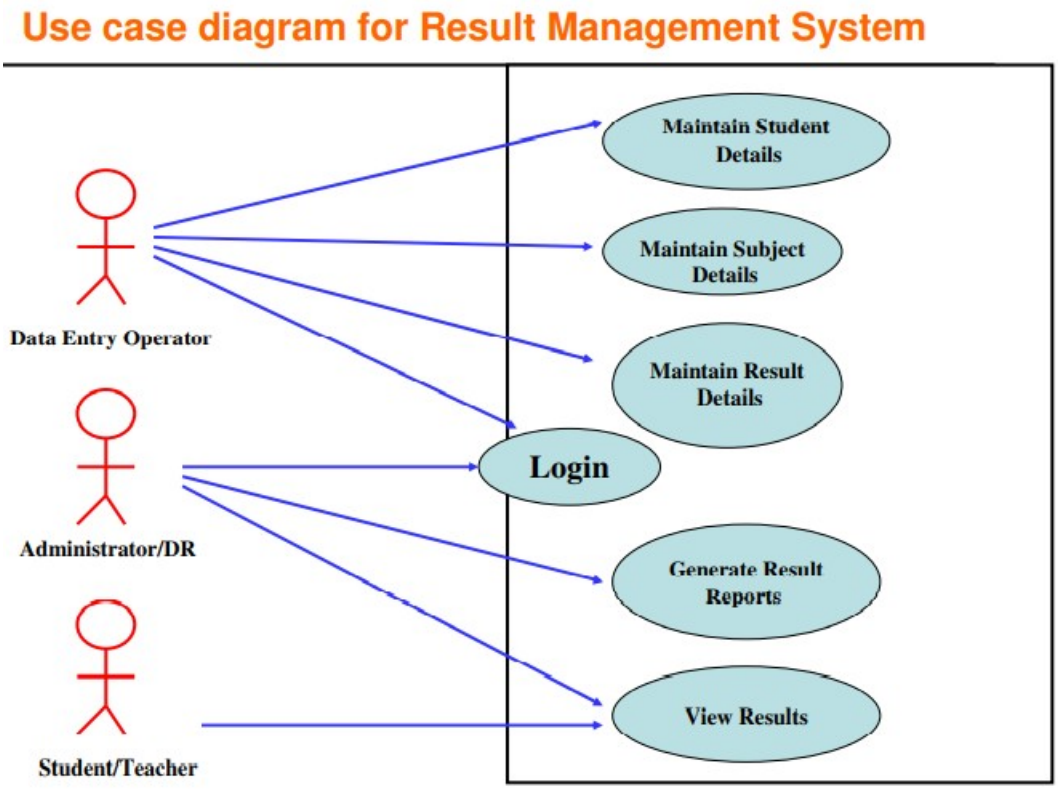
Use Case

Relationship between actors and use case and/or between the use cases.

-- Actors appear outside the rectangle.

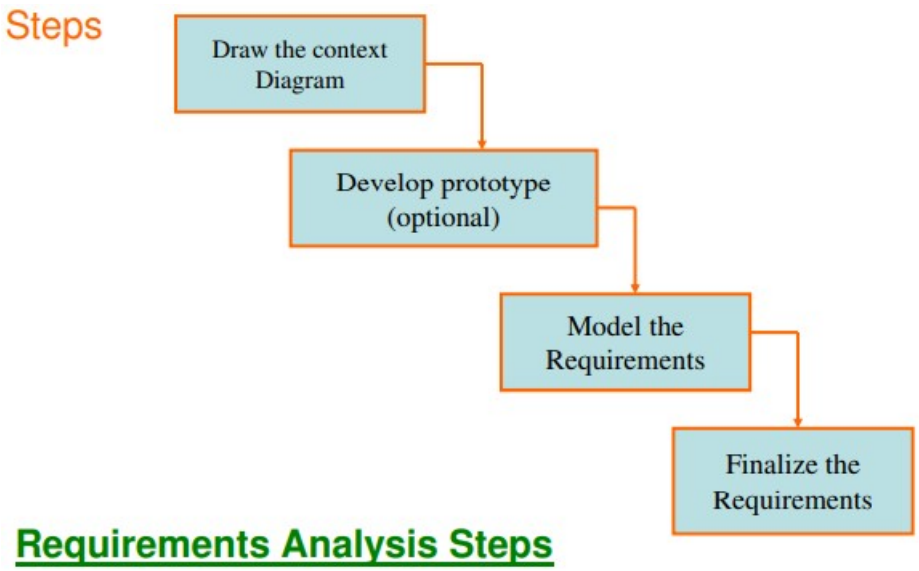
--Use cases within rectangle providing functionality.

--Relationship association is a solid line between actor & use cases.

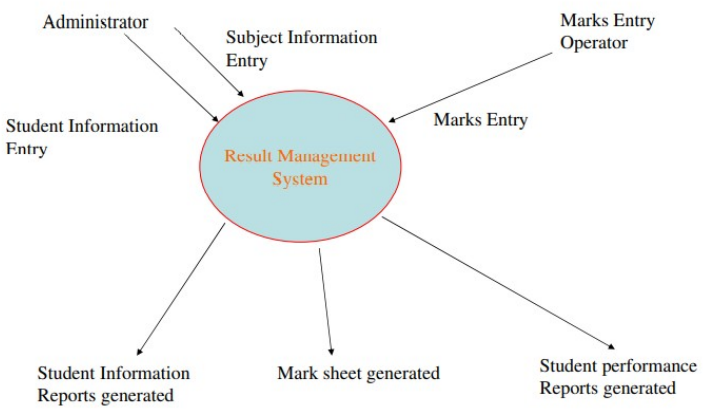



**Q30. Requirements Analysis.**

We analyze, refine and scrutinize requirements to make consistent & unambiguous requirements.




Ex:



Q31. Data Flow Diagrams

## Data Flow Diagrams

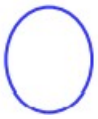
DFD show the flow of data through the system.

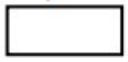
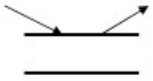
--All names should be unique

-- It is not a flow chart

-- Suppress logical decisions

-- Defer error conditions & handling until the end of the analysis

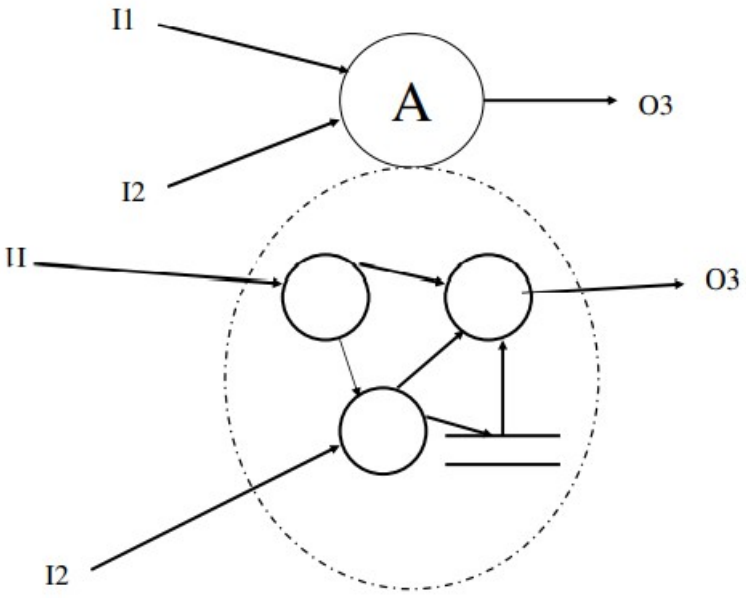
| Symbol | Name | Function |
|---|---|---|
|  | Data Flow | Connect process |
|  | Process | Perform some transformation of its input data to yield output data. |

| Symbol | Name | Function |
|---|---|---|
|  | Source or sink | A source of system inputs or sink of system outputs |
|  | Data Store | A repository of data, the arrowhead indicate net input and net outputs |

**Q32. Leveling In DFD**

Leveling DFD represent a system or software at any level of abstraction.

A level 0 DFD is called fundamental system model or context model represents entire software element as a single bubble with input and output data indicating by incoming & outgoing arrows.



## Q33. Data Dictionaries

DFD ⟶ DD

Data Dictionaries are simply repositories to store information about all data items defined in DFD.

Includes :

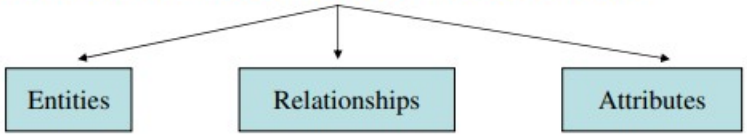
- Name of data item
- Aliases (other names for items)
- Description/Purpose
- Related data items
- Range of values
- Data flows
- Data structure definition

| Notation | Meaning |
|---|---|
| x= a+b | x consists of data element a & b |
| x={a/b} | x consists of either a or b |
| x=(a) | x consists of an optional data element a |
| x= y{a} | x consists of y or more occurrences |
| x={a}z | x consists of z or fewer occurrences of a |
| x=y{a}z | x consists of between y & z occurrences of a{ |

## Q34. ER Diagram.

### Entity-Relationship Diagrams

It is a detailed logical representation of data for an organization and uses three main constructs.




### Entities

Fundamental thing about which data may be maintained. Each entity has its own identity.

### Relationships

A relationship is a reason for associating two entity types.
Binary relationships ⟶ involve two entity types

A CUSTOMER is insured by a POLICY. A POLICY CLAIM is made against a POLICY.

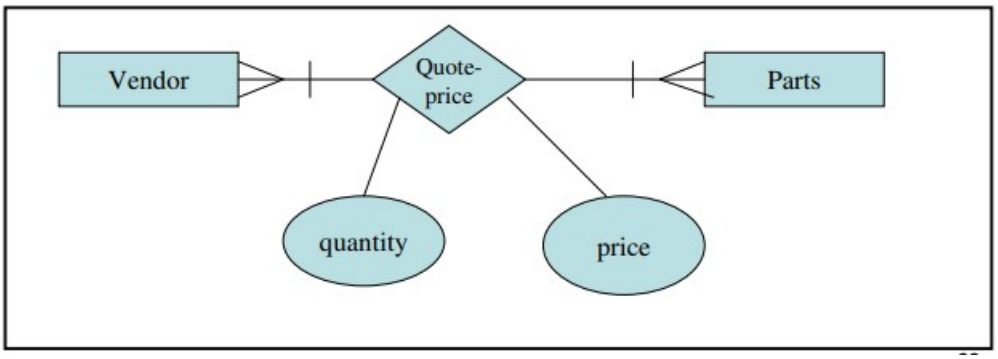
### Attributes

Each entity type has a set of attributes associated with it.

An attribute is a property or characteristic of an entity that is of interest to organization.

 Attribute

Vendors quote prices for several parts along with quantity of parts.
Draw an E-R diagram.




## Q35. Structured requirements definition (SRD)

**Structured Requirements Definition (SRD)** outlines steps to document system requirements clearly.

1. **User-Level DFD**: Create individual DFDs for each user, showing inputs (e.g., data provided by users) and outputs (e.g., system responses).
2. **Combined User-Level DFD**: Combine individual DFDs to show how all users interact with the system and each other.
3. **Application-Level DFD**: Focus on internal system processes, data stores, and data flows, showing how the system processes inputs and outputs.
4. **Application-Level Functions**: Define specific system functions based on the processes identified in the application-level DFD (e.g., validate login, process transactions).

## Q35. Requirements Documentation

This is the way of representing requirements in a consistent format.

**Requirements Documentation** is the process of recording and organizing all the system requirements gathered during the development process. It serves as a reference for stakeholders and the development team to ensure everyone has a clear understanding of what the system should do.

- **Functional Requirements**: Specify what the system must do (e.g., user login, data processing).
- **Non-Functional Requirements**: Define system qualities such as performance, security, and usability.
- **Business Requirements**: High-level needs and objectives from the business perspective.
- **Technical Requirements**: Specific technical details, such as platform and technology constraints.
- **Assumptions and Constraints**: Any assumptions made and limitations on the project.

Q36. Nature of SRS

- Correct

An SRS is correct if and only if every requirement stated therein is one that the software shall meet.

- Unambiguous

An SRS is unambiguous if and only if, every requirement stated therein has only one interpretation.

- Complete

An SRS is complete if and only if, it includes the following elements

(i) All significant requirements, whether related to functionality, performance, design constraints, attributes or external interfaces.

(ii) Responses to both valid & invalid inputs.

(iii) Full Label and references to all figures, tables and diagrams in the SRS and definition of all terms and units of measure.

- Consistent

An SRS is consistent if and only if, no subset of individual requirements described in it conflict.

- Ranked for importance and/or Stability

If an identifier is attached to every requirement to indicate either the importance or stability of that particular requirement

- Verifiable

An SRS is verifiable, if and only if, every requirement stated therein is verifiable.

- Modifiable

An SRS is modifiable, if and only if, its structure and style are such that any changes to the requirements can be made easily, completely, and consistently while retaining structure and style.

- Traceable

An SRS is traceable, if the origin of each of the requirements is clear and if it facilitates the referencing of each requirement in future development or enhancement documentation.

All of these are characteristics of a good SRS

Q37. Organization of the SRS

IEEE has published guidelines and standards to organize an SRS.
First two sections are same. The specific tailoring occurs in section-3.

1. Introduction
    - 1.1 Purpose
    - 1.2 Scope
    - 1.3 Definition, Acronyms and abbreviations
    - 1.4 References
    - 1.5 Overview

2. The Overall Description

- 2.1 Product Perspective
  - 2.1.1 System Interfaces
  - 2.1.2 Interfaces
  - 2.1.3 Hardware Interfaces
  - 2.1.4 Software Interfaces
  - 2.1.5 Communication Interfaces
  - 2.1.6 Memory Constraints
  - 2.1.7 Operations
  - 2.1.8 Site Adaptation Requirements
- 2.2 Product Functions
- 2.3 User Characteristics
- 2.4 Constraints
- 2.5 Assumptions for dependencies
- 2.6 Apportioning of requirements

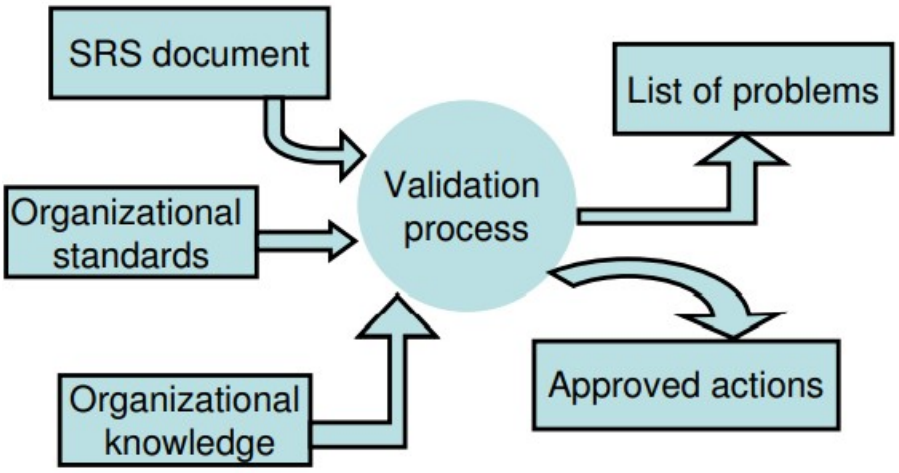
3. Specific Requirements

- 3.1 External Interfaces
- 3.2 Functions
- 3.3 Performance requirements
- 3.4 Logical database requirements
- 3.5 Design Constraints
- 3.6 Software System attributes
- 3.7 Organization of specific requirements
- 3.8 Additional Comments.

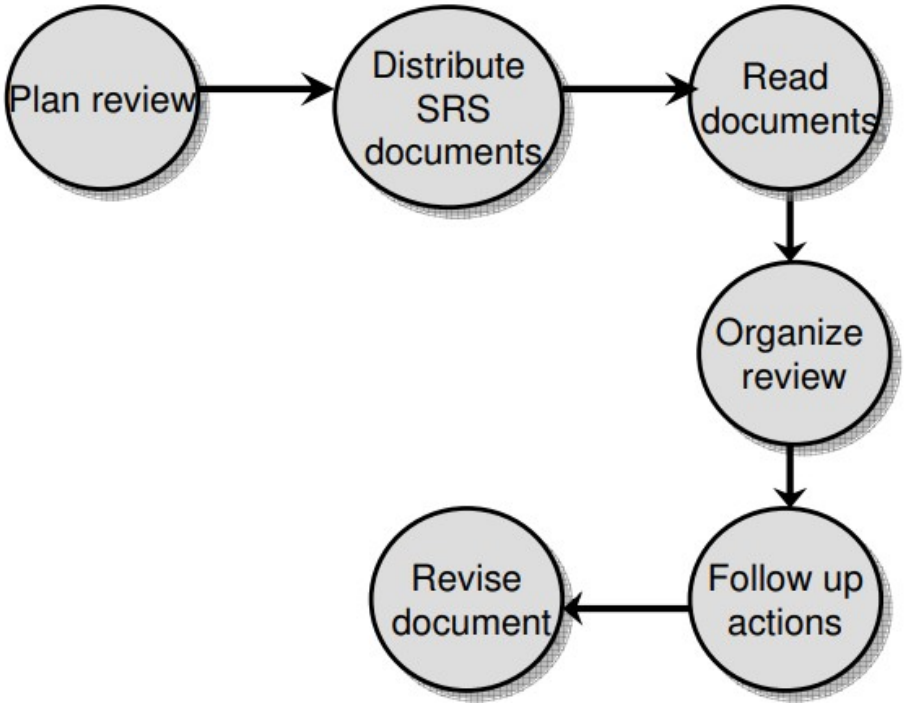
Q38. Requirements Validation

Check the document for:

- ✓ Completeness & consistency
- ✓ Conformance to standards
- ✓ Requirements conflicts
- ✓ Technical errors
- ✓ Ambiguous requirements




Q39. Requirements Review Process




Q40. Review Checklists

- ✓ Redundancy
- ✓ Completeness
- ✓ Ambiguity
- ✓ Consistency
- ✓ Organization
- ✓ Conformance
- ✓ Traceability

Q41. Requirements Management

**Requirements Management** is the process of handling and controlling requirements throughout the software development lifecycle. It ensures that all requirements are properly documented, tracked, and maintained, adapting to changes as needed. This process is crucial for project success and stakeholder satisfaction.

Key components of Requirements Management include:

1. **Requirements Identification**: Clearly defining and documenting all requirements, including functional and non-functional aspects.
2. **Requirements Traceability**: Maintaining a clear link between requirements and other project artifacts, such as design, implementation, and testing. This helps ensure that all requirements are addressed and facilitates impact analysis when changes occur.
3. **Change Control**: Establishing a formal process for managing changes to requirements. This includes evaluating the impact of changes, obtaining necessary approvals, and updating documentation accordingly.
4. **Requirements Communication**: Ensuring that all stakeholders, including developers, testers, and clients, understand the requirements and any changes made. Effective communication helps minimize misunderstandings and misalignments.
5. **Requirements Validation and Verification**: Regularly reviewing requirements to ensure they are complete, accurate, and feasible. This involves validating requirements with stakeholders and verifying that the system meets these requirements during development and testing.
6. **Requirements Prioritization**: Assessing and ranking requirements based on their importance and urgency, which helps in decision-making regarding resource allocation and scope management.

Software Engineering
Unit 2

## Q1. Software Project Planning

After the finalization of SRS, we would like to estimate size, cost and development time of the project.

**Software Project Planning** is a crucial phase in the software development lifecycle that involves defining the scope, objectives, resources, schedule, and overall strategy for a software project.

In order to conduct a successful software project, we must understand:

- Scope of work to be done
- Software Project Planning
- The risk to be incurred
- The resources required
- The task to be accomplished
- The cost to be expended
- The schedule to be followed

Software planning begins before technical work starts, continues as the software evolves from concept to reality, and culminates only when the software is retired.

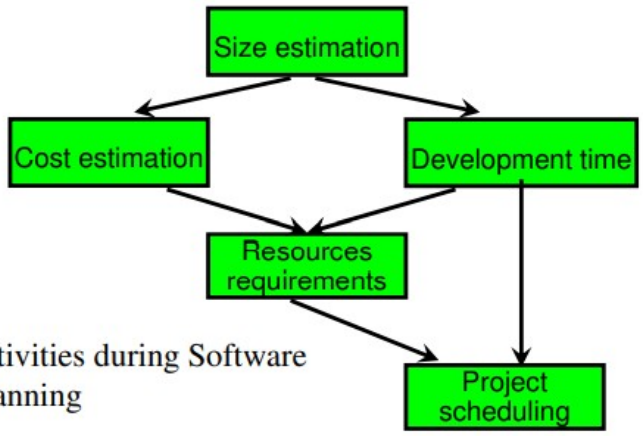



Fig. 1: Activities during Software Project Planning



## Q2. Size Estimate.

**Size estimation** in software engineering refers to the process of predicting the size of a software project, typically measured in terms of lines of code (LOC), function points (FP), or story points.

## Q3. Lines of code.

if LOC is simply a count of the number of lines then figure shown below contains 17 LOC. Rn Comment

“A line of code is any line of program text that is not a comment or blank line, regardless of the number of statements or fragments of statements on the line.

```
1.  int. sort (int x[ ], int n)
2.  {
3.  int i, j, save, im1;
4.  /*This function sorts array x in ascending order */
5.  If (n<2) return 1;
6.  for (i=2; i<=n; i++)
7.  {
8.  im1=i-1;
9.  for (j=1; j<=im; j++)
10. if (x[i] < x[j])
11. {
12. Save = x[i];
13. x[i] = x[j];
14. x[j] = save;
15. }
16. }
17. return 0;
18. }
```

This specifically includes all lines containing program header, declaration, and executable and non-executable statements".

## Q4. Function Point Analysis

Alan Albrecht while working for IBM, recognized the problem in size measurement in the 1970s, and developed a technique (which he called Function Point Analysis), which appeared to be a solution to the size measurement problem.

The principle of Albrecht's function point analysis (FPA) is that a system is decomposed into functional units.

The FPA functional units are shown in figure given below:

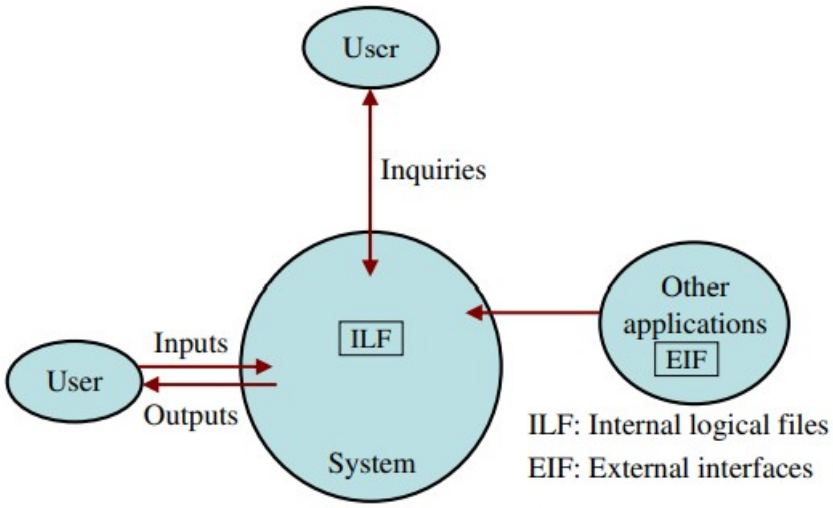



Fig. 3: FPAs functional units System

Function Point Analysis (FPA) measures software size using five functional units:

- **External Inputs (EI)**: Data or control inputs that the system processes (e.g., login forms).
- **External Outputs (EO)**: Processed data or reports provided to users (e.g., reports, invoices).
- **External Inquiries (EQ)**: Data retrieval requests without altering data (e.g., search queries).
- **Internal Logical Files (ILF)**: Internal data stored and maintained by the system (e.g., customer database).
- **External Interface Files (EIF)**: External data files used but not maintained by the system (e.g., shared files).

Special features Software Project Planning

Function point approach is independent of the language, tools, or methodologies used for implementation; i.e. they do not take into consideration programming languages, data base management systems, processing hardware or any other data base technology.

Function points can be estimated from requirement specification or design specification, thus making it possible to estimate development efforts in early phases of development.

Function points are directly linked to the statement of requirements; any change of requirements can easily be followed by a re-estimate.

Software Project Planning  Function points are based on the system user's external view of the system, non-technical users of the software system have a better understanding of what function points are measuring.

## Counting function points

| Functional Units | Weighting factors | | |
|---|---|---|---|
| | Low | Average | High |
| External Inputs (EI) | 3 | 4 | 6 |
| External Output (EO) | 4 | 5 | 7 |
| External Inquiries (EQ) | 3 | 4 | 6 |
| External logical files (ILF) | 7 | 10 | 15 |
| External Interface files (EIF) | 5 | 7 | 10 |

Table 2: UFP calculation table

| Functional Units | Count Complexity | | Complexity Totals | Functional Unit Totals |
|---|---|---|---|---|
| External Inputs (EIs) | ☐ | Low x 3 = | ☐ | |
| | ☐ | Average x 4 = | ☐ | |
| | ☐ | High x 6 = | ☐ | ☐ |
| External Outputs (EOs) | ☐ | Low x 4 = | ☐ | |
| | ☐ | Average x 5 = | ☐ | |
| | ☐ | High x 7 = | ☐ | ☐ |
| External Inquiries (EQs) | ☐ | Low x 3 = | ☐ | |
| | ☐ | Average x 4 = | ☐ | |
| | ☐ | High x 6 = | ☐ | ☐ |
| External logical Files (ILFs) | ☐ | Low x 7 = | ☐ | |
| | ☐ | Average x 10 = | ☐ | |
| | ☐ | High x 15 = | ☐ | ☐ |
| External Interface Files (EIFs) | ☐ | Low x 5 = | ☐ | |
| | ☐ | Average x 7 = | ☐ | |
| | ☐ | High x 10 = | ☐ | ☐ |
| Total Unadjusted Function Point Count | | | | ☐ |

The weighting factors are identified for all functional units and multiplied with the functional units accordingly. The procedure for the calculation of Unadjusted Function Point (UFP) is given in table shown above.

**Q5. Unadjusted Function Point**

The procedure for the calculation of UFP in mathematical form is given below:

$$UFP = \sum_{i=1}^{5} \sum_{J=1}^{3} Z_{ij} w_{ij}$$

Where i indicate the row and j indicates the column of Table 1

$W_{ij}$ : It is the entry of the $i^{th}$ row and $j^{th}$ column of the table 1

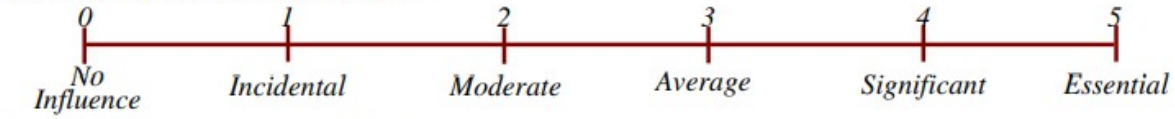
Zij : It is the count of the number of functional units of Type *i* that have been classified as having the complexity corresponding to column *j*.

**Q6 Complexity Adjustment Factor**

CAF stands for **Complexity Adjustment Factor**, and it's used to adjust the **Unadjusted Function Points (UFP)** based on the complexity of the system. It's calculated as part of the Function Point Analysis (FPA) to reflect the overall influence of various system characteristics.

Table 3 : Computing function points.

Rate each factor on a scale of 0 to 5.




Number of factors considered ( $F_i$ )

1. Does the system require reliable backup and recovery ?
2. Is data communication required ?
3. Are there distributed processing functions ?
4. Is performance critical ?
5. Will the system run in an existing heavily utilized operational environment ?
6. Does the system require on line data entry ?
7. Does the on line data entry require the input transaction to be built over multiple screens or operations ?
8. Are the master files updated on line ?
9. Is the inputs, outputs, files, or inquiries complex ?
10. Is the internal processing complex ?
11. Is the code designed to be reusable ?
12. Are conversion and installation included in the design ?
13. Is the system designed for multiple installations in different organizations ?
14. Is the application designed to facilitate change and ease of use by the user ?

**Q7. Consider a software project with the following parameters:**

1. **External Inputs (EI)**:
   - 10 with low complexity
   - 15 with average complexity
   - 17 with high complexity
2. **External Outputs (EO)**:
   - 6 with low complexity
   - 13 with high complexity
3. **External Inquiries (EQ)**:
   - 3 with low complexity
   - 4 with average complexity
   - 2 with high complexity

4. **Internal Logical Files (ILF)**:
    - 2 with average complexity
    - 1 with high complexity
5. **External Interface Files (EIF)**:
    - 9 with low complexity

In addition, the system has the following complexity adjustment factors:

1. Significant data communication is required.
2. Performance is very critical.
3. The code is designed to be moderately reusable.
4. The system is **not** designed for multiple installations across different organizations.

All other complexity adjustment factors are treated as average.

**Question**: Compute the Function Points (FP) for this project.

**Solution:** Unadjusted function points may be counted using table 2

| Functional Units | Count | Complexity | | Complexity Totals | Functional Unit Totals |
|---|---|---|---|---|---|
| External Inputs (EIs) | 10 | Low x 3 | = | 30 | |
| | 15 | Average x 4 | = | 60 | |
| | 17 | High x 6 | = | 102 | 192 |
| External Outputs (EOs) | 6 | Low x 4 | = | 24 | |
| | 0 | Average x 5 | = | 0 | |
| | 13 | High x 7 | = | 91 | 115 |
| External Inquiries (EQs) | 3 | Low x 3 | = | 9 | |
| | 4 | Average x 4 | = | 16 | |
| | 2 | High x 6 | = | 12 | 37 |
| External logical Files (ILFs) | 0 | Low x 7 | = | 0 | |
| | 2 | Average x 10 | = | 20 | |
| | 1 | High x 15 | = | 15 | 35 |
| External Interface Files (EIFs) | 9 | Low x 5 | = | 45 | |
| | 0 | Average x 7 | = | 0 | |
| | 0 | High x 10 | = | 0 | 45 |
| Total Unadjusted Function Point Count | | | | | 424 |

$$\sum_{i=1}^{14} F_i = 3+4+3+5+3+3+3+3+3+3+2+3+0+3=41$$

$$\begin{aligned} CAF &= (0.65 + 0.01 \times \Sigma F_i) \\ &= (0.65 + 0.01 \times 41) \\ &= 1.06 \\ FP &= UFP \times CAF \\ &= 424 \times 1.06 \\ &= 449.44 \end{aligned}$$

Hence $\boxed{FP = 449}$

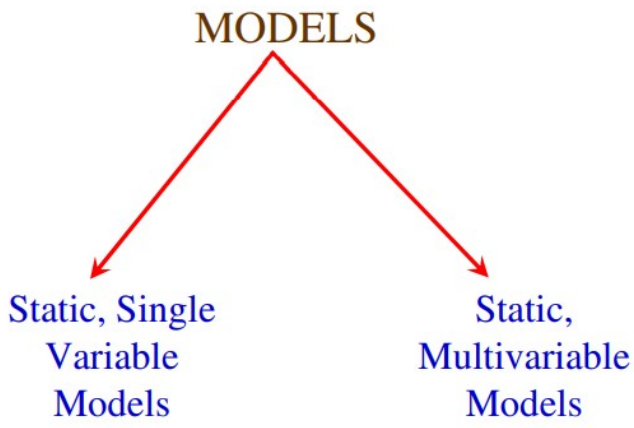
**Q8. Cost Estimation**

A number of estimation techniques have been developed and are having following attributes in common :

- Project scope must be established in advance
- Software metrics are used as a basis from which estimates are made
- The project is broken into small pieces which are estimated individually

To achieve reliable cost and schedule estimates, a number of options arise:

- Delay estimation until late in project
- Use simple decomposition techniques to generate project cost and schedule estimates
- Develop empirical models for estimation
- Acquire one or more automated estimation tools




## Static, Single Variable Models

Methods using this model use an equation to estimate the desired values such as cost, time, effort, etc. They all depend on the same variable used as predictor (say, size). An example of the most common equations is :

$$C = a L^b \quad \text{(i)}$$

C is the cost, L is the size and a,b are constants

$E = 1.4 L^{0.93}$

$DOC = 30.4 L^{0.90}$

$D = 4.6 L^{0.26}$

Effort (E in Person-months), documentation (DOC, in number of pages) and duration (D, in months) are calculated from the number of lines of code (L, in thousands of lines) used as a predictor.

## Static, Multivariable Models

These models are often based on equation (i), they actually depend on several variables representing various aspects of the software development environment, for example method used, user participation, customer oriented changes, memory constraints, etc.

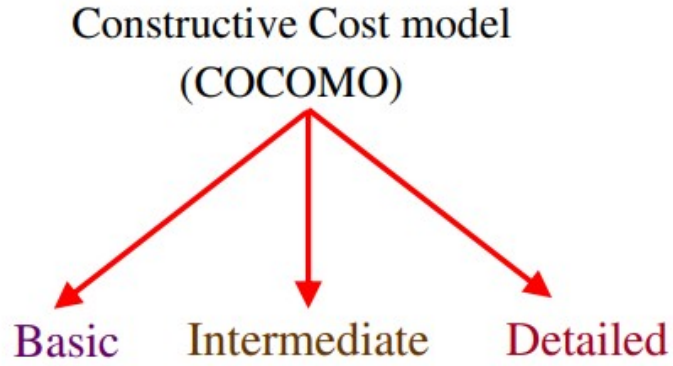
$$E = 5.2\,L^{0.91}$$

$$D = 4.1\,L^{0.36}$$

The productivity index uses 29 variables which are found to be highly correlated to productivity as follows:

$$I = \sum_{i=1}^{29} W_i X_i$$

**Q9. The Constructive Cost model (COCOMO)**

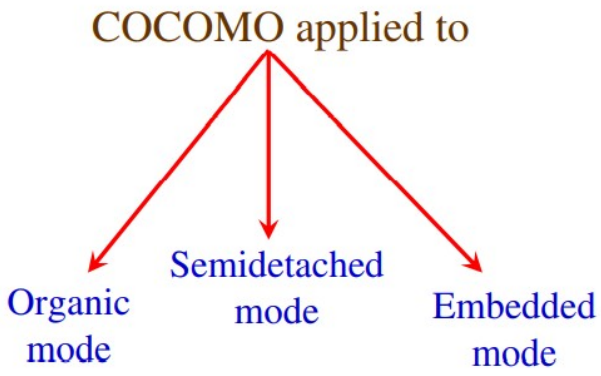



Model proposed by
B. W. Boehm's
through his book
Software Engineering Economics in 1981

**The Constructive Cost Model (COCOMO)** is a popular cost estimation model used in software engineering to predict project costs based on the size of the software and various project characteristics. Developed by Barry Boehm in 1981, it uses mathematical formulas to estimate the effort, time, and cost required to complete a software project.

**Types of COCOMO Models:**

1. **Basic COCOMO**: Provides a rough estimate of effort based on the size of the software in terms of **Lines of Code (LOC)**. It is the simplest form of COCOMO and classifies projects into three categories:




- **Organic**: Small teams working on well-understood projects.
- **Semi-detached**: Intermediate-sized projects with mixed teams and moderately complex requirements.
- **Embedded**: Complex projects involving tight hardware or software constraints.

## Basic Model

Basic COCOMO model takes the form

$$E = a_b (KLOC)^{b_b}$$

$$D = c_b (E)^{d_b}$$

where E is effort applied in Person-Months, and D is the development time in months. The coefficients $a_b$, $b_b$, $c_b$ and $d_b$ are given in table 4 (a).

| Software Project | $a_b$ | $b_b$ | $c_b$ | $d_b$ |
|---|---|---|---|---|
| Organic | 2.4 | 1.05 | 2.5 | 0.38 |
| Semidetached | 3.0 | 1.12 | 2.5 | 0.35 |
| Embedded | 3.6 | 1.20 | 2.5 | 0.32 |

**Table 4(a):** Basic COCOMO coefficients

When effort and development time are known, the average staff size to complete the project may be calculated as:

$$\text{Average staff size } (SS) = \frac{E}{D} Persons$$

When project size is known, the productivity level may be calculated as:

$$\text{Productivity } (P) = \frac{KLOC}{E} KLOC / PM$$

2. **Intermediate COCOMO**: Includes additional factors called **cost drivers**, which adjust the basic estimate based on factors like product complexity, experience, and development tools. It provides a more accurate estimate than the Basic COCOMO.

**Intermediate COCOMO cost drivers**:

**1. Product Attributes**

- Required Software Reliability (RELY)
- Database Size (DATA)
- Product Complexity (CPLX)

**2. Hardware Attributes**

- Execution Time Constraint (TIME)
- Main Storage Constraint (STOR)
- Virtual Machine Volatility (VIRT)
- Computer Turnaround Time (TURN)

**3. Personnel Attributes**

- Analyst Capability (ACAP)
- Programmer Capability (PCAP)
- Personnel Continuity (PCON)
- Application Experience (AEXP)
- Programming Language Experience (LEXP)

**4. Project Attributes**

- Use of Modern Programming Practices (MODP)
- Use of Software Tools (TOOL)
- Required Development Schedule (SCED)

| Cost Drivers | RATINGS | | | | | |
|---|---|---|---|---|---|---|
| | Very low | Low | Nominal | High | Very high | Extra high |
| **Product Attributes** | | | | | | |
| RELY | 0.75 | 0.88 | 1.00 | 1.15 | 1.40 | -- |
| DATA | -- | 0.94 | 1.00 | 1.08 | 1.16 | -- |
| CPLX | 0.70 | 0.85 | 1.00 | 1.15 | 1.30 | 1.65 |
| **Computer Attributes** | | | | | | |
| TIME | -- | -- | 1.00 | 1.11 | 1.30 | 1.66 |
| STOR | -- | -- | 1.00 | 1.06 | 1.21 | 1.56 |
| VIRT | -- | 0.87 | 1.00 | 1.15 | 1.30 | -- |
| TURN | -- | 0.87 | 1.00 | 1.07 | 1.15 | -- |

| Cost Drivers | RATINGS | | | | | |
|---|---|---|---|---|---|---|
| | Very low | Low | Nominal | High | Very high | Extra high |
| **Personnel Attributes** | | | | | | |
| ACAP | 1.46 | 1.19 | 1.00 | 0.86 | 0.71 | -- |
| AEXP | 1.29 | 1.13 | 1.00 | 0.91 | 0.82 | -- |
| PCAP | 1.42 | 1.17 | 1.00 | 0.86 | 0.70 | -- |
| VEXP | 1.21 | 1.10 | 1.00 | 0.90 | -- | -- |
| LEXP | 1.14 | 1.07 | 1.00 | 0.95 | -- | -- |
| **Project Attributes** | | | | | | |
| MODP | 1.24 | 1.10 | 1.00 | 0.91 | 0.82 | -- |
| TOOL | 1.24 | 1.10 | 1.00 | 0.91 | 0.83 | -- |
| SCED | 1.23 | 1.08 | 1.00 | 1.04 | 1.10 | -- |

## Intermediate COCOMO equations

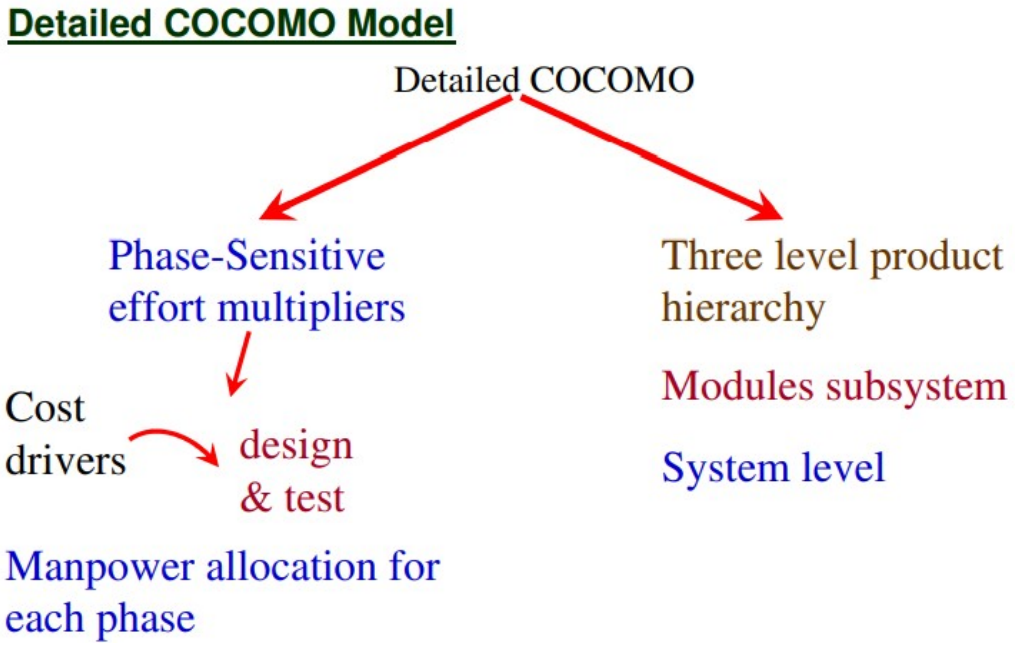
$$E = a_i (KLOC)^{b_i} * EAF$$

$$D = c_i (E)^{d_i}$$

| Project | $a_i$ | $b_i$ | $c_i$ | $d_i$ |
|---|---|---|---|---|
| Organic | 3.2 | 1.05 | 2.5 | 0.38 |
| Semidetached | 3.0 | 1.12 | 2.5 | 0.35 |
| Embedded | 2.8 | 1.20 | 2.5 | 0.32 |

**Table 6:** Coefficients for intermediate COCOMO

3. **Detailed COCOMO**: Adds further refinement by breaking the project into smaller components and calculating costs for each component individually. It provides the most detailed and accurate estimates.




## COCOMO-II

The following categories of applications / projects are identified by COCOMO-II and are shown in fig. 4 shown below:

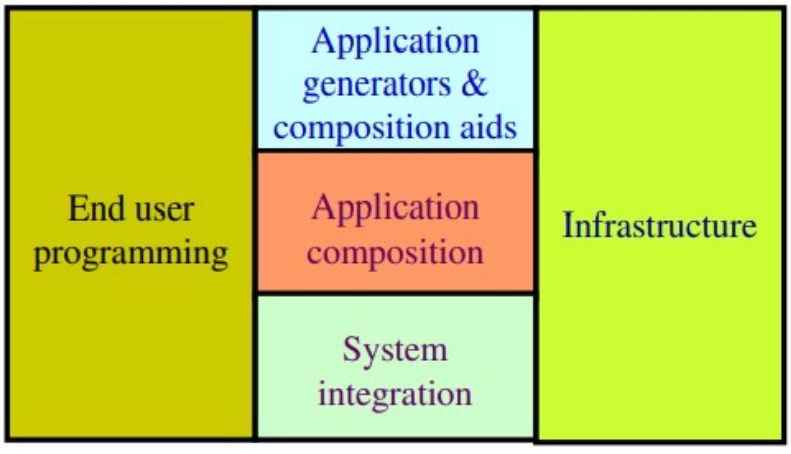
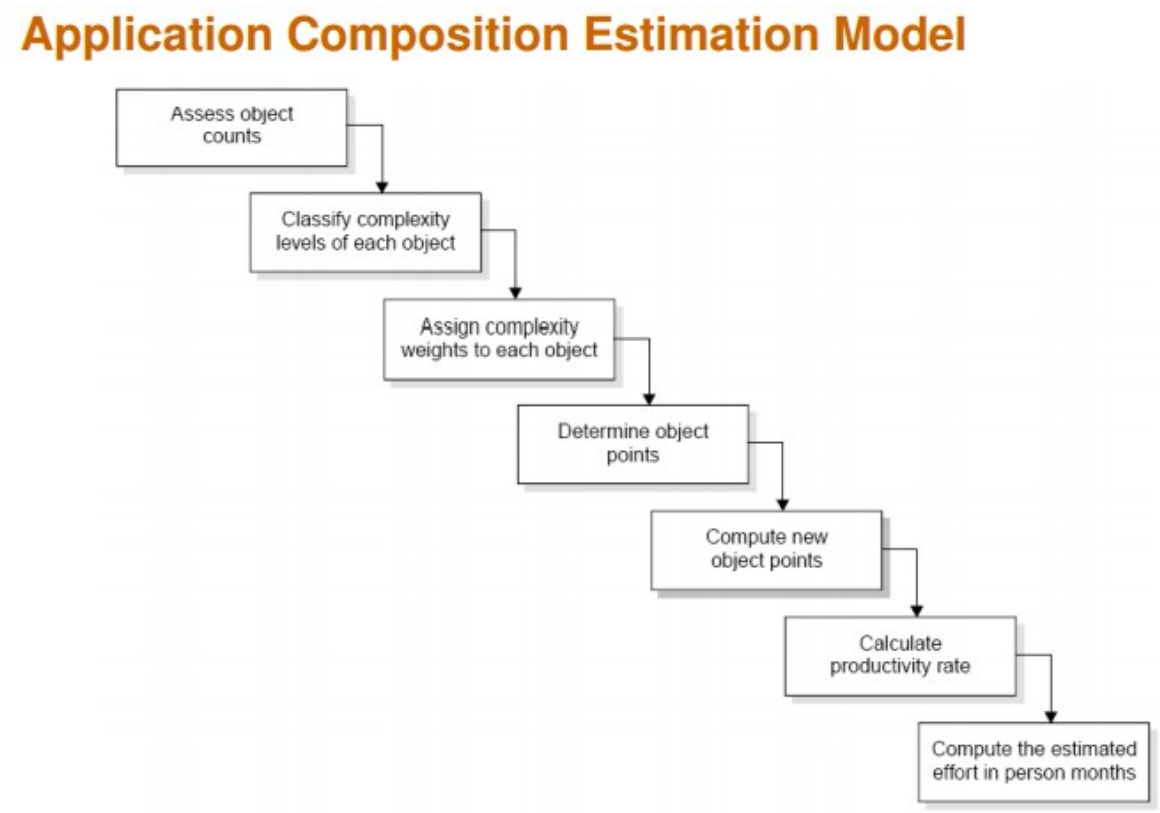



**Fig. 4 :** Categories of applications / projects

**Q10. ACEM**




| No | | types of projects | |
|---|---|---|---|
| Stage I | Application composition estimation model | Application composition | In addition to application composition type of projects, this model is also used for prototyping (if any) stage of application generators, infrastructure & system integration. |
| Stage II | Early design estimation model | Application generators, infrastructure & system integration | Used in early design stage of a project, when less is known about the project. |
| Stage III | Post architecture estimation model | Application generators, infrastructure & system integration | Used after the completion of the detailed architecture of the project. |

**Q11. Putnam Resource Allocation Model**

The **Putnam Resource Allocation Model**, also known as the **Putnam Model** or **SLIM (Software Life Cycle Management)**, is a software cost estimation model developed by Lawrence H. Putnam in the 1970s. It focuses on the relationship between the effort, time, and productivity of software

development projects. The model is based on the idea that software development follows a predictable curve, where the allocation of resources over time significantly influences project success.

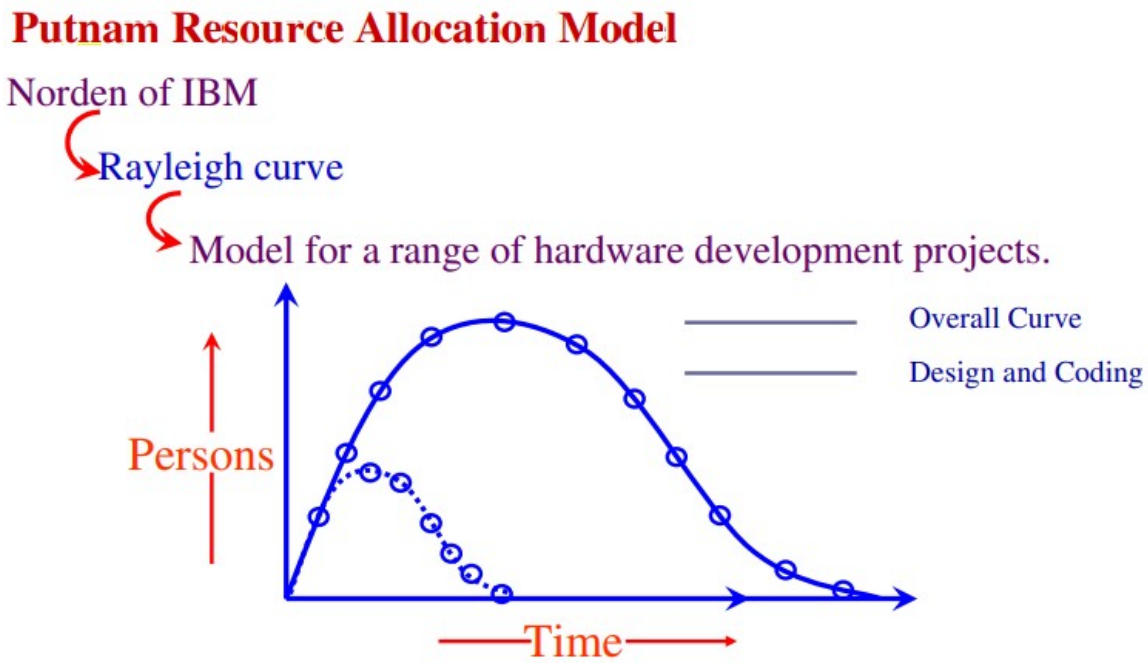



**Fig.6:** The Rayleigh manpower loading curve

**Resource Allocation**: The model emphasizes the importance of efficiently allocating resources (i.e., personnel) over the project's lifecycle. It recognizes that the timing and amount of resources directly affect project duration and quality.

- **Effort and Time**: The model establishes a relationship between the total effort (in person-months) and the development time (in months). As development time increases, the effort required decreases and vice versa.
- **Productivity**: Productivity is defined as the amount of software produced per unit of effort. The Putnam Model uses the concept of **"productivity curves,"** which illustrate how productivity changes over time based on resource allocation.
- **Quality vs. Time**: The model suggests that higher quality can be achieved by extending development time and adequately allocating resources, allowing for more thorough testing and refinement.
- **Cumulative Distribution Function**: The model uses a cumulative distribution function to estimate the probability of meeting specific project deadlines based on resource allocation and effort.

**Advantages:**

- Provides a systematic approach to resource allocation and effort estimation.
- Considers the impact of time and resources on productivity and quality.
- Useful for planning and managing large software projects.

**Disadvantages:**

- Requires accurate input data, which can be challenging to obtain.
- May not account for all real-world complexities of software development.
- Assumes a predictable development process, which may not always hold true.

## The Norden / Rayleigh Curve

The curve is modeled by differential equation

$$m(t) = \frac{dy}{dt} = 2kate^{-at^2} \qquad \text{--------- (1)}$$

$\frac{dy}{dt}$ = manpower utilization rate per unit time

a = parameter that affects the shape of the curve

K = area under curve in the interval [0, ∞ ]

t = elapsed time

Putnam observed that
Difficulty derivative relative to time

Behavior of s/w development

If project scale is increased, the development time also increase to such an extent that $\frac{k}{t_d^3}$ remains constant around a value which could be 8,15,27.

Project is difficult to develop if

- Manpower demand is high
- When time schedule is short

## Productivity Versus Difficulty

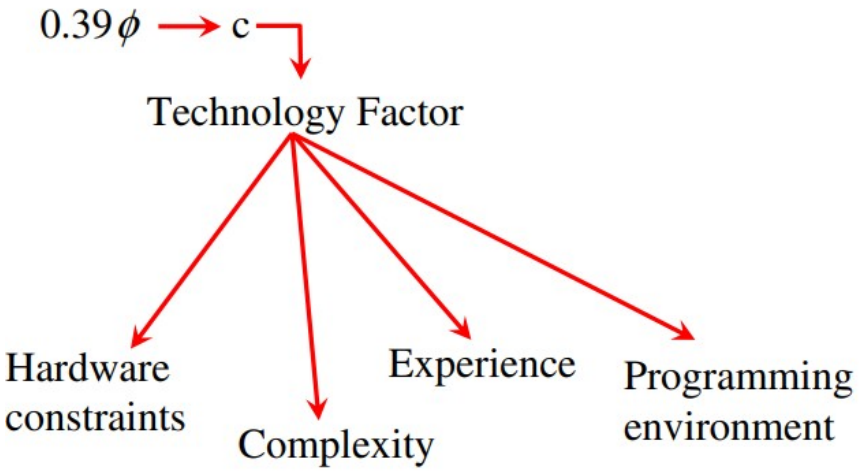
Productivity = No. of LOC developed per person-month

$$P \propto D^{\beta}$$

Avg. productivity

$$P = \frac{LOC\ produced}{cumulative\ manpower\ used\ to\ produce\ code}$$

$$S = 0.3935\,\phi\,K^{1/3}t_d^{\,4/3}$$




**Q12. Development Cycle**

## Development Subcycle

All that has been discussed so far is related to project life cycle as represented by project curve

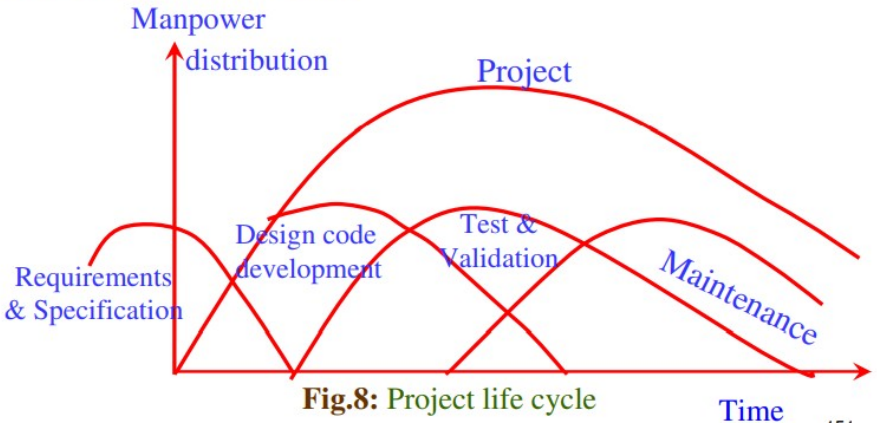
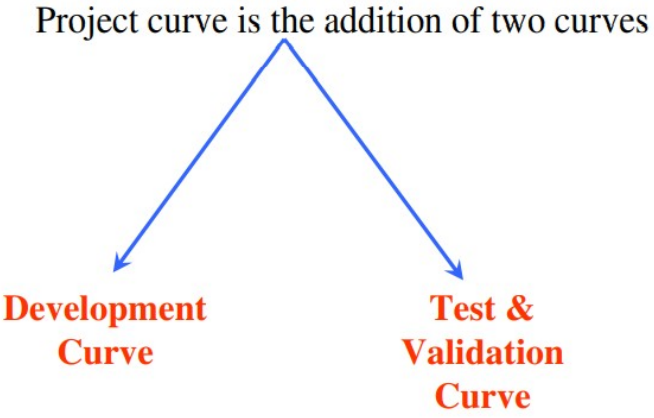



**Fig.8:** Project life cycle

**Project life cycle**




**Q13. Risk Management**

## Software Risk Management

- We Software developers are extremely optimists.
- We assume, everything will go exactly as planned.
- Other view
  - not possible to predict what is going to happen ?

Software surprises

- Never good news

Risk management is required to reduce this surprise factor

Dealing with concern before it becomes a crisis.

Quantify probability of failure & consequences of failure.

## What is risk ?

Tomorrow's problems are today's risks.

*"Risk is a problem that may cause some loss or threaten the success of the project, but which has not happened yet".*

Risk management is the process of identifying addressing and eliminating these problems before they can damage the project

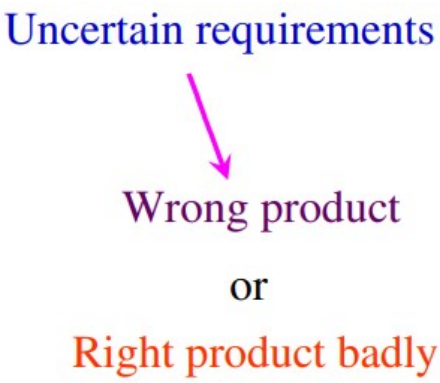
## Q14. Typical Software Risk

Capers Jones has identified the top five risk factors that threaten projects in different applications.

***1. Dependencies on outside agencies or factors.***

• Availability of trained, experienced persons • Inter group dependencies • Customer-Furnished items or information • Internal & external subcontractor relationships

***2. Requirement issues***




• Lack of clear product vision • Unprioritized requirements • Lack of agreement on product requirements • New market with uncertain needs • Rapidly changing requirements • Inadequate Impact analysis of requirements changes

***3. Management Issues***

Project managers usually write the risk management plans, and most people do not wish to air their weaknesses in public.

• Inadequate planning • Inadequate visibility into actual project status • Unclear project ownership and decision making • Staff personality conflicts • Unrealistic expectation • Poor communication

***4. Lack of knowledge***

• Inadequate training • Poor understanding of methods, tools, and techniques • Inadequate application domain experience • New Technologies • Ineffective, poorly documented or neglected processes

***5. Other risk categories***

• Unavailability of adequate testing facilities • Turnover of essential personnel • Unachievable performance requirements • Technical approaches that may not work

## Q15. Risk Management Activities

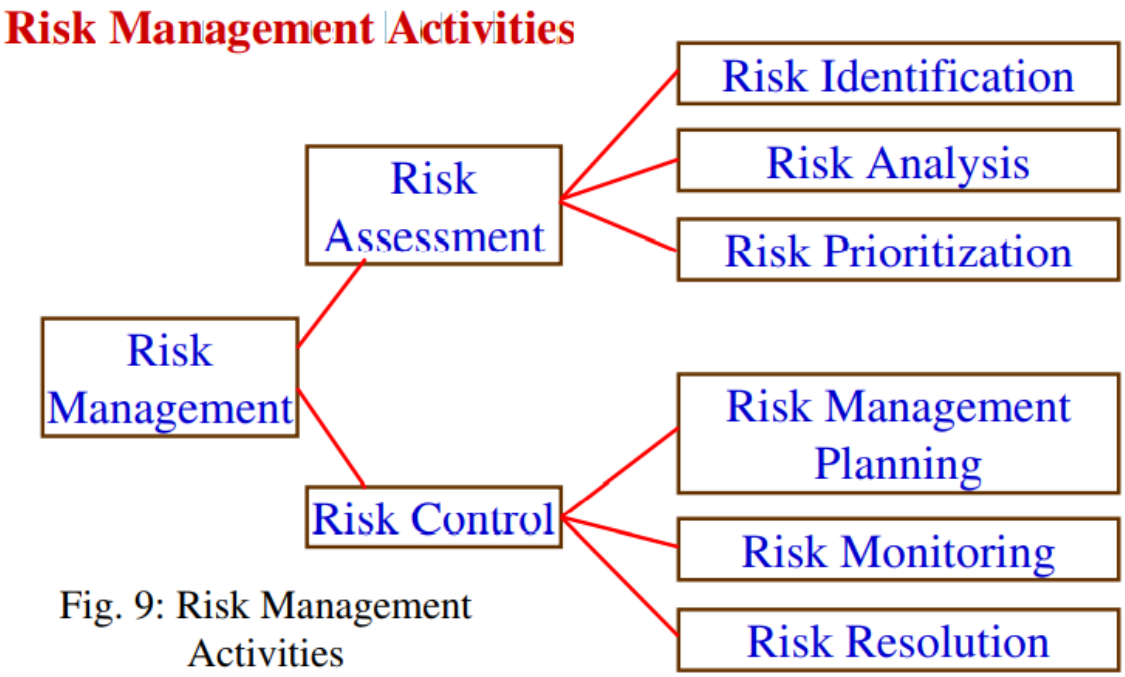



Fig. 9: Risk Management Activities

**Risk Assessment**

- **Risk Identification**: Systematically identifying potential risks that could impact the project.
- **Risk Analysis**: Examining how changes in risk input variables might affect project outcomes.
- **Risk Prioritization**: Focusing on identifying and addressing severe risks based on their impact and likelihood.

**Risk Controlling**

- **Risk Management Planning**: Developing a comprehensive plan to manage each significant risk identified.
- **Risk Monitoring**: Continuously tracking identified risks and assessing their status throughout the project lifecycle.
- **Risk Resolution**: Implementing the planned strategies to effectively manage and mitigate each identified risk.

## Q16. Software Design Document (SDD)

A **Software Design Document (SDD)** is a comprehensive document that outlines the design of a software system. It serves as a blueprint for the development team and stakeholders, detailing how the system will be constructed to meet specified requirements

The SDD serves multiple purposes:

- Provides a clear understanding of how the system will be built and how it will function.
- Facilitates communication among stakeholders, including developers, project managers, and clients.
- Acts as a reference throughout the software development lifecycle, ensuring alignment with design goals and requirements.

**Q17. Design Framework**

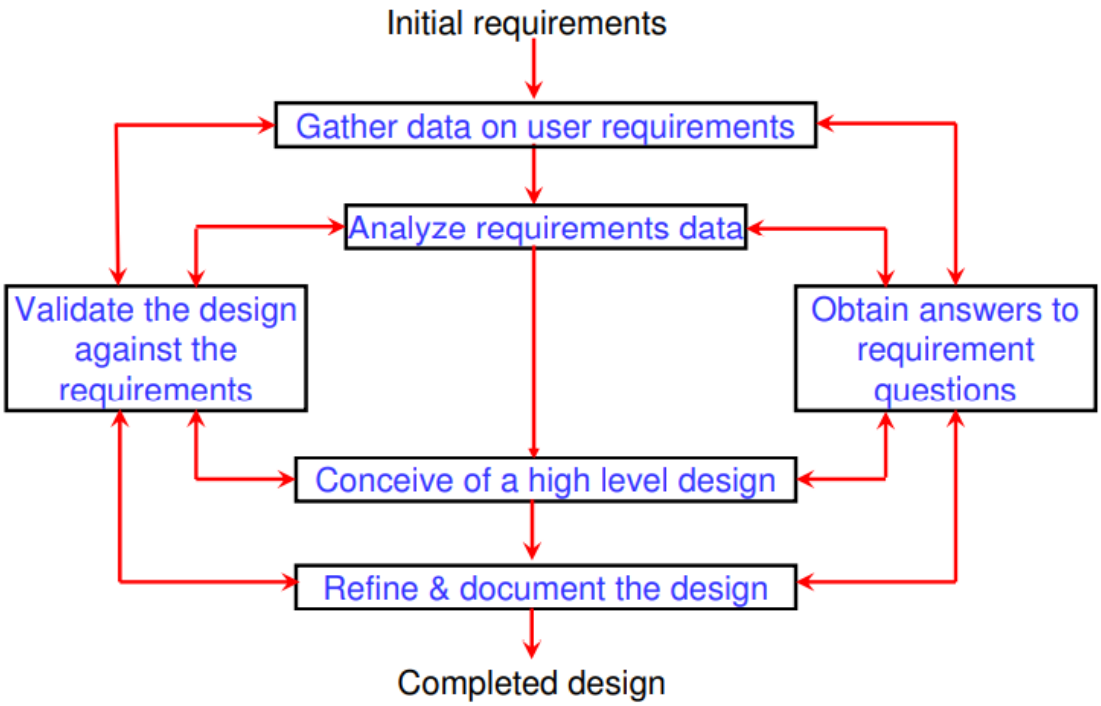



Fig. 1 : Design framework

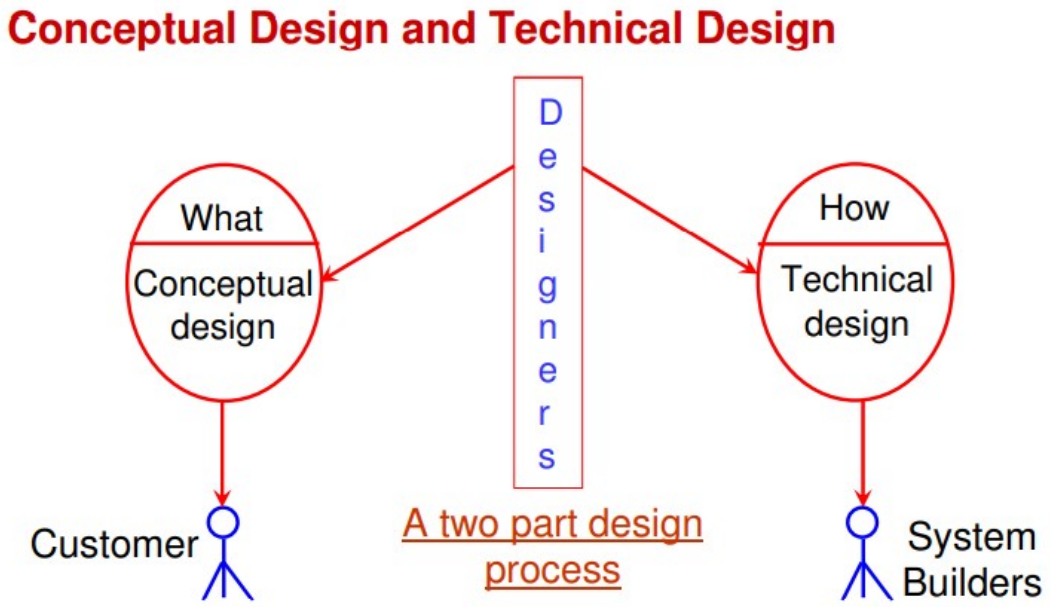



Fig. 2 : A two part design process

The design needs to be

Correct & complete
Understandable
At the right level
Maintainable

## Q18. Modularity

A modular system consist of well defined manageable units with well defined interfaces among the units

Properties :
i. Well defined subsystem
ii. Well defined purpose
iii. Can be separately compiled and stored in a library.
iv. Module can use other modules
v. Module should be easier to use than to build
vi. Simpler from outside than from the inside.

Modularity is the single attribute of software that allows a program to be intellectually manageable. It enhances design clarity, which in turn eases implementation, debugging, testing, documenting, and maintenance of software product.

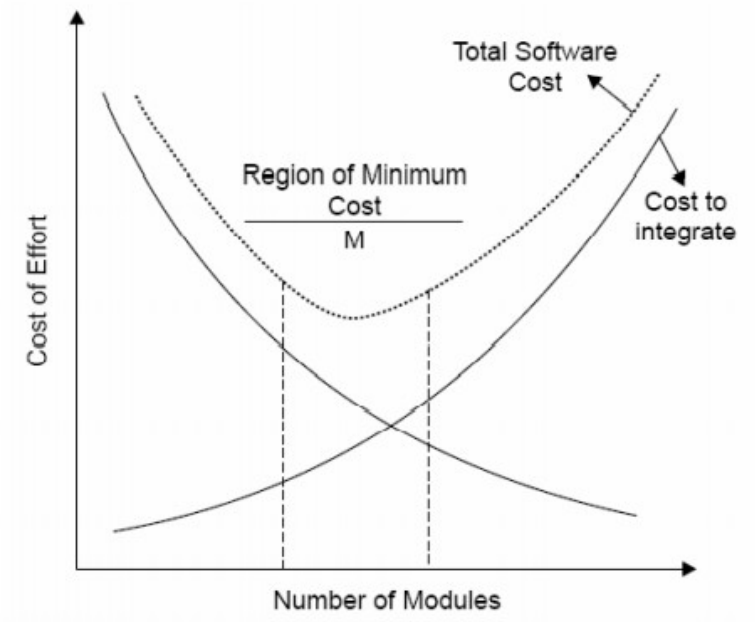
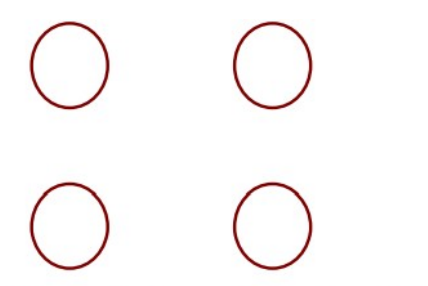



Fig. 4 : Modularity and software cost

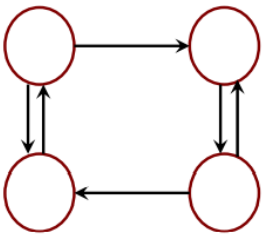
## Q19 Module Coupling

Coupling is the measure of the degree of interdependence between modules.



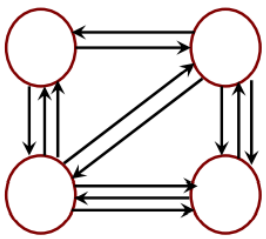
(Uncoupled : no dependencies)

(a)



Loosely coupled: some dependencies

(B)



Highly coupled: many dependencies

(C)

This can be achieved as: Controlling the number of parameters passed amongst modules. Avoid passing undesired data to calling module. Maintain parent / child relationship between calling & called modules. Pass data, not the control information.

**Q20. Types of Coupling:**

Consider the example of editing a student record in a 'student information system'.

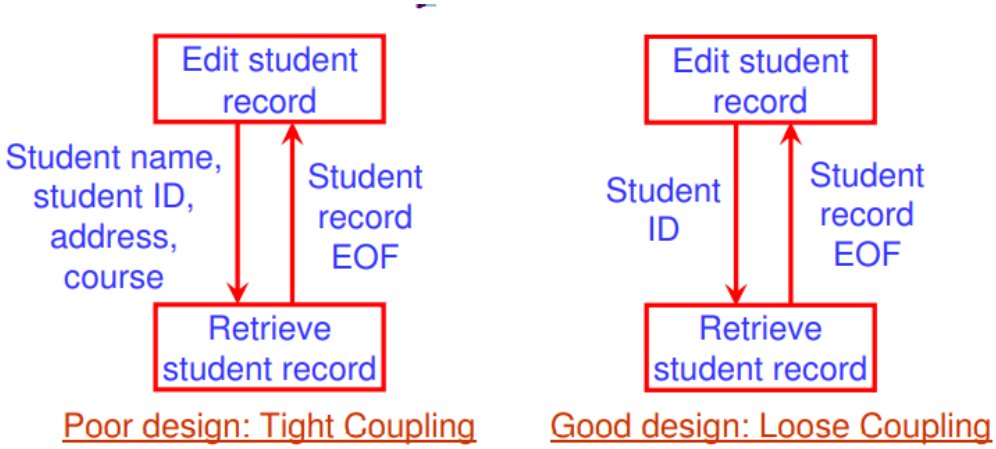



Fig. 6 : Example of coupling

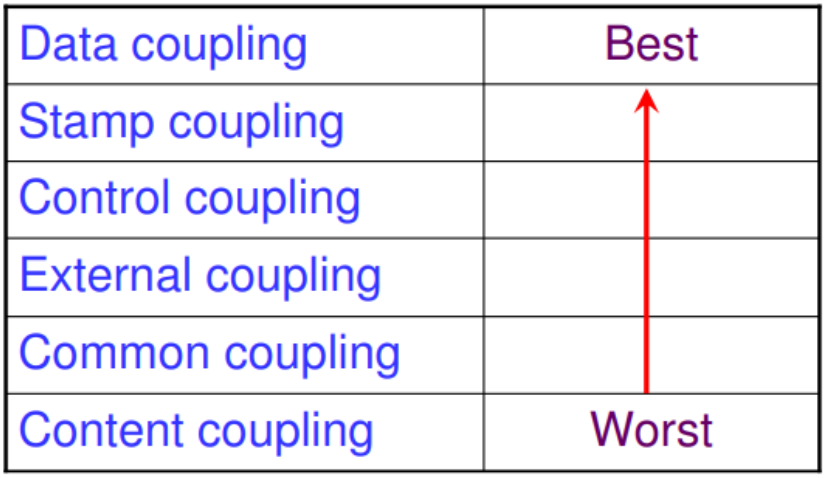



Fig. 7 : The types of module coupling

## Data coupling

The dependency between module A and B is said to be data coupled if their dependency is based on the fact they communicate by only passing of data. Other than communicating through data, the two modules are independent.

## Stamp coupling

Stamp coupling occurs between module A and B when complete data structure is passed from one module to another.

## Control coupling

Module A and B are said to be control coupled if they communicate by passing of control information. This is usually accomplished by means of flags that are set by one module and reacted upon by the dependent module.

## Common coupling

With common coupling, module A and module B have shared data. Global data areas are commonly found in programming languages. Making a change to the common data means tracing back to all the modules which access that data to evaluate the effect of changes.

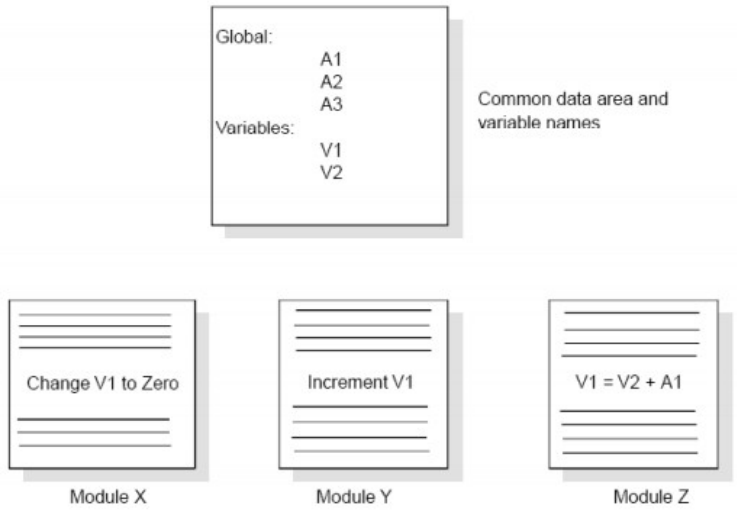



Fig. 8 : Example of common coupling

## Content coupling

Content coupling occurs when module A changes data of module B or when control is passed from one module to the middle of another. In Fig. 9, module B branches into D, even though D is supposed to be under the control of C.

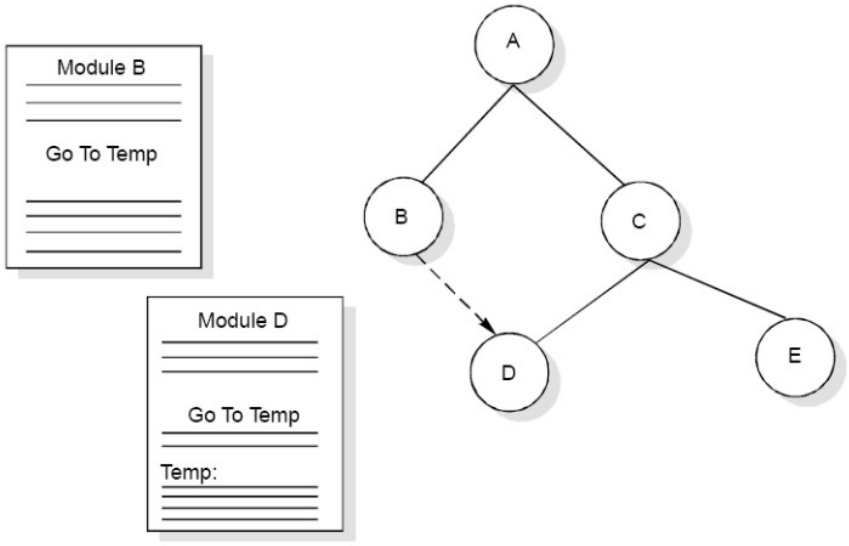



Fig. 9 : Example of content coupling

**Q21 Module Cohesion**

Cohesion is a measure of the degree to which the elements of a module are functionally related.

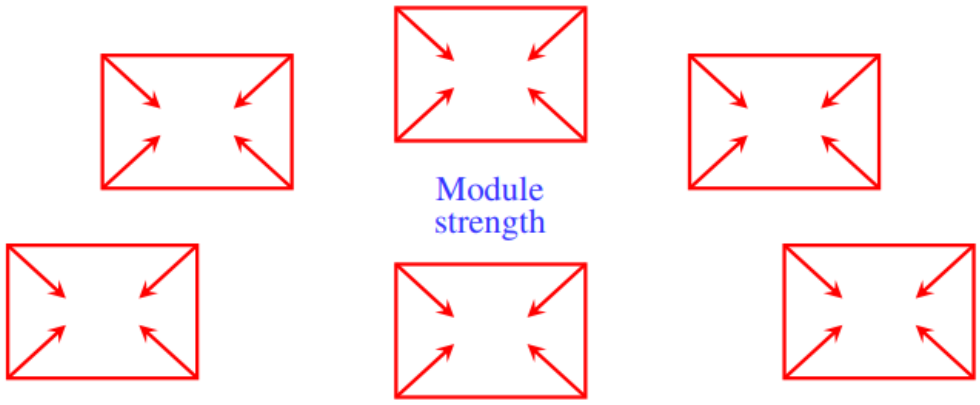



Fig. 10 : Cohesion=Strength of relations within modules

## Types of cohesion

- Functional cohesion
- Sequential cohesion
- Procedural cohesion
- Temporal cohesion
- Logical cohesion
- Coincident cohesion

| | |
|---|---|
| Functional Cohesion | Best (high) |
| Sequential Cohesion | |
| Communicational Cohesion | |
| Procedural Cohesion | |
| Temporal Cohesion | |
| Logical Cohesion | |
| Coincidental Cohesion | Worst (low) |

Fig. 11 : Types of module cohesion

Functional Cohesion
A and B are part of a single functional task. This is very good reason for them to be contained in the same procedure.

Sequential Cohesion
Module A outputs some data which forms the input to B. This is the reason for them to be contained in the same procedure.

Procedural Cohesion
Procedural Cohesion occurs in modules whose instructions although accomplish different tasks yet have been combined because there is a specific order in which the tasks are to be completed.

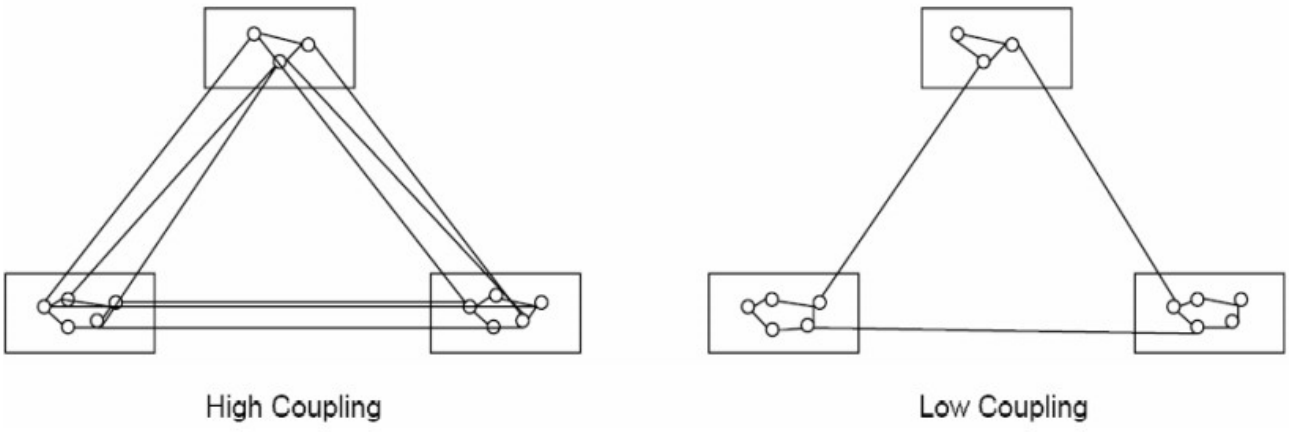
Temporal Cohesion
Module exhibits temporal cohesion when it contains tasks that are related by the fact that all tasks must be executed in the same time-span.

Logical Cohesion
Logical cohesion occurs in modules that contain instructions that appear to be related because they fall into the same logical class of functions.

Coincidental Cohesion
Coincidental cohesion exists in modules that contain instructions that have little or no relationship to one another.

## Q22. Relationship between Cohesion & Coupling

If the software is not properly modularized, a host of seemingly trivial enhancement or changes will result into death of the project. Therefore, a software engineer must design the modules with goal of high cohesion and low coupling




## Q23. STRATEGY OF DESIGN

A good system design strategy is to organize the program modules in such a way that are easy to develop and latter to, change. Structured design techniques help developers to deal with the size and complexity of programs. Analysts create instructions for the developers about how code should be written and how pieces of code should fit together to form a program. It is important for two reasons:

First, even pre-existing code, if any, needs to be understood, organized and pieced together.

Second, it is still common for the project team to have to write some code and produce original programs that support the application logic of the system.

**Q24. Bottom-Up Design:**

Bottom-up tree structure These modules are collected together in the form of a “library”.

Bottom-up design is a software design approach where individual modules or components are developed first, starting from the lower-level functionalities. These small, independent modules are created, tested, and combined to form more complex systems, building up towards the higher-level system requirements.

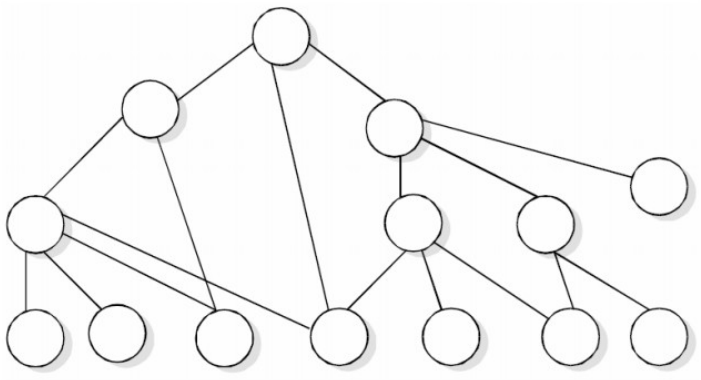


Fig. 13 : Bottom-up tree structure

**Q25. Top-Down Design**

A top down design approach starts by identifying the major modules of the system, decomposing them into their lower level modules and iterating until the desired level of detail is achieved. This is stepwise refinement; starting from an abstract design, in each step the design is refined to a more concrete level, until we reach a level where no more refinement is needed and the design can be implemented directly.

**Q26. Hybrid Design**

## Hybrid Design

For top-down approach to be effective, some bottom-up approach is essential for the following reasons:

- To permit common sub modules.
- Near the bottom of the hierarchy, where the intuition is simpler, and the need for bottom-up testing is greater, because there are more number of modules at low levels than high levels.
- In the use of pre-written library modules, in particular, reuse of modules.

## Q27. FUNCTION ORIENTED DESIGN

Function Oriented design is an approach to software design where the design is decomposed into a set of interacting units where each unit has a clearly defined function. Thus, system is designed from a functional viewpoint.

Function-oriented design is a software design approach that focuses on decomposing the system into a set of functions or procedures. These functions represent specific tasks or operations that the system performs, often derived from the system's requirements. Key characteristics include:

## Q28. Design Notations

Design notations are largely meant to be used during the process of design and are used to represent design or design decisions. For a function oriented design, the design can be represented graphically or mathematically by the following:

- Data flow diagrams
- Data Dictionaries
- Structure Charts
- Pseudocode

### Structure Chart

It partition a system into block boxes. A black box means that functionality is known to the user without the knowledge of internal design.

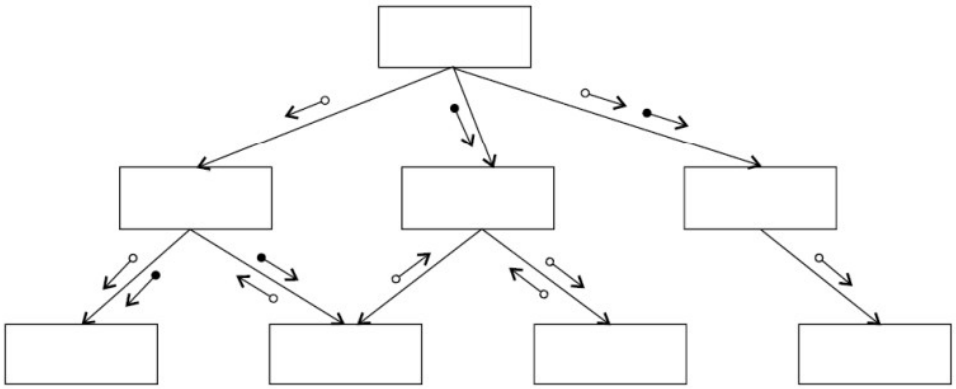


Fig. 16 : Hierarchical format of a structure chart

Pseudocode

Pseudocode notation can be used in both the preliminary and detailed design phases. Using pseudocode, the designer describes system characteristics using short, concise, English language phrases that are structured by key words such as It-Then-Else, While-Do, and End.

## Q29. Functional Procedure Layers

Function are built in layers, Additional notation is used to specify details.

- Level 0
  - Function or procedure name
  - Relationship to other system components (e.g., part of which system, called by which routines, etc.)
  - Brief description of the function purpose.
  - Author, date
- Level 1
  - Function Parameters (problem variables, types, purpose, etc.)
  - Global variables (problem variable, type, purpose, sharing information)
  - Routines called by the function
  - Side effects
  - Input/Output Assertions
- Level 2
  - Local data structures (variable etc.)
  - Timing constraints
  - Exception handling (conditions, responses, events)
  - Any other limitations
- Level 3
  - Body (structured chart, English pseudo code, decision tables, flow charts, etc.)

## Q30. IEEE Recommended practice for software design descriptions (IEEE STD 1016-1998)

- **Scope**

An SDD is a representation of a software system that is used as a medium for communicating software design information.

- **References**

i. IEEE std 830-1998, IEEE recommended practice for software requirements specifications.

ii. IEEE std 610.12-1990, IEEE glossary of software engineering terminology.

- **Definitions**

i. **Design entity.** An element (Component) of a design that is structurally and functionally distinct from other elements and that is separately named and referenced.

ii. **Design View.** A subset of design entity attribute information that is specifically suited to the needs of a software project activity.

iii. **Entity attributes.** A named property or characteristics of a design entity. It provides a statement of fact about the entity.

iv. **Software design description (SDD).** A representation of a software system created to facilitate analysis, planning, implementation and decision making.

## Q31. Design Description Organization

Each design description writer may have a different view of what are considered the essential aspects of a software design. The organization of SDD is given in table 1. This is one of the possible ways to organize and format the SDD. Software Design Design Description Organization

A recommended organization of the SDD into separate design views to facilitate information access and assimilation is given in table

1. Introduction
   - 1.1 Purpose
   - 1.2 Scope
   - 1.3 Definitions and acronyms
2. References
3. Decomposition description
   - 3.1 Module decomposition
     - 3.1.1 Module 1 description
     - 3.1.2 Module 2 description
   - 3.2 Concurrent Process decompostion
     - 3.2.1 Process 1 description
     - 3.2.2 Process 2 description
   - 3.3 Data decomposition
     - 3.3.1 Data entity 1 description
     - 3.3.2 Data entity 2 description

**Cont...**

4. Dependency description
   - 4.1 Intermodule dependencies
   - 4.2 Interprocess dependencies
   - 4.3 Data dependencies
5. Interface description
   - 5.1 Module Interface
     - 5.1.1 Module 1 description
     - 5.1.2 Module 2 description
   - 5.2 Process interface
     - 5.2.1 Process 1 description
     - 5.2.2 Process 2 description
6. Detailed design
   - 6.1 Module detailed design
     - 6.1.1 Module 1 detail
     - 6.1.2 Module 2 detail
   - 6.2 Data detailed design
     - 6.2.1 Data entry 1 detail
     - 6.2.2 Data entry 2 detail

Table 1: Organization of SDD

| Design View | Scope | Entity attribute | Example representation |
|---|---|---|---|
| Decomposition description | Partition of the system into design entities | Identification, type purpose, function, subordinate | Hierarchical decomposition diagram, natural language |
| Dependency description | Description of relationships among entities of system resources | Identification, type, purpose, dependencies, resources | Structure chart, data flow diagrams, transaction diagrams |
| Interface description | List of everything a designer, developer, tester needs to know to use design entities that make up the system | Identification, function, interfaces | Interface files, parameter tables |
| Detail description | Description of the internal design details of an entity | Identification, processing, data | Flow charts, PDL etc. |

Table 2: Design views

--

**Q31. Object Oriented Design**

Object oriented design is the result of focusing attention not on the function performed by the program, but instead on the data that are to do manipulated by the program. Thus, it is orthogonal to function oriented design.

Object Oriented Design begins with an examination of the real world "things" that are part of the problem to be solved. These things (which we will call objects) are characterized individually in terms of their attributes and behavior.

The various terms related to object design are:

### i. Objects

The word "Object" is used very frequently and conveys different meaning in different circumstances. Here, meaning is an entity able to save a state (information) and which offers a number of operations (behavior) to either examine or affect this state. An object is characterized by number of operations and a state which remembers the effect of these operations.

### ii. Messages

Objects communicate by message passing. Messages consist of the identity of the target object, the name of the requested operation and any other operation needed to perform the function. Message are often implemented as procedure or function calls.

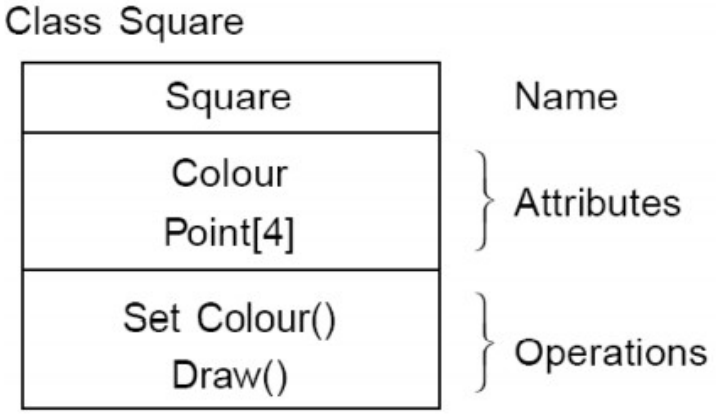
### iii. Abstraction

In object oriented design, complexity is managed using abstraction. Abstraction is the elimination of the irrelevant and the amplification of the essentials.

### iv. Class

In any system, there shall be number of objects. Some of the objects may have common characteristics and we can group the objects according to these characteristics. This type of grouping is known as a class. Hence, a class is a set of objects that share a common structure and a common behavior.

We may define a class "car" and each object that represent a car becomes an instance of this class. In this class "car", Indica, Santro, Maruti, Indigo are instances of this class as shown in fig. 20.



## v. Attributes

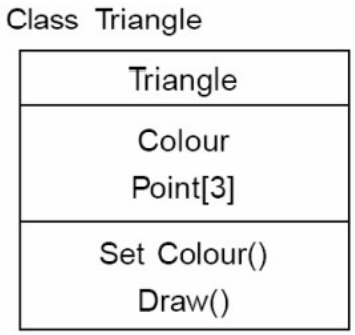
An attributes is a data value held by the objects in a class. The square class has two attributes: a colour and array of points. Each attributes has a value for each object instance. The attributes are shown as second part of the class as shown in fig. 21.

## vi. Operations

An operation is a function or transformation that may be applied to or by objects in a class. In the square class, we have two operations: set colour() and draw(). All objects in a class share the same operations. An object "knows" its class, and hence the right implementation of the operation. Operation are shown in the third part of the class as indicated in fig. 21.

## vii. Inheritance

Imagine that, as well as squares, we have triangle class. Fig. 22 shows the class for a triangle.



## viii. Polymorphism

When we abstract just the interface of an operation and leave the implementation to subclasses it is called a polymorphic operation and process is called polymorphism.

## ix. Encapsulation (Information Hiding)

Encapsulation is also commonly referred to as “Information Hiding”. It consists of the separation of the external aspects of an object from the internal implementation details of the object.

## x. Hierarchy

Hierarchy involves organizing something according to some particular order or rank. It is another mechanism for reducing the complexity of software by being able to treat and express sub-types in a generic way.

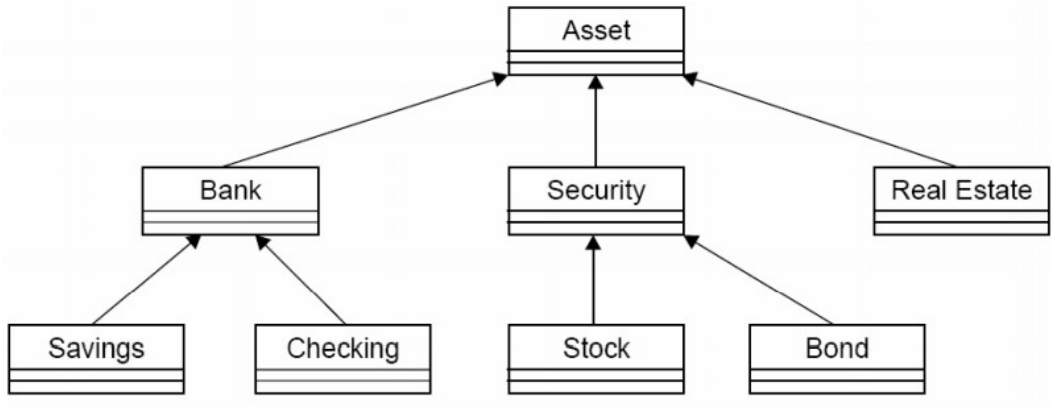



Fig. 24: Hierarchy

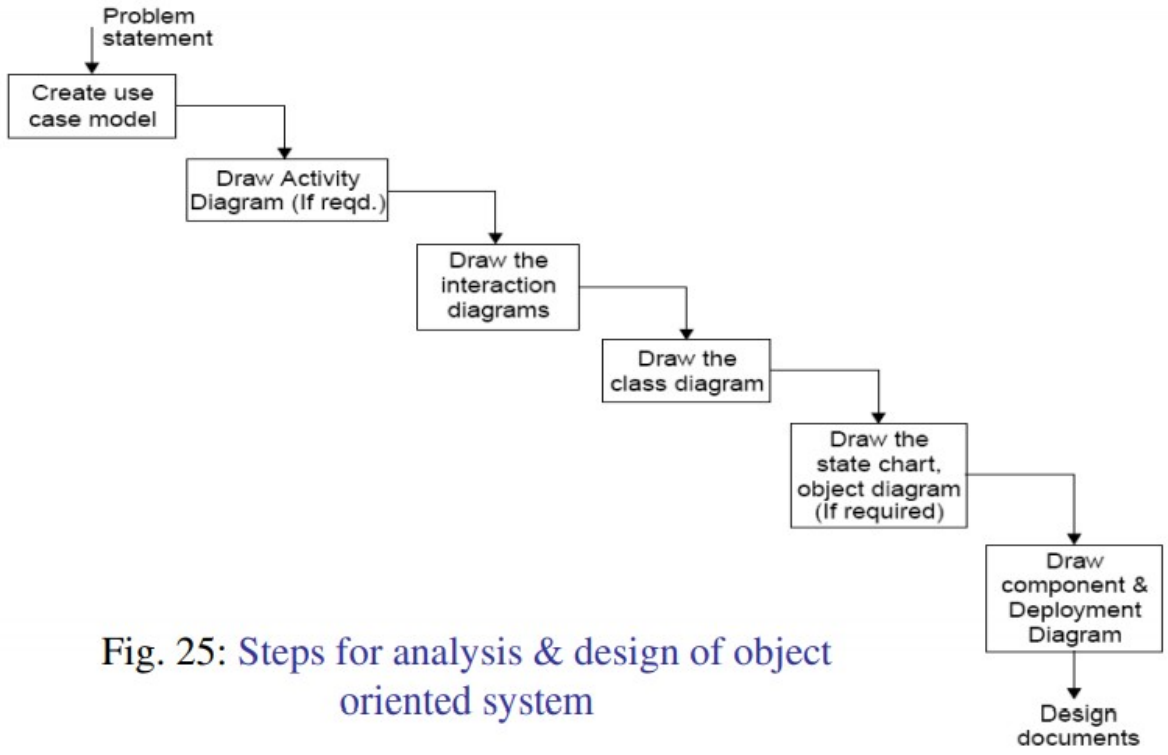



Fig. 25: Steps for analysis & design of object oriented system

## Q32. User Interface Design

The user interface is the **front-end application** view to which the **user interacts** to use the software. The software becomes more popular if its user interface is:

1. **Attractive**
2. **Simple to use**
3. **Responsive in a short time**
4. **Clear to understand**
5. **Consistent on all interface screens**

### Types of User Interface

1. **Command Line Interface:** The Command Line Interface provides a command prompt, where the user types the command and feeds it to the system. The user needs to remember the syntax of the command and its use.
2. **Graphical User Interface:** Graphical User Interface provides a simple interactive interface to interact with the system. GUI can be a combination of both hardware and software. Using GUI, the user interprets the software.

PYQ:

Q1. Discuss the need and importance of SRS.

The Software Requirements Specification (SRS) is essential because it clearly defines the software's functionality, constraints, and interfaces, ensuring alignment between stakeholders and developers. It helps prevent misunderstandings, serves as a reference throughout development, aids in testing, and improves project management and quality control.

Q4 Compare Facilitated application specification technique with brainstorming sessions. (Tabular)

| Aspect | Facilitated Application Specification Technique (FAST) | Brainstorming Sessions |
|---|---|---|
| Purpose | To gather and structure requirements collaboratively | To generate a wide range of ideas and solutions |
| Facilitator | Requires a trained facilitator to guide the session | May or may not have a facilitator |
| Focus | Structured, focused on defining software specifications | Freeform, focused on idea generation |
| Participants | Involves stakeholders, developers, and users | Involves any group, often multidisciplinary |
| Documentation | Produces formal requirements and specifications | Primarily informal, focused on idea capture |
| Output | Detailed requirements and functional specifications | Collection of creative ideas or potential solutions |
| Methodology | Follows a step-by-step, structured approach | Open-ended, less structured |
| Timeframe | Usually more time-consuming due to formal processes | Typically shorter, as it encourages rapid idea generation |

Q5. What are size metrics? How FP mertic is advantageous over LOC metric?

**Size Metrics**: These are measures used to estimate or assess the size of a software system. Common size metrics include Lines of Code (LOC) and Function Points (FP). They help in project estimation, cost analysis, and productivity assessment.

**FP vs. LOC**:

- **FP Metric**: Measures software size based on functionality, considering inputs, outputs, user interactions, and files.
- **Advantages over LOC**:

- Language-independent, making it suitable for comparing systems in different languages.
- Focuses on user requirements, not just code length.
- More accurate for effort estimation in early stages of developmen

**Q6. Data Structure Matrices**:
Data structure matrices are used to represent relationships between data entities in a structured format, typically in a tabular form. They help visualize connections and dependencies between data components, aiding in understanding data flow and organization within a system.

**Q7. Software Prototyping**:
Software prototyping is the process of creating a preliminary version (prototype) of a software application to visualize and validate requirements. Prototypes can be low-fidelity (paper sketches) or high-fidelity (interactive models). This approach helps gather user feedback, refine requirements, and reduce risks before full-scale development.

**Q8. Time vs. Cost Trade-off in Putnam Resource Allocation Model**:
The Putnam model suggests that there is a trade-off between time and cost in software development. If a project is expedited (shortened time), it typically requires more resources (higher costs) to maintain quality and meet deadlines. Conversely, extending the timeline can reduce costs but may increase project risks and complexity. The optimal allocation seeks to balance resource usage and project deadlines for efficiency.

Q9. Write the name phase of software cycle for which IEEE recommended standard IEEE 830-1993 is used.

The IEEE recommended standard IEEE 830-1993 is used in the **Requirements Specification** phase of the software development lifecycle. It provides guidelines for writing software requirements specifications (SRS), ensuring clarity, consistency, and completeness in documenting requirements.

Q10. Write the full form of FAST and QFD techniques.

- **FAST**: **Facilitated Application Specification Technique**
- **QFD**: **Quality Function Deployment**

Q11. Explain the Walston-Felix model.

The **Walston-Felix model** is a software metrics model designed to predict software development effort based on the size of the software and its complexity. It provides a mathematical approach to estimate the resources required for software projects. The model is particularly focused on the relationship between lines of code (LOC) and effort in software development.

## Key Aspects of the Walston-Felix Model:

1. **Effort Estimation**: The model estimates the effort required (usually in person-hours) to develop a software project based on its size measured in LOC.

2. **Mathematical Formula**: The basic formula used in the Walston-Felix model is:

$$E = a \times (LOC)^b$$

   - **E**: Estimated effort (in person-hours)
   - **a**: A constant that represents the productivity of the development team
   - **LOC**: The size of the software in lines of code
   - **b**: An exponent that represents the non-linear relationship between size and effort (typically greater than 1).

Q12. What is Risk Exposure? What technique is used to control each risk?

**Risk Exposure** refers to the potential impact or consequences of a risk on a project, typically quantified as the product of the probability of the risk occurring and the potential loss or damage it could cause. It helps prioritize risks based on their significance to the project's success, enabling project managers to allocate resources effectively for risk management.

The formula for Risk Exposure is:

$$\text{Risk Exposure} = \text{Probability of Risk} \times \text{Impact of Risk}$$

## Techniques to Control Risks:

1. **Avoidance**: Altering the project plan to eliminate the risk or protect the project objectives from its impact. For example, choosing a different technology that poses fewer risks.
2. **Mitigation**: Implementing actions to reduce the likelihood or impact of the risk. This could include additional testing, improved training, or enhancing system security.
3. **Transfer**: Shifting the risk to a third party, such as through insurance or outsourcing. This technique is often used to manage financial risks.
4. **Acceptance**: Acknowledging the risk and deciding to proceed without specific action, typically when the risk is minor or the cost of mitigation exceeds the potential impact. This may involve creating contingency plans to manage the consequences if the risk occurs.
5. **Contingency Planning**: Developing backup plans to address risks that are accepted or unavoidable. This ensures that if the risk occurs, there are predefined steps to minimize its impact.

**Q1. Define Software Metrics?**

Software metrics can be defined as “The continuous application of measurement based techniques to the software development process and its products to supply meaningful and timely management information, together with the use of those techniques to improve that process and its product”.

**Q2. Why do we need software metrics?**

Software metrics help answer critical questions like measuring software size, estimating development costs, predicting bugs, determining testing timelines, assessing module complexity, strength, coupling, reliability, test technique effectiveness, and evaluating the strength of programs and test suites.

**Q3. Area of application of software metrics?**

The most established area of software metrics is cost and size estimation techniques.
The prediction of quality levels for software, often in terms of reliability, is another area where software metrics have an important role to play.
The use of software metrics to provide quantitative checks on software design is also a well established area.

**Q4. What are the categories of metrics?**

- Categories of Metrics
  i. **Product metrics:** describe the characteristics of the product such as size, complexity, design features, performance, efficiency, reliability, portability, etc.
  ii. **Process metrics:** describe the effectiveness and quality of the processes that produce the software product. Examples are:
    - effort required in the process
    - time to produce the product
    - effectiveness of defect removal during development
    - number of defects found during testing
    - maturity of the process

**Q5. What are token count?**

The size of the vocabulary of a program, which consists of the number of unique tokens used to build a program is defined as:

$$\eta = \eta_1 + \eta_2$$

where

$\eta$ : vocabulary of a program

$\eta_1$ : number of unique operators

$\eta_2$ : number of unique operands

The length of the program in the terms of the total number of tokens used is:

$N = N_1 + N_2$

where

$N$ : program length

$N_1$ : total occurrences of operators

$N_2$ : total occurrences of operands

**Q6. What is Halstead Software Metric Science?**

Halstead Software Metric Science, introduced by Maurice Halstead in 1977, measures software code complexity, effort, and maintainability. It treats software as a collection of tokens (operators and operands) and applies mathematical formulas to quantify various program attributes.

**Key Concepts:**

1. Operators: Symbols or keywords that perform actions (e.g., `+`, `if`, `return`).
2. Operands: Variables, constants, or data being manipulated.

**Core Metrics:**

1. Program vocabulary (n): Total unique operators (n1) and operands (n2):
   n = n1 + n2
2. Program length (N): Total occurrences of operators (N1) and operands (N2):
   N = N1 + N2
3. Volume (V): Size of the implementation in terms of information:
   V = N * log2(n)
4. Difficulty (D): Complexity of the program:
   D = (n1 / 2) * (N2 / n2)
5. Effort (E): Effort required to develop or understand the code:
   E = D * V
6. Time to implement (T): Approximation of development time:
   T = E / 18
7. Bugs (B): Estimated number of defects in the program:
   B = V / 3000

**Significance:**

Halstead metrics provide a quantitative way to evaluate code readability, maintainability, and complexity. They are useful for predicting effort, identifying complex modules, and improving software quality.

**Q7. What is the volume metric of the program?**

$$V = N * \log_2 \eta$$

The unit of measurement of volume is the common unit for size “bits”. It is the actual size of a program if a uniform binary encoding for the vocabulary is used.

**Q8. What is Program Level, Program Difficulty, Effort?**

Program Level

$$L = V^* / V$$

The value of L ranges between zero and one, with L=1 representing a program written at the highest possible level (i.e., with minimum size).

$V*=(n1+n2)\cdot\log2(n)$ (ideal volume)

Program Difficulty

$$D = 1 / L$$

As the volume of an implementation of a program increases, the program level decreases and the difficulty increases. Thus, programming practices such as redundant usage of operands, or the failure to use higher-level control constructs will tend to increase the volume as well as the difficulty.

Effort

$$E = V / L = D * V$$

The unit of measurement of E is elementary mental discriminations.

**Q9. What are the rules for counting in C language?**

The rules for counting tokens in the C language, based on Halstead's software metrics, can be summarized as follows:

1. **Comments**: Comments are not counted as they do not contribute to the program's operational complexity.

2. **Identifiers and function declarations**: These are not counted as operands in the metric. Only variables and constants are considered operands.
3. **Variables and constants**: Both are considered **operands**.
4. **Global variables**: When used across multiple modules of the same program, global variables are counted as multiple occurrences of the same operand.
5. **Local variables**: Local variables with the same name in different functions are considered **unique operands**.
6. **Function calls**: Function calls are considered **operators**, as they represent actions or operations.
7. **Control and looping statements**:
    - All control statements like `if`, `else`, `switch`, `case`, etc., and looping statements like `for`, `while`, and `do-while` are counted as **operators**.
8. **Switch statement**: The `switch` and all `case` statements are considered **operators**.
9. **Brackets, commas, and terminators**: These symbols (e.g., `()`, `{}`, `[]`, `,`, and `;`) are considered **operators**.
10. **GOTO statement**: The `GOTO` is counted as an **operator**, and the label is counted as an **operand**.
11. **Unary and binary operators**:
    - The unary and binary occurrences of operators like `+`, `-`, and `*` are treated separately.
12. **Array variables**:
    - In expressions like `array-name[index]`, `array-name` and `index` are considered **operands**, while the `[]` is counted as an **operator**.
13. **Structure variables**:
    - In expressions like `struct-name.member-name` or `struct-name->member-name`, the `struct-name` and `member-name` are **operands**, while `.` and `->` are considered **operators**.
14. **Preprocessor directives**: All preprocessor directives (e.g., `#define`, `#include`) are ignored in the token count.

**Q10. What is the Estimate Program Length?**

$$\hat{N} = \eta_1 \log_2 \eta_1 + \eta_2 \log_2 \eta_2$$

**Q11. Write all the variated Metrics of Halstead?**

- Potential Volume

$$V^* = (2+\eta_2^*)\log_2(2+\eta_2^*)$$

## Estimated Program Level / Difficulty

Halstead offered an alternate formula that estimate the program level.

$$\hat{L} = 2\eta_2 / (\eta_1 N_2)$$

where

$$\hat{D} = \frac{1}{\hat{L}} = \frac{\eta_1 N_2}{2\eta_2}$$

- Effort and Time

$$E = V / \hat{L} = V * \hat{D}$$

$$= (n_1 N_2 N \log_2 \eta) / 2\eta_2$$

$$T = E / \beta$$

β is normally set to 18 since this seemed to give best results in Halstead's earliest experiments, which compared the predicted times with observed programming times, including the time for design, coding, and testing.

**Q12. What is Language Level?**

$$\lambda = L \times V^* = L^2 V$$

## Components:

1. **L** is the **language level** (as previously defined).
2. **V** is the **actual program volume** (total number of operators and operands).
3. **V*** is the **ideal program volume** (based on unique operators and operands).
4. The formula essentially multiplies the **program level (L)** by the **ideal volume (V*)** or squares the program level and multiplies by the actual volume.

## Meaning of Language Level (λ):

- **λ (Language level)** is a measure of the complexity of the programming language and the efficiency of the code.
- A **higher value of λ** suggests that the language and code structure are more complex and closer to the ideal in terms of its volume and level, making it potentially harder to maintain or understand.
- A **lower value of λ** means the language and code are more efficient, with simpler constructs and fewer variations, closer to an idealized minimal form.

| *Language* | *Language Level* $\lambda$ | *Variance* $\sigma$ |
|---|---|---|
| PL/1 | 1.53 | 0.92 |
| ALGOL | 1.21 | 0.74 |
| FORTRAN | 1.14 | 0.81 |
| CDC Assembly | 0.88 | 0.42 |
| PASCAL | 2.54 | – |
| APL | 2.42 | – |
| C | 0.857 | 0.445 |

**Table 1:** Language levels

Q13.

Consider the program shown in Table 3. Calculate the various software science metrics.

```
#include < stdio.h >
#define MAXLINE 100
int getline(char line[],int max);
int strindex(char source[],char search for[]);
char pattern[ ]="ould";
int main()
{
      char line[MAXLINE];
      int found = 0;
      while(getline(line,MAXLINE)>0)
            if(strindex(line, pattern)>=0)
            {
                  printf("%s",line);
                  found++;
            }
      return found;
}
```

```
int getline(char s[],int lim)
{
    int c,i=0;
    while(--lim > 0 && (c=getchar())!= EOF && c!='\n')
        s[i++]=c;
    if(c=='\n')
        s[i++] = c;
    s[i] = '\0';
    return i;
}
int strindex(char s[],char t[])
{
    int i,j,k;
    for(i=0;s[i] !='\0';i++)
    {
        for(j=i,k=0;t[k] != '\0',s[j] ==t[k];j++,k++);
        if(k>0 && t[k] =='\0')
            return i;
    }
    return -1;
}
```

**Table 3**

## Solution

List of operators and operands are given in Table 4.

| *Operators* | *Occurrences* | *Operands* | *Occurrences* |
|---|---|---|---|
| main () | 1 | — | — |
| – | 1 | **Extern variable** pattern | 1 |
| for | 2 | **main function** line | 3 |
| = = | 3 | found | 2 |
| ! = | 4 | **getline function** s | 3 |
| getchar | 1 | lim | 1 |

| | | | |
|---|---|---|---|
| ( ) | 1 | *c* | 5 |
| && | 3 | *i* | 4 |
| – – | 1 | **Strindex function** *s* | 2 |
| return | 4 | *t* | 3 |
| + + | 6 | *i* | 5 |
| printf | 1 | *j* | 3 |
| > = | 1 | *k* | 6 |
| strindex | 1 | **Numerical Operands** 1 | 1 |
| If | 3 | MAXLINE | 1 |
| > | 3 | 0 | 8 |
| getline | 1 | '\0' | 4 |
| while | 2 | '\n' | 2 |
| { } | 5 | **strings "ould"** | 1 |
| = | 10 | — | — |
| [ ] | 9 | — | — |
| , | 6 | — | — |
| ; | 14 | — | — |
| EOF | 1 | — | — |
| $\eta_1 = 24$ | $N_1 = 84$ | $\eta_2 = 18$ | $N_2 = 55$ |

Program vocabulary $\eta = 42$

Program length $N = N_1 + N_2$

$= 84 + 55 = 139$

Estimated length $\hat{N} = 24\log_2 24 + 18\log_2 18 = 185.115$

% error $= 24.91$

Program volume $V = 749.605$ bits

Estimated program level $= \frac{2}{\eta_1} \times \frac{\eta_2}{N_2}$

$$= \frac{2}{24} \times \frac{18}{55} = 0.02727$$

Minimal volume $V^*=20.4417$

Effort $= V / \hat{L}$

$$= \frac{748.605}{.02727}$$

= 27488.33 elementary mental discriminations.

Time T = $E / \beta = \frac{27488.33}{18}$

= 1527.1295 seconds

= 25.452 minutes

**Q14. What is Data Structure Metrics? How can we calculate amount of data in a program ?**

Essentially the need for software development and other activities are to process data. Some data is input to a system, program or module; some data may be used internally, and some data is the output from a system, program, or module.

## Data Structure Metrics

| *Program* | *Data Input* | *Internal Data* | *Data Output* |
|---|---|---|---|
| Payroll | Name/ Social Security No./ Pay Rate/ Number of hours worked | Withholding rates<br>Overtime factors<br>Insurance premium Rates | Gross pay withholding<br>Net pay<br>Pay ledgers |
| Spreadsheet | Item Names/ Item amounts/ Relationships among items | Cell computations<br>Sub-totals | Spreadsheet of items and totals |
| Software Planner | Program size/ No. of software developers on team | Model parameters<br>Constants<br>Coefficients | Est. project effort<br>Est. project duration |

**Fig.1:** Some examples of input, internal, and output data

That's why an important set of metrics which capture in the amount of data input, processed in an output form software. A count of this data structure is called Data Structured Metrics. In these

concentrations is on variables (and given constant) within each module & ignores the input-output dependencies.

There are some Data Structure metrics to compute the effort and time required to complete the project. These considerations are:

1. The Amount of Data.
2. The Usage of data within a Module.
3. Program weakness.
4. The sharing of Data among Modules.

**Q15. Define the following Terms:-**

1. **Live Variable.**
2. **Variable Span.**

✓ Live Variables

Definitions :

1. A variable is live from the beginning of a procedure to the end of the procedure.
2. A variable is live at a particular statement only if it is referenced a certain number of statements before or after that statement.
3. A variable is live from its first to its last references within a procedure.

✓ Variable spans

```
...
21          scanf ("%d %d, &a, &b)
...
32          x =a;
...
45          y = a – b;
...
53          z = a;
...
60          printf ("%d %d, a, b);
...
```

**Fig.:** Statements in ac program referring to variables a and b.

The size of a span indicates the number of statements that pass between successive uses of a variables

**Q16. Explain all the metric or considerations in data structure metrics.**

**1. The Amount of Data:** One method for determining the amount of data is to count the number of entries in the cross-reference list. The Amount of Data A variable is a string of alphanumeric characters that is defined by a developer and that is used to represent some value during either compilation or execution

To measure the amount of Data, there are further many different metrics, and these are:

- **Number of variable (VARS):** In this metric, the Number of variables used in the program is counted.
- **Number of Operands ($\eta_2$):** In this metric, the Number of operands used in the program is counted.
- **Total number of occurrence of the variable (N2):** In this metric, the total number of occurrence of the variables are computed

$$\eta_2 = \text{VARS} + \text{unique constants} + \text{labels.}$$

Halstead introduced a metric that he referred to as $\eta_2$ to be a count of the operands in a program – including all variables, constants, and labels. Thus,

$$\eta_2 = VARS + \text{unique constants} + \text{labels}$$

Consider,

```
int a = 5;        // Variable 'a' initialized with constant 5
int b = 10;       // Variable 'b' initialized with constant 10
int c = a + b;    // Operand 'a' and 'b' used
int d = c * 2;    // Operand 'c' and constant 2 used
int e = b + d;    // Operand 'b' and 'd' used

if (a > b) {      // 'a' and 'b' used in condition
   goto label1;   // Label used
}
label1:
```

## Step 1: Count the VARS (Variables)

The variables in the program are:

- a
- b
- c
- d
- e

So, **VARS = 5**.

## Step 2: Count the Unique Constants

The constants used are:

- `5` (assigned to `a`)
- `10` (assigned to `b`)
- `2` (used in `c * 2`)

So, there are **3 unique constants**: `5`, `10`, and `2`.

## Step 3: Count the Labels

The program has the label:

- `label1` (used in the `goto` statement)

So, there is **1 label**.

## Step 4: Compute n2

Using the formula:

**n2 = VARS + unique constants + labels**

**n2 = 5 (variables) + 3 (unique constants) + 1 (label)**

**n2 = 9**

**2. The Usage of data within a Module:** The measure this metric, the average numbers of live variables are computed. A variable is live from its first to its last references within the procedure.

$$\text{Average no of Live variables (LV)} = \frac{\text{Sum of count live variables}}{\text{Sum of count of executable statements}}$$

Consider,

```
int a = 5;
int b = 10;
int c = a + b;
a = c;
b = a;
```

## Live Variable Analysis:

At each program point, we track which variables are live (used later in the code).

- **Point 1**: `int a = 5;`
  Live variables: None, because `a` is just assigned a value and not used yet.
- **Point 2**: `int b = 10;`
  Live variables: None, because `b` is also just assigned a value and not used yet.

- **Point 3**: `int c = a + b;`
  Live variables: `{a, b}` because `a` and `b` are used to compute `c`.
- **Point 4**: `a = c;`
  Live variables: `{c}` because `c` is used to assign to `a`, but `b` is no longer used.
- **Point 5**: `b = a;`
  Live variables: `{a}` because `a` is used to assign to `b`, but `c` is no longer needed.

## Average of Live Variables:

Now, to compute the **average number of live variables**, we track the number of live variables at each point and calculate the average:

- Point 1: 0 live variables
- Point 2: 0 live variables
- Point 3: 2 live variables (`a`, `b`)
- Point 4: 1 live variable (`c`)
- Point 5: 1 live variable (`a`)

**Average number of live variables** = (0 + 0 + 2 + 1 + 1) / 5 = **0.8**

Thus, the average number of live variables in the given program is **0.8**.

**3. Program weakness:** Program weakness depends on its Modules weakness. If Modules are weak(less Cohesive), then it increases the effort and time metrics required to complete the project.

$$\gamma = \frac{\text{Sum of count of live variables}}{\text{Total number of variables}}$$

Module Weakness (WM) = LV* γ

A program is normally a combination of various modules; hence, program weakness can be a useful measure and is defined as:

$$\mathbf{WP} = \frac{\left(\sum_{i=1}^{m} WM_i\right)}{m}$$

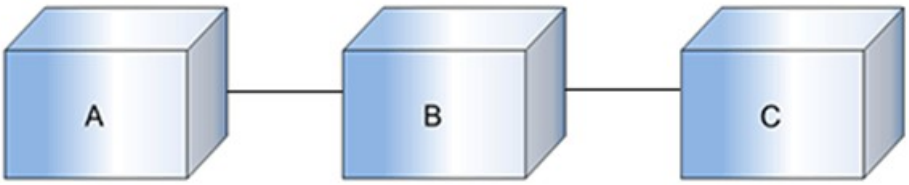
Where

**$WM_i$**: Weakness of the ith module

**WP**: Weakness of the program

**m**: No of modules in the program

**4. There Sharing of Data among Module:** As the data sharing between the Modules increases (higher Coupling), no parameter passing between Modules also increased, As a result, more effort and time are required to complete the project. So Sharing Data among Module is an important metrics to calculate effort and time.




Three modules from an imaginary program

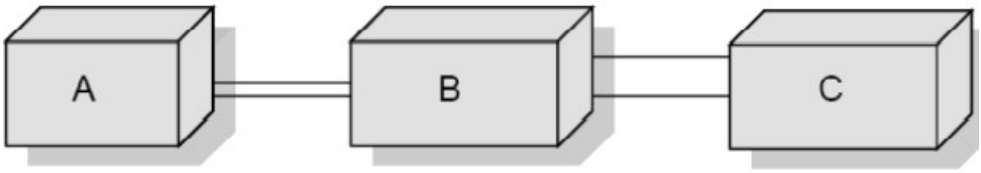



**Fig.11:** "Pipes" of data shared among the modules

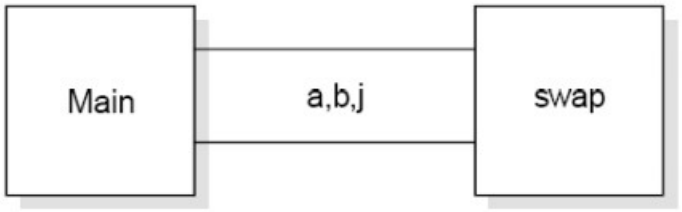



**Fig.12:** The data shared in program bubble

**Q17. What is Information Flow Metrics?**

**Information Flow Metrics** are software metrics used to assess the flow of information within a software system, particularly the movement and control of data between modules, functions, or components. These metrics help to evaluate the dependencies, data sharing, and communication between different parts of a program, which can be crucial for understanding its complexity, maintainability, and security.

## Information Flow Metrics

| | |
|---|---|
| Component | : Any element identified by decomposing a (software) system into its constituent parts. |
| Cohesion | : The degree to which a component performs a single function. |
| Coupling | : The term used to describe the degree of linkage between one component to others in the same system. |

**Q18. Explain the basic information flow model.**

The basic information flow model helps in assessing the flow of control and data between components within a system. It is commonly used to evaluate how tightly or loosely components are coupled in terms of control and data passing. The model defines three primary metrics: FAN IN, FAN OUT, and the Information Flow (IF) index for a component. These metrics provide insight into how components interact with each other, which can help evaluate system complexity, modularity, and maintainability.

## Explanation of Metrics:

1. **FAN IN**:
    - FAN IN refers to the number of other components that can call or pass control to a given component.
    - This is a measure of how many components are dependent on the component in question.
    - Example: If Component A is called by Component B, C, and D, then FAN IN of A is 3.
2. **FAN OUT**:
    - FAN OUT refers to the number of components that are called by the given component.
    - This is a measure of how many components are dependent on the component in question.
    - Example: If Component A calls Component E and F, then FAN OUT of A is 2.
3. **Information Flow Index (IF)**:
    - The Information Flow (IF) index of a component measures the interaction between components in terms of both control and data flow. It is derived from FAN IN and FAN OUT using the formula:

$$IF(A) = [FAN\ IN(A) \times FAN\ OUT\ (A)]^2$$

    - This formula gives a numerical value representing the complexity of the interactions involving the component, with higher values indicating a higher level of information flow.

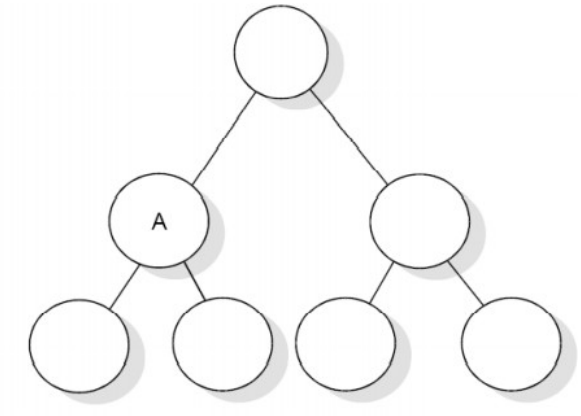



**Fig.13:** Aspects of complexity

## Steps for Deriving Information Flow Metrics:

1. **Note the Level of Each Component**:
    - Determine the hierarchy or level of each component in the system design. The system is usually organized in levels or layers based on the system architecture.
2. **Calculate FAN IN for Each Component**:
    - For each component, count the number of components that can call or pass control to it. If the component is at the highest level, assign a FAN IN of 1 (or 0 in some cases, depending on the system's conventions). This ensures that components at the top level are appropriately modeled.
3. **Calculate FAN OUT for Each Component**:
    - For each component, count the number of components that it can call or pass control to. If a component does not call any other components, its FAN OUT is assigned a value of 1.
4. **Calculate IF (Information Flow) for Each Component**:
    - Use the formula to calculate the IF value for each component:

$$IF(A) = [FAN\ IN(A) \times FAN\ OUT\ (A)]^2$$

    - This provides a numeric measure of how information flows into and out of the component.
5. **Calculate LEVEL SUM**:
    - For each level in the system design, sum the IF values for all components within that level. This gives a total measure of information flow within that particular level.
6. **Calculate SYSTEM SUM**:
    - Sum the IF values for all components across the entire system to get the SYSTEM SUM, representing the overall information flow for the entire system.
7. **Rank Components**:
    - For each level in the system design, rank the components based on their FAN IN, FAN OUT, and IF values. This helps in identifying which components have the highest levels of interactivity and complexity.
8. **Visual Representation**:
    - Plot histograms or line plots for FAN IN, FAN OUT, and IF values for each level, as well as the LEVEL SUM values for each level. This visual representation helps to understand the distribution of information flow across components and levels in the system.

## Example:

Consider a simple system with three components:

- Component A: FAN IN = 1, FAN OUT = 2
- Component B: FAN IN = 2, FAN OUT = 1
- Component C: FAN IN = 1, FAN OUT = 1

Using the formula for IF:

- IF(A) = (1 × 2)^2 = 4
- IF(B) = (2 × 1)^2 = 4
- IF(C) = (1 × 1)^2 = 1
- LEVEL SUM: The sum of IF values at the level, e.g., IF(A) + IF(B) + IF(C) = 4 + 4 + 1 = 9.
- SYSTEM SUM: The total IF value for the entire system is 9 (for this example, it is the same as the LEVEL SUM since there's only one level).

**Q19. Write another Method of Information Flow Model?**

The more sophisticated method of the Information Flow Model builds on the basic model by considering additional factors such as data elements read and written, as well as parameters passed to and from components. This method provides a more detailed understanding of the flow of information and control within a system, accounting for interactions at a more granular level.

## Metrics:

1. **FAN IN(A)**:
    - This represents the total incoming flow to a component A. It includes:
        - **a**: The number of components that call A (i.e., the number of components that invoke or depend on A).
        - **b**: The number of parameters passed to A from components higher in the hierarchy (components that depend on A).
        - **c**: The number of parameters passed to A from components lower in the hierarchy (components that A depends on).
        - **d**: The number of data elements read by component A.

    The formula for FAN IN is:

    FAN_IN(A)=a+b+c+d

    This metric provides a measure of how much external information flows into component A, including control flow (i.e., how many components call A) and data flow (i.e., how many parameters and data elements are passed to it).

2. **FAN OUT(A)**:
    - This represents the total outgoing flow from component A. It includes:
        - **e**: The number of components called by A (i.e., the components that A calls).
        - **f**: The number of parameters passed from A to components higher in the hierarchy (components that A calls and sends information to).
        - **g**: The number of parameters passed from A to components lower in the hierarchy (components that A calls and receives information from).
        - **h**: The number of data elements written to by A.

The formula for FAN OUT is:

FAN_OUT(A)=e+f+g+h

This metric provides a measure of how much information flows out of component A, including both control flow (i.e., the number of components A calls) and data flow (i.e., the number of parameters and data elements passed from it).

## Detailed Information Flow Index (IF):

Given the enhanced FAN IN and FAN OUT metrics, we can calculate the **Information Flow Index (IF)** of component A in a more sophisticated manner:

$$IF(A) = [FAN\ IN(A) \times FAN\ OUT\ (A)]^2$$

This equation remains the same as in the basic model, but now it incorporates the additional data factors (read and written elements, parameters passed to and from components) for a more comprehensive view of the information flow in the system.

## Example:

Consider a system with component A and the following values:

- **a** (number of components calling A) = 3
- **b** (number of parameters passed to A from higher components) = 2
- **c** (number of parameters passed to A from lower components) = 1
- **d** (number of data elements read by A) = 4

For FAN IN(A):

FAN_IN(A)=3+2+1+4=10

Now, for FAN OUT(A):

- **e** (number of components called by A) = 2
- **f** (number of parameters passed from A to higher components) = 3
- **g** (number of parameters passed from A to lower components) = 2
- **h** (number of data elements written to by A) = 5

For FAN OUT(A):

FAN_OUT(A)=2+3+2+5=12

Now, using the formula for the Information Flow Index (IF):

IF(A)= {12*10}^2 = 14400

This is the more sophisticated measure of information flow for component A, incorporating both control flow and data flow considerations.

**Q20. What are object-oriented metrics? State all object oriented metrics? What are the importance of these metrics?**

| | | |
|---|---|---|
| 9 | Cohesion | The degree to which the methods within a class are related to one another. |
| 10 | Coupling | Object A is coupled to object B, if and only if A sends a message to B. |

Object-oriented (OO) metrics are used to measure various aspects of an object-oriented software system. These metrics help assess the design, quality, and maintainability of an object-oriented program. Object-oriented metrics are based on the concepts of objects, classes, inheritance, polymorphism, encapsulation, and other features that define object-oriented programming.

- Measuring on class level
  - coupling
  - inheritance
  - methods
  - attributes
  - cohesion
- Measuring on system level

## Size Metrics:

- **Number of Methods per Class (NOM)**: Counts the total methods in a class.
- **Number of Attributes per Class (NOA)**: Counts the total attributes in a class.
- **Weighted Number of Methods per Class (WMC)**: Measures the complexity by summing the complexities of all methods in a class.

## Coupling Metrics:

- **Response for a Class (RFC)**: Measures the number of methods (internal and external) that can be executed in response to a message to an object.
- **Data Abstraction Coupling (DAC)**: Counts the number of abstract data types in a class.
- **Message Passing Coupling (MPC)**: Counts the number of send statements (messages) in a class.
- **Coupling Factor (CF)**: Ratio of actual coupling to the maximum possible coupling in the system.
- **Coupling Between Objects (CBO)**: Measures the number of classes to which a particular class is coupled.

## Cohesion Metrics:

- **Lack of Cohesion in Methods (LCOM)**: Measures how closely the methods in a class are related to each other.
- **Tight Class Cohesion (TCC)**: Percentage of pairs of public methods that share common attributes.
- **Loose Class Cohesion (LCC)**: Similar to TCC but also considers indirectly connected methods.

- **Information-based Cohesion (ICH)**: Measures the number of invocations between methods of the same class, weighted by the number of parameters of the invoked methods.

## Inheritance Metrics:

- **Depth of Inheritance Tree (DIT)**: The level of a class in the inheritance hierarchy.
- **Number of Children (NOC)**: The number of immediate subclasses of a class.
- **Attribute Inheritance Factor (AIF)**: The ratio of inherited attributes to total attributes in a class.
- **Method Inheritance Factor (MIF)**: The ratio of inherited methods to the total number of methods in a class system.

Importance of metrics in object-oriented software development:

1. **Quality Assurance**: Evaluate code and design quality.
2. **Project Management**: Assist in estimating timelines and resource allocation.
3. **Performance Evaluation**: Gauge component performance for optimization.
4. **Maintainability**: Identify complex or tightly coupled classes for refactoring.
5. **Cost Estimation**: Provide data for better development cost estimates.
6. **Risk Management**: Identify high-risk areas for proactive mitigation.

### Q21. Explain Use-Case Oriented Metrics?

Use-Case Oriented Metrics focus on measuring the complexity of a software system based on its use cases and actors.

- **Actors** are entities interacting with the system (users, other systems). They are classified as:
    - **Simple**: System with a defined interface (weight = 1).
    - **Average**: System interacting via text-based protocols (weight = 2).
    - **Complex**: Person interacting through a GUI (weight = 3).
- **Use Cases** define system functionality and are categorized by the number of transactions:
    - **Simple**: 3 or fewer transactions (weight = 5).
    - **Average**: 4 to 7 transactions (weight = 10).
    - **Complex**: More than 7 transactions (weight = 15).

The total weight is calculated by multiplying the number of actors or use cases by their respective weights and summing the results. This helps estimate the complexity and effort required for development and testing.

### Q22. What are the Web Engineering Project Metrics?

Web Engineering Project Metrics help assess the scale and complexity of a web project. Key metrics include:

1. **Number of Static Web Pages**: Count of pages with fixed content.
2. **Number of Dynamic Web Pages**: Pages with content that changes based on user input.
3. **Number of Internal Page Links**: Links between pages for navigation.
4. **Word Count**: Total number of words across all pages.
5. **Web Page Similarity**: Measures content uniqueness.
6. **Web Page Search and Retrieval**: Efficiency of the site's search functionality.

7. **Number of Static Content Objects**: Non-interactive elements like images or CSS.
8. **Number of Dynamic Content Objects**: Interactive elements like scripts or live data.

These metrics help estimate development time, optimize performance, and improve site design.

## Q23. Explain Metric Analysis.

Metric Analysis uses statistical techniques to interpret software metrics. Key methods include:

1. **Summary Statistics**: Mean, median, max, and min values to summarize data.
2. **Graphical Representations**: Visuals like histograms, pie charts, and box plots to display data distribution.
3. **Principal Component Analysis (PCA)**: Simplifies complex data by identifying key factors.
4. **Regression and Correlation**: Predicts values (regression) and measures relationships (correlation) between variables.
5. **Reliability Models**: Predicts future software reliability based on past performance.

## Q24. What are the problem with metric data?

Problems with metric data include:

1. **Normal Distribution**: Metrics may not follow a normal distribution, making standard statistical techniques less effective.
2. **Outliers**: Extreme values can skew results and distort analysis.
3. **Measurement Scale**: Inconsistent or inappropriate scales can lead to inaccurate comparisons.
4. **Multicollinearity**: High correlation between metrics can make it difficult to identify individual metric effects.

## Q25. What are the Pattern of Successful Applications?

The patterns of successful application of metrics include:

1. **Any metric is better than none**: Even basic metrics provide valuable insights.
2. **Automation is essential**: Automating the collection and analysis of metrics saves time and improves accuracy.
3. **Empiricism is better than theory**: Rely on actual data and observations rather than just theoretical assumptions.
4. **Use multifactor rather than single metrics**: A combination of metrics offers a more comprehensive view of the software.
5. **Don’t confuse productivity metrics with complexity metrics**: Understand the difference between measuring output and measuring the complexity of the code.
6. **Let them mature**: Metrics improve over time as they evolve and adapt to the project.
7. **Maintain them**: Continuously monitor and update metrics to keep them relevant.
8. **Let them die**: Discard outdated or ineffective metrics that no longer provide value.

Q26. Explain Hardware Reliability Curve in Detail.

There are three phases in the life of any hardware component i.e., burn-in, useful life & wear-out.

In **burn-in phase,** failure rate is quite high initially, and it starts decreasing gradually as the time progresses.

During **useful life period**, failure rate is approximately constant.

Failure rate increase in **wear-out phase** due to wearing out/aging of components. The best period is useful life period. The shape of this curve is like a "bath tub" and that is why it is known as bath tub curve. The "bath tub curve" is given in Fig.7.1.

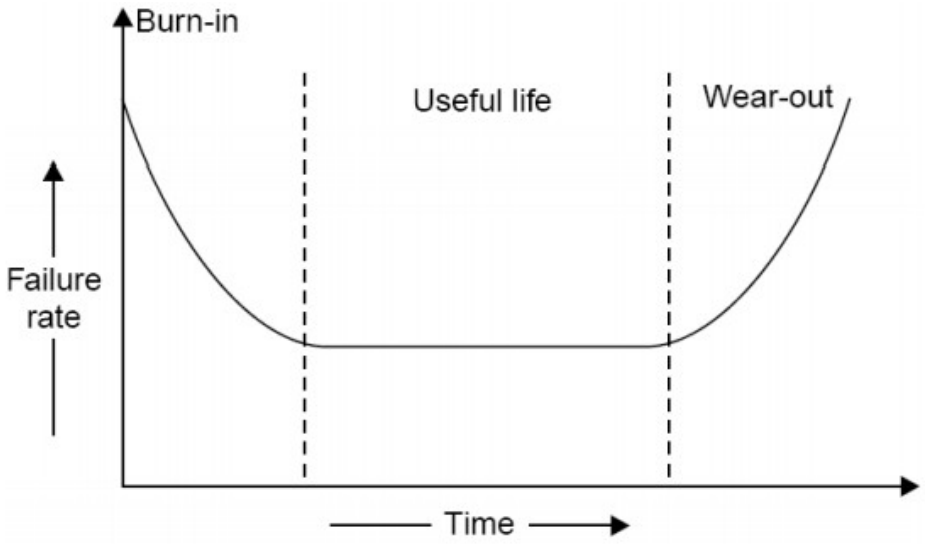



**Fig. 7.1:** Bath tub curve of hardware reliability.

Q27. Explain Software Reliability Curve in Detail.

We do not have wear out phase in software. The expected curve for software is given in fig. 7.2.

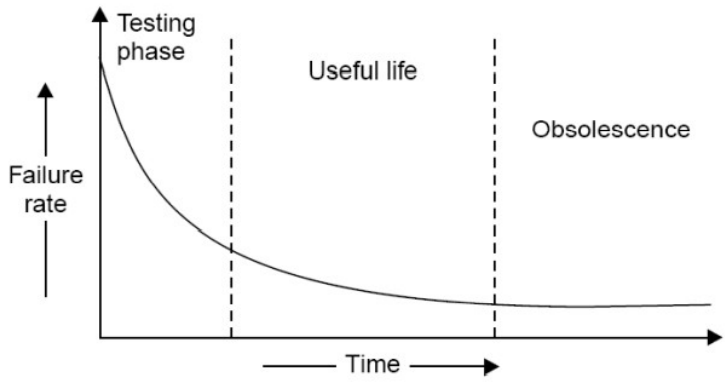



**Fig. 7.2:** Software reliability curve (failure rate versus time)

Software may be retired only if it becomes obsolete. Some of contributing factors are given below:

- ✓ change in environment
- ✓ change in infrastructure/technology
- ✓ major change in requirements
- ✓ increase in complexity
- ✓ extremely difficult to maintain
- ✓ deterioration in structure of the code
- ✓ slow execution speed
- ✓ poor graphical user interfaces

Q28. What is Software Reliability?

"Software reliability means operational reliability. Who cares how many bugs are in the program?

As per IEEE standard: "Software reliability is defined as the ability of a system or component to perform its required functions under stated conditions for a specified period of time".

Software reliability is also defined as the probability that a software system fulfills its assigned task in a given environment for a predefined number of input cases, assuming that the hardware and the inputs are free of error.

"It is the probability of a failure free operation of a program for a specified time in a specified environment".

Q29. What is Failures and Faults?

A fault is the defect in the program that, when executed under particular conditions, causes a failure.

Failure behavior is affected by two principal factors:

- ✓ the number of faults in the software being executed.
- ✓ the execution environment or the operational profile of execution.

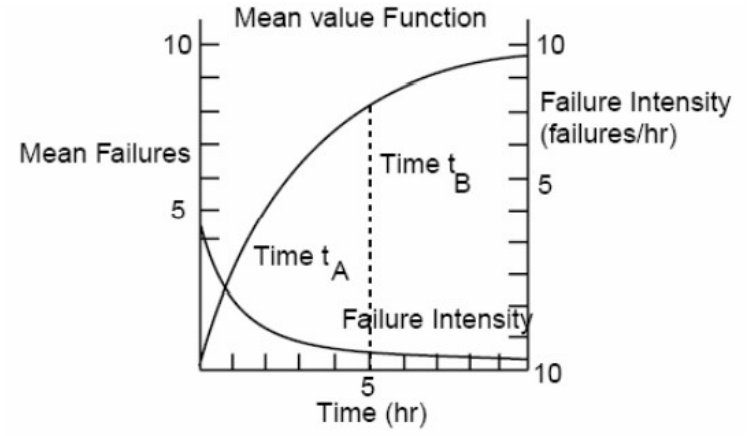



**Fig. 7.3:** Mean Value & failure intensity functions.

There are four general ways of characterising failure occurrences in time:

1. time of failure,
2. time interval between failures,
3. cumulative failure experienced up to a given time,
4. failures experienced in a time interval.

Q30. What is Environment and Operation Profile?

The environment is described by the operational profile. The proportion of runs of various types may vary, depending on the functional environment. Examples of a run type might be:

1. a particular transaction in an airline reservation system or a business data processing system,
2. a specific cycle in a closed loop control system (for example, in a chemical process industry),
3. a particular service performed by an operating system for a user.

**Q32. Show the relation between Reliability and failure intensity.**

Fig.7.7 shows how failure intensity and reliability typically vary during a test period, as faults are removed.

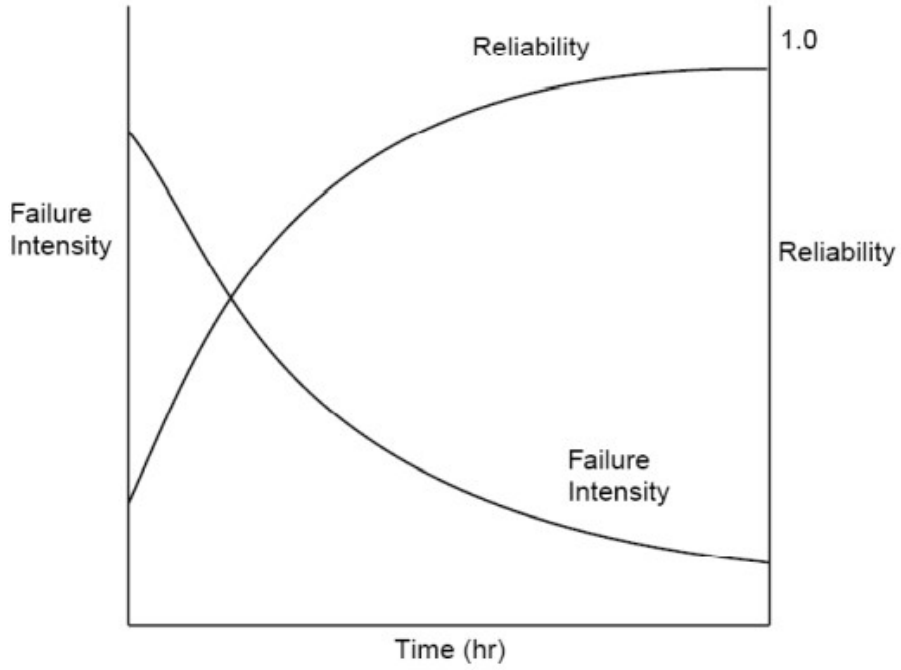



**Fig. 7.7:** Reliability and failure intensity

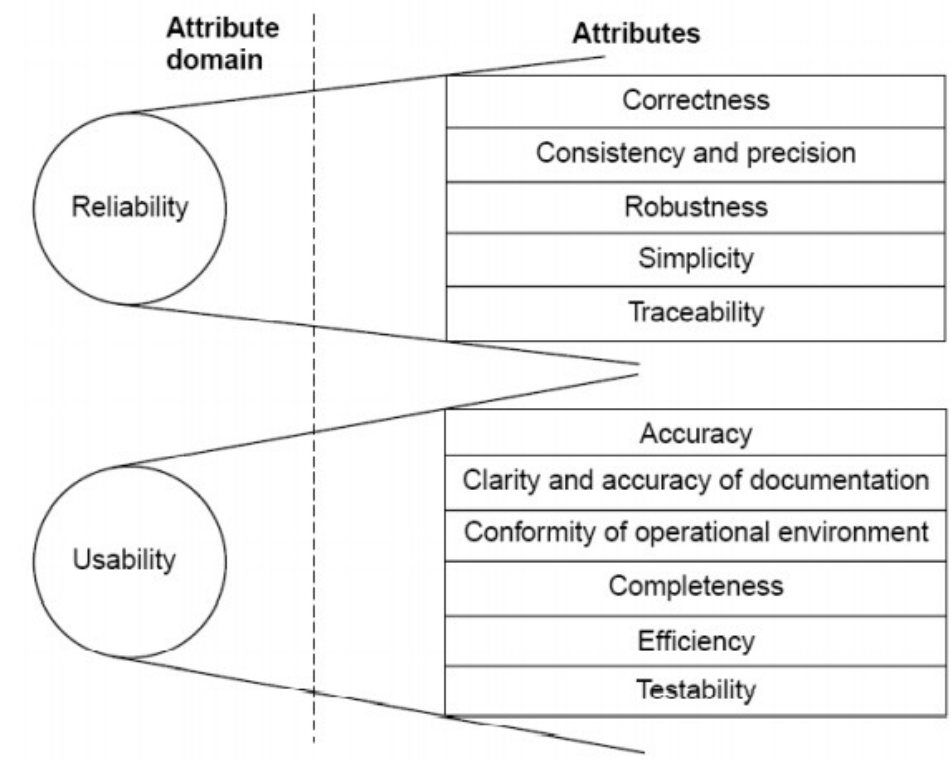
Q33. What are the uses of Reliability Studies?

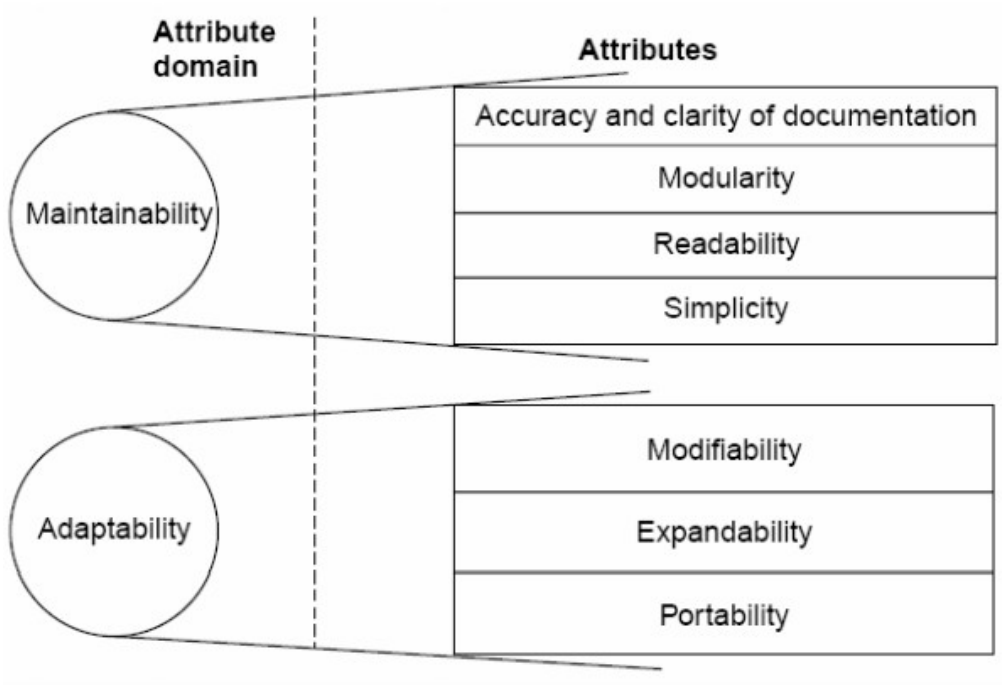
1. you can use software reliability measures to evaluate software engineering technology quantitatively.
2. software reliability measures offer you the possibility of evaluating development status during the test phases of a project.
3. one can use software reliability measures to monitor the operational performance of software and to control new features added and design changes made to the software.
4. a quantitative understanding of software quality and the various factors influencing it and affected by it enriches into the software product and the software development process.

**Q34. What do you mean by Software Quality? What are the attributes of software quality?**

**Software Quality** is defined as the degree to which a software product meets the requirements specified by the customer and its intended functionality, and performs effectively, efficiently, and reliably under specified conditions.

In other words, it refers to the conformance to specifications, the ability to meet user expectations, and the software's overall performance, reliability, and maintainability.




**Q35. Explain McCall Software Quality Model.**

The McCall Software Quality Model defines software quality based on three key perspectives: **Product Revision**, **Product Transition**, and **Product Operation**. Each perspective is associated with quality factors that influence software performance and usability.

1. **Product Operation**: Focuses on the software's functionality and performance during use.
    - Factors: Correctness, Reliability, Efficiency, Integrity, Usability.
2. **Product Revision**: Deals with the ability to modify, adapt, and maintain the software.
    - Factors: Maintainability, Flexibility, Testability.
3. **Product Transition**: Addresses the software's adaptability to new environments or platforms.

- Factors: Portability, Reusability, Interoperability.

These factors are further broken into metrics and criteria to assess specific aspects of software quality. The model ensures that both user needs and developer requirements are met effectively.

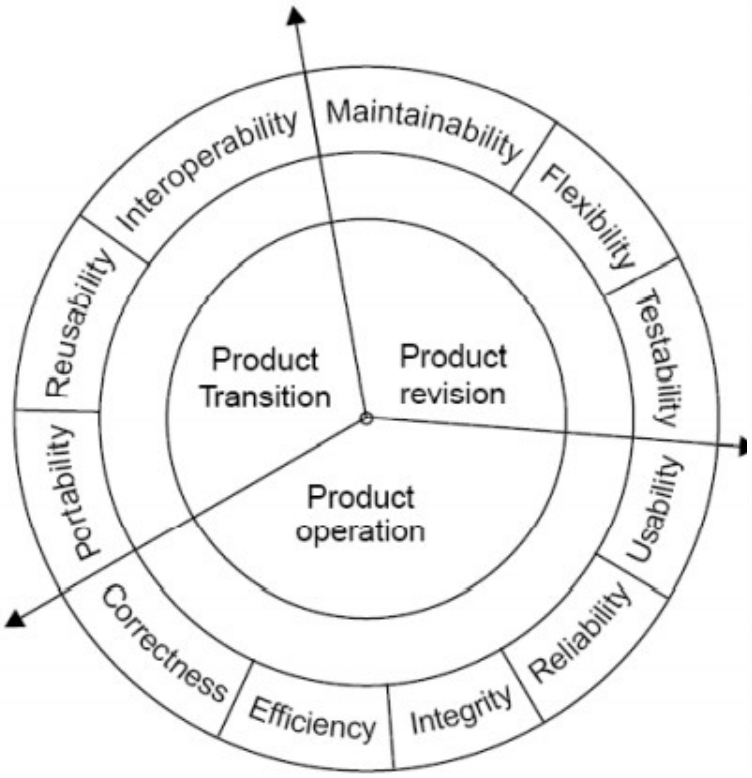



**Fig 7.9:** Software quality factors

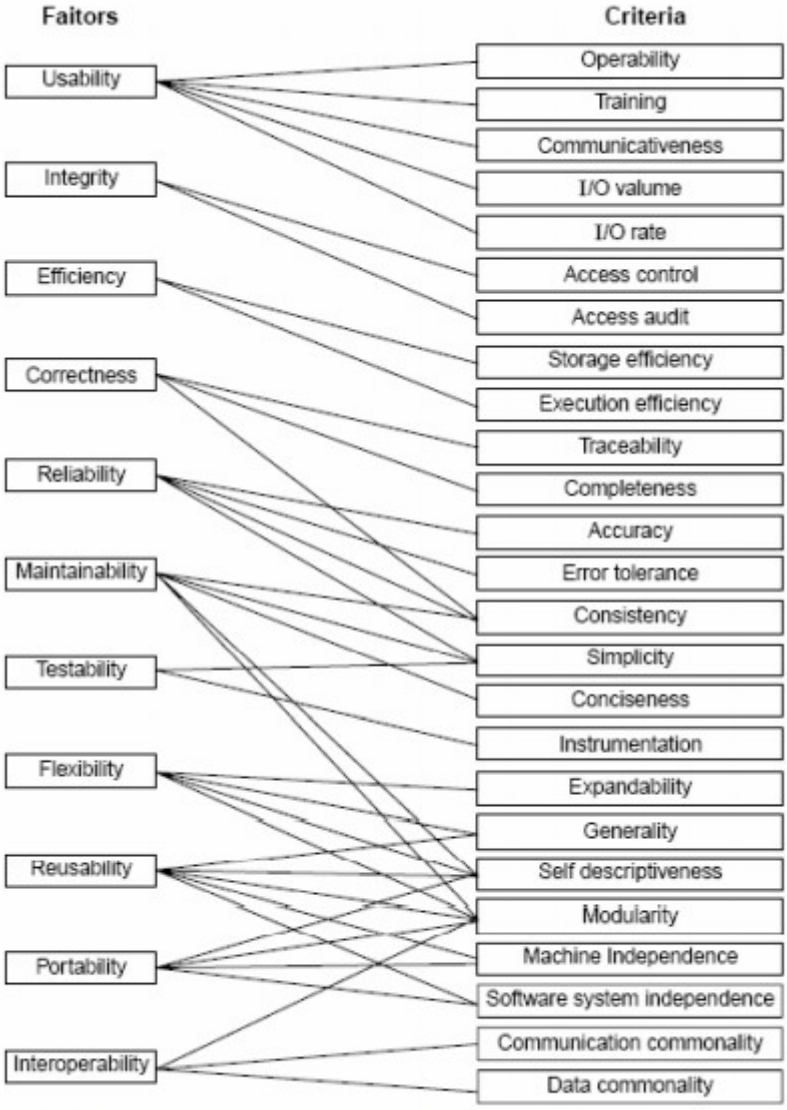



**Fig 7.10:** McCall's quality model

## Q36. Explain the Boehm Software Quality Model.

In the **Boehm Software Quality Model**, the constructs are categorized as **primary use**, **intermediate constructs**, and **primitive constructs**, forming a hierarchical structure for evaluating software quality:

1. **Primary Use**:
   These represent the top-level goals or perspectives of software quality, focusing on the software's overall purpose:
   - **As-is Utility**: How effectively the software serves its intended purpose in operation.
   - **Maintainability**: The ease of modifying the software to meet future needs.
   - **Portability**: The ability to adapt the software to different environments.

2. **Intermediate Constructs**:
   These link the primary uses to measurable attributes. They break down high-level goals into manageable sub-goals:
   - Examples include **Testability**, **Understandability**, **Modifiability**, **Efficiency**, **Reliability**, and **Human Engineering**.
3. **Primitive Constructs**:
   These are the lowest-level attributes that directly influence the intermediate constructs. They are measurable and form the foundation of the model:
   - Examples include **Code Clarity**, **Modularity**, **Error Frequency**, **Response Time**, and **Resource Utilization**.

By linking these constructs, the Boehm model enables a structured and measurable approach to evaluating software quality across various dimensions.

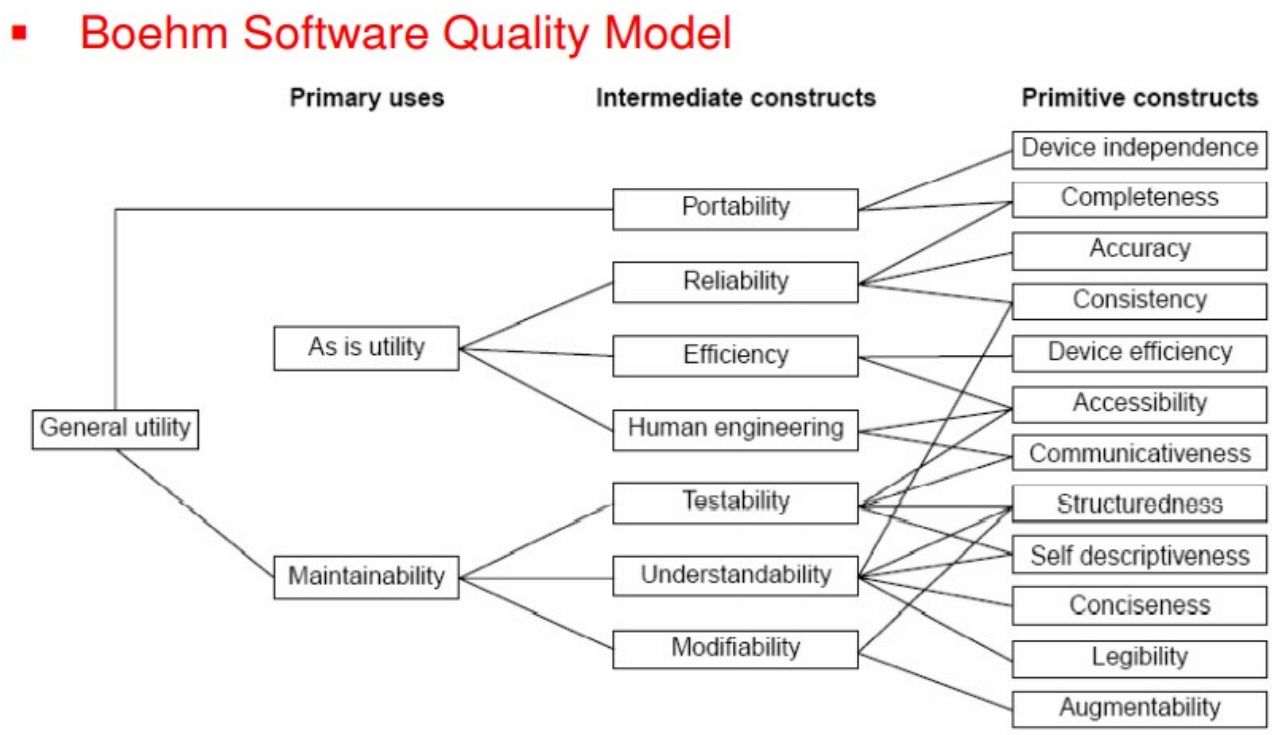



**Fig.7.11:** The Boehm software quality model

**Q37. Explain ISO 9126 Software Quality Model.**

## ISO 9126 Software Model

ISO 9126 focuses specifically on **software product quality**, breaking it into six quality characteristics and their sub-characteristics:

1. **Functionality**:
   - Suitability: How well the software meets functional requirements.
   - Accuracy: Producing correct results.
   - Interoperability: Ability to interact with other systems.
   - Security: Protecting data and operations.

2. **Reliability**:
    - Maturity: Resistance to failures.
    - Fault tolerance: Capability to maintain performance during errors.
    - Recoverability: Ability to recover from failures.
3. **Usability**:
    - Understandability: Easy to comprehend.
    - Learnability: Ease of learning.
    - Operability: Simplicity in operation.
4. **Efficiency**:
    - Time behavior: Speed of operations.
    - Resource utilization: Optimal use of resources.
5. **Maintainability**:
    - Analyzability: Easy to identify defects.
    - Changeability: Simple to modify.
    - Stability: Consistency after changes.
    - Testability: Easy to test.
6. **Portability**:
    - Adaptability: Ability to work in different environments.
    - Installability: Simplicity of installation.
    - Replaceability: Substitutability with similar systems.

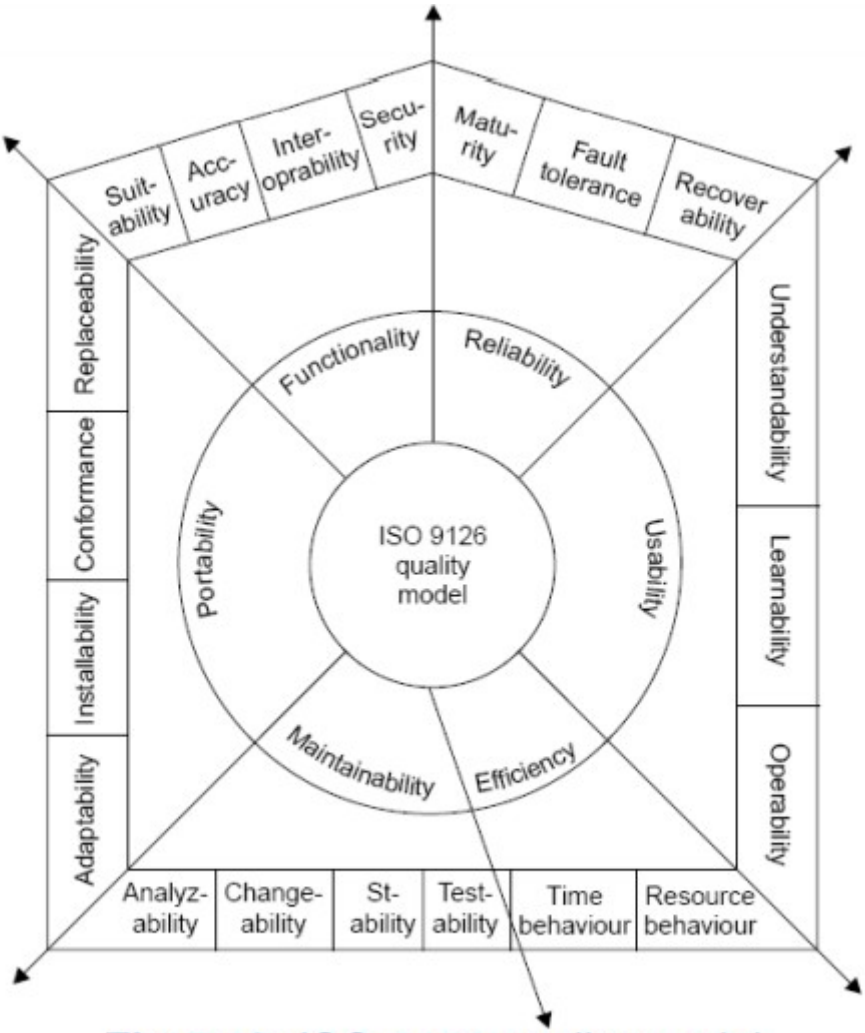



Fig.7.12: ISO 9126 quality model

**Q36. Explain the Basic and Logarithmic Poisson Model and is there significance in reliability studies?**

$$\frac{d\lambda}{d\mu} = -\lambda_0 \theta \exp(-\mu\theta) \qquad \frac{d\lambda}{d\mu} = -\theta\lambda$$

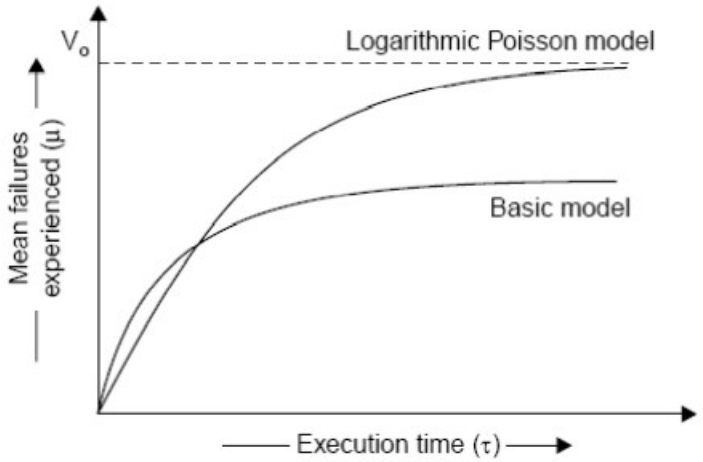



**Fig.7.19:** Relationship between

## Basic Poisson Model

**Description**:

- The basic Poisson model assumes that events (e.g., system failures) occur randomly and independently over time. It is characterized by a single parameter, λ (lambda), which represents the average rate of occurrence (mean number of events in a specified interval).
- The probability of observing k events in a time interval t is given by the formula:

$$P(X = k) = \frac{e^{-\lambda t}(\lambda t)^k}{k!}$$

where $e$ is the base of the natural logarithm, and $k!$ is the factorial of $k$.

**Significance in Reliability Studies**:

- **Failure Rate Estimation**: The basic Poisson model helps estimate the failure rate of systems, enabling organizations to understand how often failures are likely to occur.
- **Reliability Prediction**: It allows for predicting system reliability over time by modeling how the failure rate may change with time or usage.
- **Maintenance Planning**: By understanding failure patterns, organizations can optimize maintenance schedules and resource allocation, reducing downtime and costs.

**Logarithmic Poisson Model**

Logarithmic Poisson Execution Time Model

Failure Intensity

$$\lambda(\mu) = \lambda_0 \exp(-\theta\mu)$$

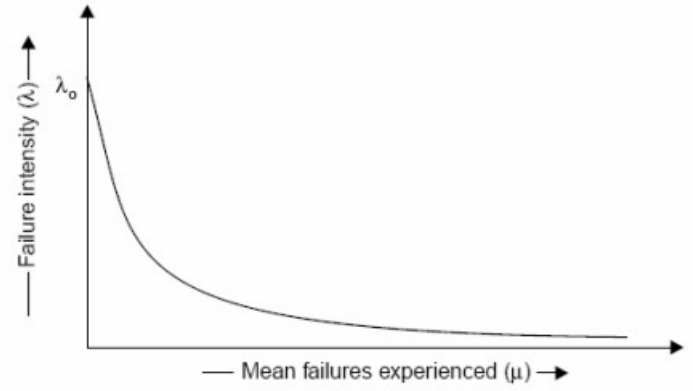



**Fig.7.18:** Relationship between μ & λ

$$\mu(\tau) = \frac{1}{\theta} Ln(\lambda_0 \theta\tau + 1)$$

$$\lambda(\tau) = \lambda_0 / (\lambda_0 \theta\tau + 1)$$

Using statistical modeling and software reliability theory, models of software failures (uncovered during testing) as a function of execution time can be developed [MUS89]. A version of the failure model, called a logarithmic Poisson execution-time model, takes the form:

$$f(t) = \frac{1}{p} \ln(l_0 pt + 1) \qquad (18\text{-}1)$$

where:

- $f(t)$ = cumulative number of failures expected to occur once the software has been tested for a certain amount of execution time, $t$.
- $l_0$ = the initial software failure intensity (failures per time unit) at the beginning of testing.
- $p$ = the exponential reduction in failure intensity as errors are uncovered and repairs are made.

The instantaneous failure intensity, $l(t)$, can be derived by taking the derivative of $f(t)$:

$$l(t) = \frac{l_0}{l_0 pt + 1} \qquad (18\text{-}2)$$

Using the relationship noted in Equation (18-2), testers can predict the drop-off of errors as testing progresses. The actual error intensity can be plotted against the predicted curve (Figure 18.3). If the actual data gathered during testing and the logarithmic Poisson execution time model are reasonably close to one another over a number of data points, the model can be used to predict the total testing time required to achieve an acceptably low failure intensity.

**Q37. Explain Jelinski-Moranda Model and Bug Seeding Model.**

The Jelinski-Moranda model is a **software reliability growth model** that assumes a constant failure rate for each fault in a system. It focuses on predicting the reliability of software systems based on the detection and correction of software faults.

- The Jelinski-Moranda Model

$$\lambda(t) = \phi(N - i + 1)$$

where

$\phi$ = Constant of proportionality

N = Total number of errors present

I = number of errors found by time interval $t_i$

The Bug Seeding Model is a fault injection technique used to assess the effectiveness of a testing process by intentionally introducing known bugs into the software and measuring how many are detected.

- The Bug Seeding Model

The bug seeding model is an outgrowth of a technique used to estimate the number of animals in a wild life population or fish in a pond.

$$\frac{N_t}{N+N_t} = \frac{n_t}{n+n_t}$$

$$\hat{N} = \frac{n}{n_t} N_t$$

$$N = \frac{n}{n_s} N_s$$

**Q38. Explain CMM (Capability Maturity Model) in Detail.**

It is a strategy for improving the software process, irrespective of the actual life cycle model use.

The **Capability Maturity Model (CMM)** is a structured framework for improving software processes within an organization. It provides a strategy for enhancing software quality, reducing risks, and increasing efficiency, irrespective of the specific software life cycle model used. The CMM defines five levels of process maturity, each building on the practices of the previous level.

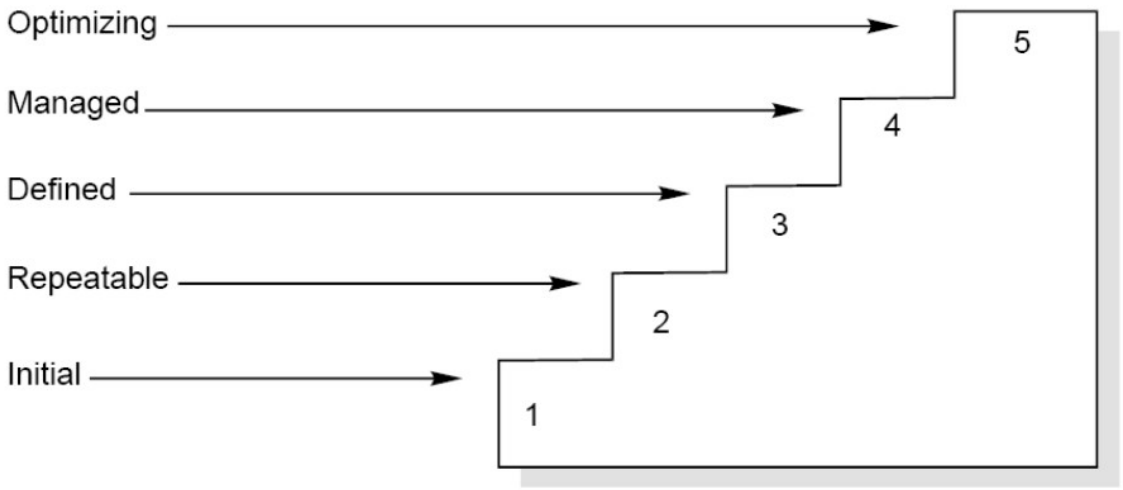



**Fig.7.23:** Maturity levels of CMM

| Maturity Level | Characterization |
|---|---|
| Initial | Adhoc Process |
| Repeatable | Basic Project Management |
| Defined | Process Definition |
| Managed | Process Measurement |
| Optimizing | Process Control |

**Fig.7.24:** The five levels of CMM

At the **Initial Level**, the software development process is chaotic and unstructured, with no defined procedures or consistency. Success heavily relies on the skills and efforts of individual team members rather than on established organizational processes.

## Key Characteristics:

1. **Ad Hoc Processes:**
   Processes are reactive and improvised, with no predefined or repeatable methods.
2. **Unpredictability:**
   Project outcomes are highly unpredictable, with frequent schedule slippages and cost overruns.
3. **Dependency on Individuals:**
   Success depends on individual efforts and heroics, making it hard to replicate past successes.

4. **No Process Control:**
   There are no established metrics or controls to monitor project progress or quality.
5. **High Risk of Failure:**
   Lack of structure often leads to inconsistent performance, with a high probability of project failure.

## ▪ Key Process Areas

The key process areas at level 2 focus on the software project's concerns related to establishing basic project management controls, as summarized below:

| | |
|---|---|
| Requirements Management (RM) | Establish a common relationship between the customer requirements and the developers in order to understand the requirements of the project. |
| Software Project Planning (PP) | Establish reasonable plans for performing the software engineering and for managing the software project. |
| Software Project Tracking and Oversight (PT) | Establish adequate visibility into actual progress so that management can take effective actions when the software project's performance deviates significantly from the software plans. |
| Software Subcontract Management (SM) | Select qualified software subcontractors and manage them effectively. |
| Software Quality Assurance (QA) | Provide management with appropriate visibility into the process being used by the software project and of the products being built. |
| Software Configuration Management (CM) | Establish and maintain the integrity of the products of the software project throughout the project's software life cycle. |

The key process areas at level 3 address both project and organizational issues, as summarized below:

| | |
|---|---|
| Organization Process Focus (PF) | Establish the organizational responsibility for software process activities that improve the organization's overall software process capability. |
| Organization Process Definition (PD) | Develop and maintain a usable set of software process assets that improve process performance across the projects and provide a basis for cumulative, long-term benefits to the organization. |
| Training Program (TP) | Develop the skills and knowledge of individuals so that they can perform their roles effectively and efficiently. |
| Integrated Software Management (IM) | Integrate the software engineering and management activities into a coherent, defined software process that is tailored from the organization's standard software process and related process assets. |

| | |
|---|---|
| Software Product Engineering (PE) | Consistently perform a well-defined engineering process that integrates all the software engineering activities to produce correct, consistent software products effectively and efficiently. |
| Inter group Coordination (IC) | Establish a means for the software engineering group to participate actively with the other engineering groups so the project is better able to satisfy the customer's needs effectively and efficiently. |
| Peer Reviews (PR) | Remove defects from the software work products early and efficiently. An important corollary effect is to develop a better understanding of the software work products and of the defects that can be prevented. |

The key process areas at level 4 focus on establishing a quantitative understanding of both the software process and the software work products being built, as summarized below:

| | |
|---|---|
| Quantitative Process Management (QP) | Control the process performance of the software project quantitatively. |
| Software Quality Management (QM) | Develop a quantitative understanding of the quality of the project's software products and achieve specific quality goals. |

The key process areas at level 5 cover the issues that both the organization and the projects must address to implement continuous and measurable software process improvement, as summarized below:

| | |
|---|---|
| Defect Prevention (DP) | Identify the causes of defects and prevent them from recurring. |
| Technology Change Management (TM) | Identify beneficial new technologies (i.e., tools, methods, and processes) and transfer them into the organization in an orderly manner. |
| Process Change Management (PC) | Continually improve the software processes used in the organization with the intent of improving software quality, increasing productivity, and decreasing the cycle time for product development. |

- ## Common Features

| | |
|---|---|
| Commitment to Perform (CO) | Describes the actions the organizations must take to ensure that the process is established and will endure. It includes practices on policy and leadership. |
| Ability to Perform (AB) | Describes the preconditions that must exist in the project or organization to implement the software process competently. It includes practices on resources, organizational structure, training, and tools. |
| Activities Performed (AC) | Describes the role and procedures necessary to implement a key process area. It includes practices on plans, procedures, work performed, tracking, and corrective action. |

| | |
|---|---|
| Measurement and Analysis (ME) | Describes the need to measure the process and analyze the measurements. It includes examples of measurements. |
| Verifying Implementation (VE) | Describes the steps to ensure that the activities are performed in compliance with the process that has been established. It includes practices on management reviews and audits. |

**Q39. Explain ISO 9001 Software Quality Model. Differentiate Between ISO 9001 and CMM in tabular form.**

## ISO 9001 Software Quality Model

ISO 9001 is an internationally recognized standard for quality management systems (QMS), including software quality management. It provides guidelines to ensure that organizations consistently produce products and services meeting customer and regulatory requirements.

**Key Features of ISO 9001:**

1. **Management Responsibility**: Focus on leadership and commitment.
2. **Quality System**: Documentation and implementation of a quality management system.
3. **Contract Review**: Ensuring customer requirements are met before signing contracts.
4. **Design Control**: Systematic control of the design process to ensure quality.
5. **Document Control**: Proper management of documents to ensure accessibility and updates.
6. **Purchasing**: Ensuring the quality of purchased materials.
7. **Product Identification and Traceability**: Tracking products from development to delivery.
8. **Process Control**: Managing processes to ensure quality standards.
9. **Inspection and Testing**: Verifying product compliance through testing.
10. **Nonconforming Product Control**: Handling defective products systematically.
11. **Corrective Action**: Identifying and resolving root causes of issues.
12. **Internal Quality Audits**: Regular reviews of the QMS.
13. **Training**: Ensuring employees are competent for their roles.
14. **Statistical Techniques**: Using statistical methods to ensure process reliability.

## Differences Between ISO 9001 and CMM

| Aspect | ISO 9001 | CMM (Capability Maturity Model) |
|---|---|---|
| **Focus** | Quality management system for organizations. | Process maturity and continuous improvement. |
| **Purpose** | Ensures compliance with quality standards. | Improves software process capabilities. |
| **Scope** | Broad, applies to various industries. | Specific to software development processes. |
| **Key Features** | Management responsibility, process control. | Maturity levels, process improvement focus. |
| **Emphasis** | Meeting standards and consistency. | Continuous process improvement. |
| **Structure** | Requirements-based (document compliance). | Evolutionary (maturity levels). |

| Aspect | ISO 9001 | CMM (Capability Maturity Model) |
|---|---|---|
| **Outcome** | Certification for quality management. | Assessment of process maturity. |
| **Biggest Similarity** | "Say what you do; do what you say." | Same principle applies. |

**Key Similarities**

- Both emphasize clear documentation and adherence to processes.
- Both aim for reliable, high-quality output.
- The bottom line for both is ensuring processes are defined, implemented, and continuously improved.

Software Engineering | Unit – 4

Q1. What is Testing?

“Testing is the process of executing a program with the intent of finding errors.”

Q2. Why should we do testing?

Although software testing is itself an expensive activity, yet launching of software without testing may lead to cost potentially much higher than that of testing, specially in systems where human safety is involved. In the software life cycle the earlier the errors are discovered and removed, the lower is the cost of their removal.

Q3. What is fault?

A fault is the representation of an error, where representation is the mode of expression, such as narrative text, data flow diagrams, ER diagrams, source code etc. Defect is a good synonym for fault.

Q4. What is failure?

A failure occurs when a fault executes. A particular fault may cause different failures, depending on how it has been exercised.

Q5. What is a Test Case?

Test case describes an input description and an expected output description.

| Test Case ID | |
|---|---|
| Section-I<br>(Before Execution) | Section-II<br>(After Execution) |
| Purpose : | Execution History: |
| Pre condition: (If any) | Result: |
| Inputs: | If fails, any possible reason (Optional); |
| Expected Outputs: | Any other observation: |
| Post conditions: | Any suggestion: |
| Written by: | Run by: |
| Date: | Date: |

**Fig. 2:** Test case template

The set of test cases is called a test suite. Hence any combination of test cases may generate a test suite.

Q6. What is Verification?

Verification is the process of evaluating a system or component to determine whether the products of a given development phase satisfy the conditions imposed at the start of that phase.

Q7. What is Validation?

Validation is the process of evaluating a system or component during or at the end of development process to determine whether it satisfies the specified requirements .
Testing= Verification+Validation

Q8. What is Acceptance Testing?

The term Acceptance Testing is used when the software is developed for a specific customer. A series of tests are conducted to enable the customer to validate all requirements. These tests are conducted by the end user / customer and may range from adhoc tests to well planned systematic series of tests.

Q9. What is Functional Testing?

Functional testing is defined as a type of testing that verifies that each function of the software application works in conformance with the requirement and specification. This testing is not concerned with the source code of the application. Each functionality of the software application is tested by providing appropriate test input, expecting the output, and comparing the actual output with the expected output.

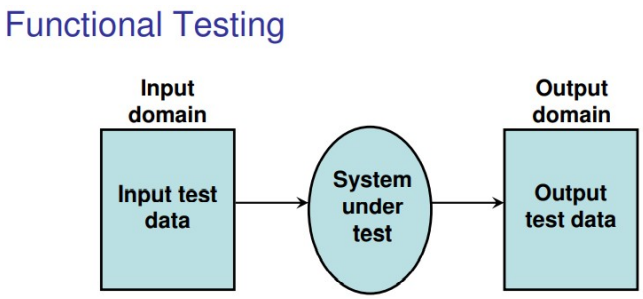



Fig. 3: Black box testing

Q10. What is Boundary Value Analysis?

Boundary Value Analysis is based on testing the boundary values of valid and invalid partitions. The behavior at the edge of the equivalence partition is more likely to be incorrect than the behavior within the partition, so boundaries are an area where testing is likely to yield defects.

It checks for the input values near the boundary that have a higher chance of error. Every partition has its maximum and minimum values and these maximum and minimum values are the boundary values of a partition.

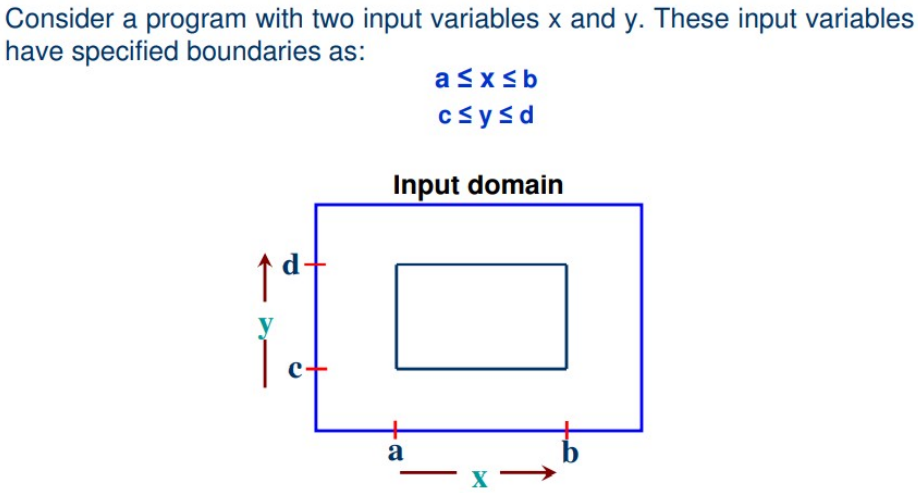



**Fig.4:** Input domain for program having two input variables

Q11. What is Robustness Testing?

It is nothing but the extension of boundary value analysis. Here, we would like to see, what happens when the extreme values are exceeded with a value slightly greater than the maximum, and a value slightly less than minimum. It means, we want to go outside the legitimate boundary of input domain. This extended form of boundary value analysis is called robustness testing.

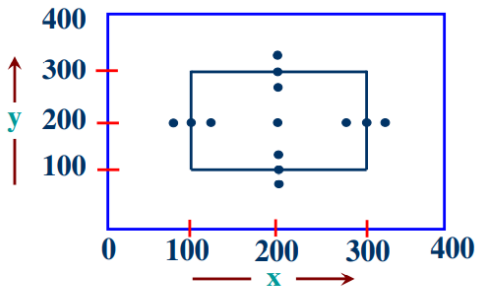



**Fig. 8.6:** Robustness test cases for two variables x and y with range [100,300] each

Q12. What is Worst Case testing?

Boundary Value analysis uses the critical fault assumption and therefore only tests for a single variable at a time assuming its extreme values. By disregarding this assumption we are able to test the outcome if more than one variable were to assume its extreme value. In an electronic circuit this is called Worst Case Analysis. In Worst-Case testing we use this idea to create test cases.

Q13. What is Equivalence Class Testing?

Equivalence Class Testing, which is also known as **Equivalence Class Partitioning (ECP)** and **Equivalence Partitioning**, is an important software testing technique used by the team of testers for grouping and partitioning of the test input data, which is then used for the purpose of testing the software product into a number of different classes.

These different classes resemble the specified requirements and common behaviour or attribute(s) of the aggregated inputs. Thereafter, the test cases are designed and created based on each class attribute(s) and one element or input is used from each class for the test execution to validate the software functioning, and simultaneously validates the similar working of the software product for all the other inputs present in their respective classes.

**Two steps are required to implementing this method:**

1. The equivalence classes are identified by taking each input condition and partitioning it into valid and invalid classes. For example, if an input condition specifies a range of values from 1 to 999, we identify one valid equivalence class [1<item<999]; and two invalid equivalence classes [item<1] and [item>999].

2. Generate the test cases using the equivalence classes identified in the previous step. This is performed by writing test cases covering all the valid equivalence classes. Then a test case is written for each invalid equivalence class so that no test contains more than one invalid class. This is to ensure that no two invalid classes mask each other.

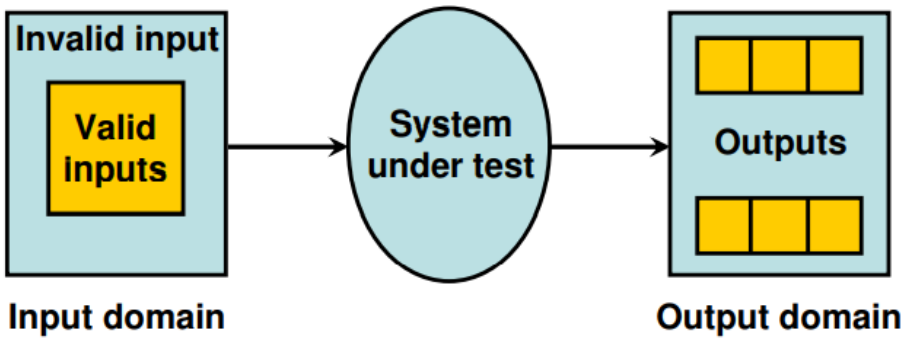



Fig. 7: Equivalence partitioning

Most of the time, equivalence class testing defines classes of the input domain. However, equivalence classes should also be defined for output domain. Hence, we should design equivalence classes based on input and output domain.

Q14. What is Decision Table Testing?

Decision tables are used in various engineering fields to represent complex logical relationships. This testing is a very effective tool in testing the software and its requirements management. The output may be dependent on many input conditions and decision tables give a tabular view of various combinations of input conditions and these conditions are in the form of True(T) and False(F). Also, it provides a set of conditions and its corresponding actions required in the testing.

| Condition Stub | | Entry | | | | | | |
|---|---|---|---|---|---|---|---|---|
| | $C_1$ | True | | | | False | | |
| | $C_2$ | True | | False | | True | | False |
| | $C_3$ | True | False | True | False | True | False | --- |
| Action Stub | $a_1$ | X | X | | | X | | |
| | $a_2$ | X | | X | | | X | |
| | $a_3$ | | X | | | X | | |
| | $a_4$ | | | | X | | X | X |

Table 2: Decision table terminology

**In software testing, the decision table has 4 parts which are divided into portions and are given below :**

1. **Condition Stubs :** The conditions are listed in this first upper left part of the decision table that is used to determine a particular action or set of actions.
2. **Action Stubs :** All the possible actions are given in the first lower left portion (i.e, below condition stub) of the decision table.
3. **Condition Entries :** In the condition entry, the values are inputted in the upper right portion of the decision table. In the condition entries part of the table, there are multiple rows and columns which are known as Rule.

4. **Action Entries :** In the action entry, every entry has some associated action or set of actions in the lower right portion of the decision table and these values are called outputs.

Q15. What is Cause Effect Graphing Technique?

**Cause Effect Graphing based technique** is a technique in which a graph is used to represent the situations of combinations of input conditions. The graph is then converted to a decision table to obtain the test cases. Cause-effect graphing technique is used because boundary value analysis and equivalence class partitioning methods do not consider the combinations of input conditions. But since there may be some critical behaviour to be tested when some combinations of input conditions are considered, that is why cause-effect graphing technique is used.

The basic notation for the graph is shown in fig. 8

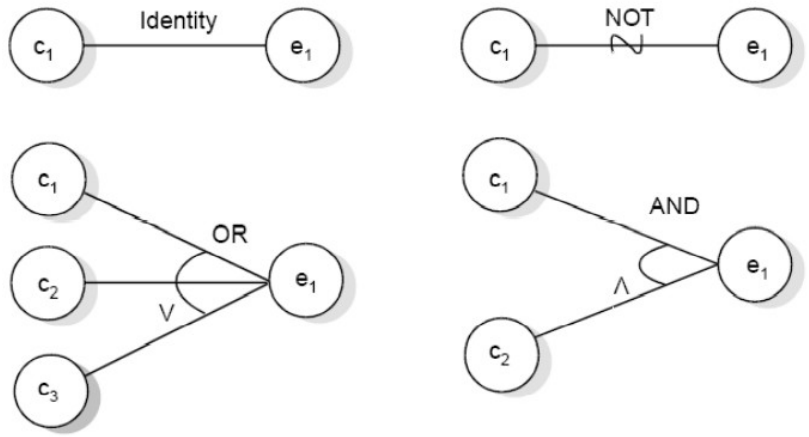
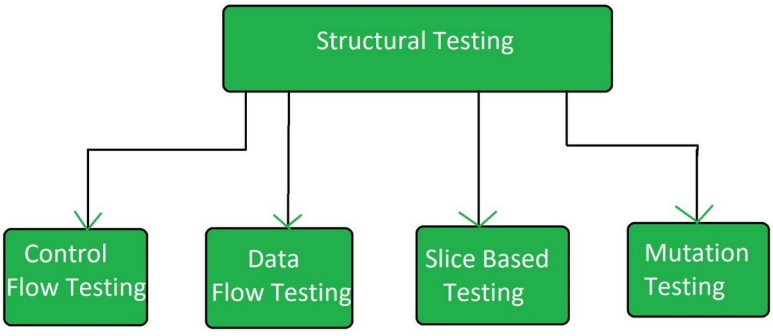



**Fig.8. 8 : Basic cause effect graph symbols**

Q16. What is Structural Testing?

**Structural testing** is a type of software testing that uses the internal design of the software for testing or in other words the software testing which is performed by the team which knows the development phase of the software, is known as structural testing.

Structural testing is related to the internal design and implementation of the software i.e. it involves the development team members in the testing team. It tests different aspects of the software according to its types. Structural testing is just the opposite of behavioral testing.



Q17. What is Path Testing?

Structural testing is related to the internal design and implementation of the software i.e. it involves the development team members in the testing team. It tests different aspects of the software according to its types. Structural testing is just the opposite of behavioral testing.

This type of testing involves:
1. generating a set of paths that will cover every branch in the program.
2. finding a set of test cases that will execute every path in the set of program paths

Q18. What is Control Flow Testing?

Control flow testing is a type of structural testing that uses the programs's control flow as a model. The entire code, design and structure of the software have to be known for this type of testing. Often this type of testing is used by the developers to test their own code and implementation. This method is used to test the logic of the code so that required result can be obtained.

The control flow of a program can be analysed using a graphical representation known as flow graph. The flow graph is a directed graph in which nodes are either entire statements or fragments of a statement, and edges represents flow of control.

Q19. What is Data Flow Testing?

It uses the control flow graph to explore the unreasonable things that can happen to data. The detection of data flow anomalies are based on the associations between values and variables. Without being initialized usage of variables. Initialized variables are not used once.

i. Statements where variables receive values.
ii. Statements where these values are used or referenced.

As we know, variables are defined and referenced throughout the program. We may have few define/ reference anomalies:

i. A variable is defined but not used/ referenced.
ii. A variable is used but never defined.
iii. A variable is defined twice before it is used.

**Definitions**

The definitions refer to a program P that has a program graph G(P) and a set of program variables V. The G(P) has a single entry node and a single exit node. The set of all paths in P is PATHS(P)

(i) **Defining Node:** Node n ∈ G(P) is a defining node of the variable $v \in V$, written as DEF ($v$, $n$), if the value of the variable $v$ is defined at the statement fragment corresponding to node $n$.

(ii) **Usage Node:** Node n ∈ G(P) is a usage node of the variable $v \in V$, written as USE ($v$, $n$), if the value of the variable $v$ is used at statement fragment corresponding to node $n$. A usage node USE ($v$, n) is a predicate use (denote as p) if statement n is a predicate statement otherwise USE ($v$, n) is a computation use (denoted as $c$).

(iii) **Definition use:** A definition use path with respect to a variable v (denoted du-path) is a path in PATHS(P) such that, for some $v \in V$, there are define and usage nodes DEF(v, m) and USE(v, n) such that m and n are initial and final nodes of the path.

(iv) **Definition clear :** A definition clear path with respect to a variable v (denoted dc-path) is a definition use path in PATHS(P) with initial and final nodes DEF (*v*, m) and USE (*v*, n), such that no other node in the path is a defining node of *v*.

The du-paths and dc-paths describe the flow of data across source statements from points at which the values are defined to points at which the values are used. The du-paths that are not definition clear are potential trouble spots.

Hence, our objective is to find all du-paths and then identity those du-paths which are not dc-paths. The steps are given in Fig. 27. We may like to generate specific test cases for du-paths that are not dc-paths.

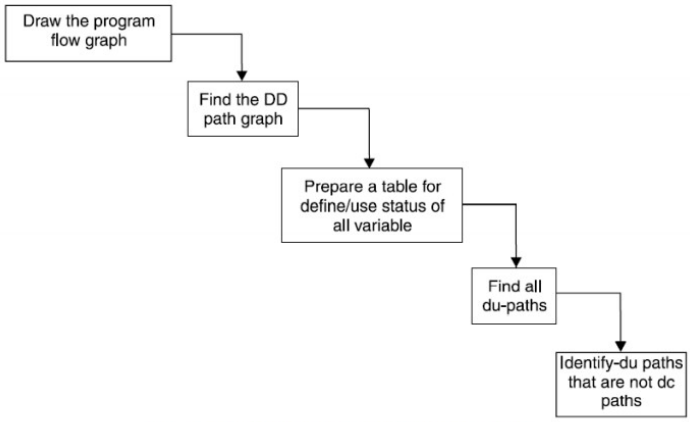



**Fig. 27 : Steps for data flow testing**

Q20. What is Mutation Testing?

**Mutation Testing** is a type of Software Testing that is performed to design new software tests and also evaluate the quality of already existing software tests. Mutation testing is related to modification a program in small ways. It focuses to help the tester develop effective tests or locate weaknesses in the test data used for the program.

Mutation testing is a fault based technique that is similar to fault seeding, except that mutations to program statements are made in order to determine properties about test cases. it is basically a fault simulation technique.

Multiple copies of a program are made, and each copy is altered; this altered copy is called a mutant. Mutants are executed with test data to determine whether the test data are capable of detecting the change between the original program and the mutated program.

Q21. Difference between is Unit Testing and Integration Testing.

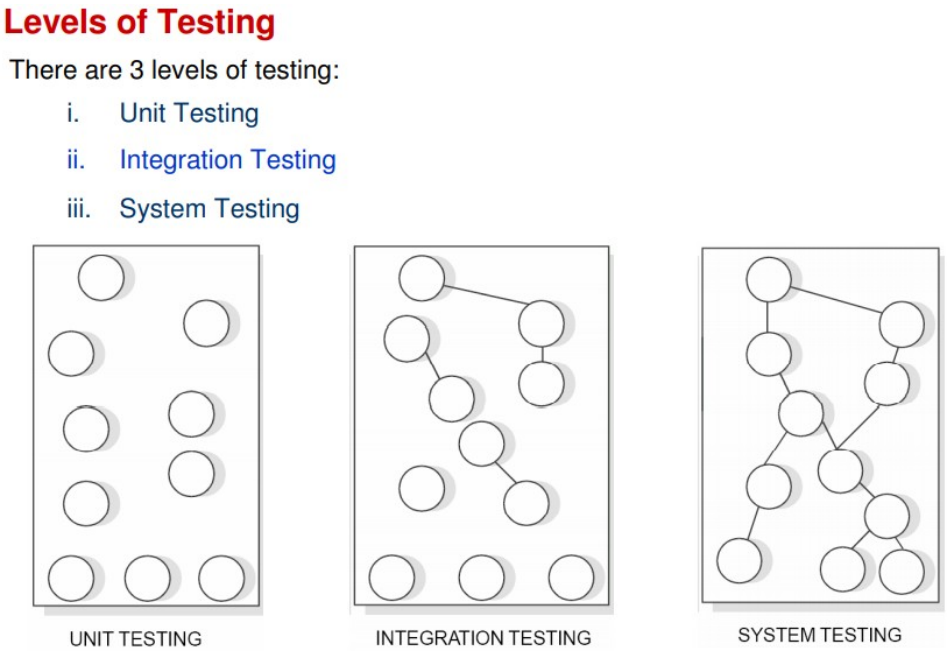
| S. No. | Unit Testing | Integration Testing |
|---|---|---|
| 1. | In unit testing, each module of the software is tested separately. | In integration testing, all modules of the software are tested combined. |
| 2. | In unit testing tester knows the internal design of the software. | Integration testing doesn’t know the internal design of the software. |
| 3. | Unit testing is performed first of all testing processes. | Integration testing is performed after unit testing and before system testing. |
| 4. | Unit testing is white box testing. | Integration testing is black box testing. |
| 5. | Unit testing is performed by the developer. | Integration testing is performed by the tester. |
| 6. | Detection of defects in unit testing is easy. | Detection of defects in integration testing is difficult. |
| 7. | It tests parts of the project without waiting for others to be completed. | It tests only after the completion of all parts. |
| 8. | Unit testing is less costly. | Integration testing is more costly. |
| 9. | Unit testing is responsible to observe only the functionality of the individual units. | Error detection takes place when modules are integrated to create an overall system. |
| 10. | Module specification is done initially. | Interface specification is done initially. |
| 11. | The proper working of your code with the external dependencies is not ensured by unit testing. | The proper working of your code with the external dependencies is ensured by integration testing. |
| 12. | Maintenance is cost effective. | Maintenance is expensive. |
| 13. | Fast execution as compared to integration testing. | Its speed is slow because of the integration of modules. |
| 14. | Unit testing results in in-depth exposure to the code. | Integration testing results in the integration structure’s detailed visibility. |

Q22. Difference Between Unit Testing and System Testing

| Aspects | Unit Testing | System Testing |
|---|---|---|
| **Scope of testing** | Tests individual units or components of the software application in isolation | Tests the complete system or software application |
| **Timing of testing** | Typically automated and performed by developers during the coding phase | Typically manual and performed by independent testers after the completion of integration testing |
| **Focus of testing** | Focuses on verifying the functionality of each unit or component of the software application | Focuses on verifying the functionality of the system as a whole |
| **Testing environment** | Tests are executed in a controlled environment, typically using mock objects or test doubles to isolate the unit being tested | Tests are executed in a production-like environment to simulate real-world usage scenarios |

| Aspects | Unit Testing | System Testing |
|---|---|---|
| | | |
| **Testing frameworks** | Uses unit test frameworks such as JUnit, NUnit, or PHPUnit to automate the testing process | Uses system testing frameworks such as Selenium, HP UFT, or IBM Rational Functional Tester to automate the testing process |
| **Purpose of testing** | Enables early detection and elimination of defects in the code | Enables validation of the entire software application to ensure it meets the specified requirements |
| **Skills required** | Requires a solid understanding of the code and the ability to write effective test cases | Requires a comprehensive understanding of the software application and the ability to write effective test cases |
| **Feedback** | Provides fast feedback on the quality of individual units or components | Provides a comprehensive understanding of the quality and readiness of the entire software application |
| **Execution** | Typically executed by developers in a continuous integration and delivery (CI/CD) pipeline | Typically executed by independent testers in a separate testing environment |
| **Types of testing** | Tests for both functional and non-functional aspects of the software application | Tests for both functional and non-functional aspects of the software application, including performance, security, and usability testing |

Q23. What are the levels of testing?



Q24. What is Debugging?

The goal of testing is to identify errors (bugs) in the program. The process of testing generates symptoms, and a program's failure is a clear symptom of the presence of an error. After getting a symptom, we begin to investigate the cause and place of that error. After identification of place, we examine that portion to identify the cause of the problem. This process is called debugging.

Q25. What is Induction Debugging Approach?

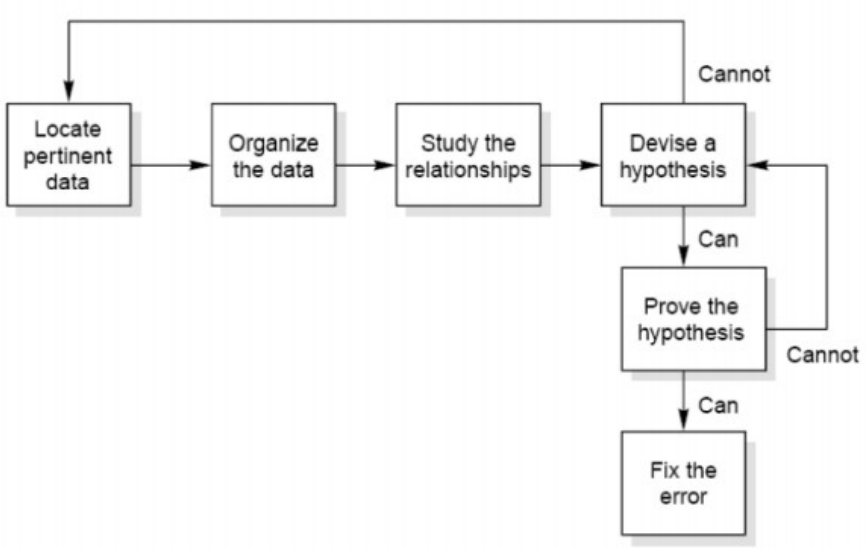



Fig. 32 : The inductive debugging process

Q26. What is Deduction Debugging Approach?

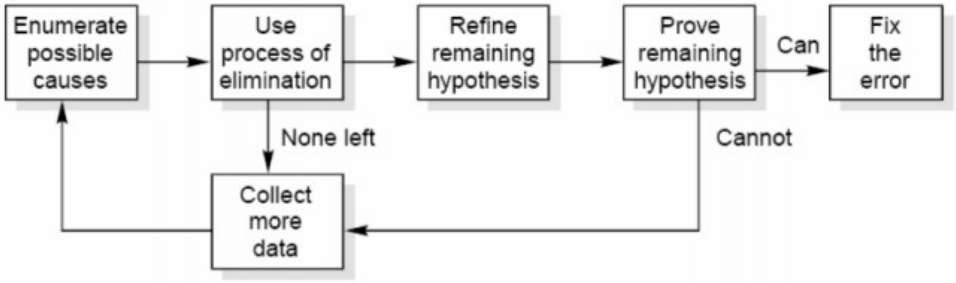



Fig. 33 : The inductive debugging process

Q27. What are Testing Tools?

One way to improve the quality & quantity of testing is to make the process as pleasant as possible for the tester. This means that tools should be as concise, powerful & natural as possible.
The two broad categories of software testing tools are :

- Static
- Dynamic

Q28. What are the types of testing tools?

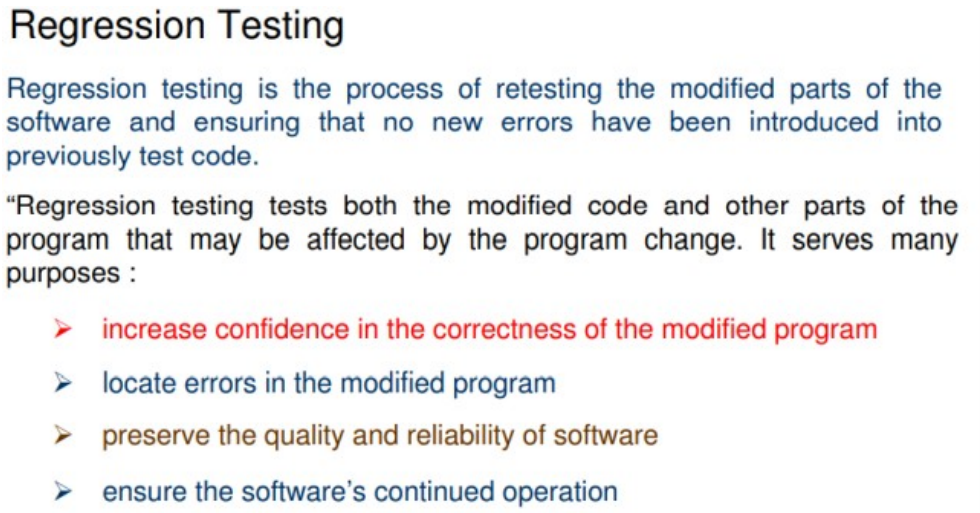
**Static Test Tools:** Static test tools are used to work on the static testing processes. In the testing through these tools, the typical approach is taken. These tools do not test the real execution of the software. Certain input and output are not required in these tools. Static test tools consist of the following:

- **Flow analyzers:** Flow analyzers provides flexibility in the data flow from input to output.
- **Path Tests:** It finds the not used code and code with inconsistency in the software.
- **Coverage Analyzers:** All rationale paths in the software are assured by the coverage analyzers.
- **Interface Analyzers:** They check out the consequences of passing variables and data in the modules.

**Dynamic Test Tools:** Dynamic testing process is performed by the dynamic test tools. These tools test the software with existing or current data. Dynamic test tools comprise the following:

- **Test driver:** The test driver provides the input data to a module-under-test (MUT).
- **Test Beds:** It displays source code along with the program under execution at the same time.
- **Emulators:** Emulators provide the response facilities which are used to imitate parts of the system not yet developed.
- **Mutation Analyzers:** They are used for testing the fault tolerance of the system by knowingly providing the errors in the code of the software.

Q29. What is Regression Testing?



Q30. Difference between Development and Regression Testing?

- Development Testing Versus Regression Testing

| Sr. No. | Development testing | Regression testing |
|---|---|---|
| 1. | We create test suites and test plans | We can make use of existing test suite and test plans |
| 2. | We test all software components | We retest affected components that have been modified by modifications. |
| 3. | Budget gives time for testing | Budget often does not give time for regression testing. |
| 4. | We perform testing just once on a software product | We perform regression testing many times over the life of the software product. |
| 5. | Performed under the pressure of release date of the software | Performed in crisis situations, under greater time constraints. |

Q31. Difference between Functional Testing and Structural Testing?

| **Structural Testing** | **Functional Testing** |
|---|---|
| This test evaluates the code structure or internal implementation of the code. | This test checks whether the software is functioning in accordance with functional requirements and specifications. |
| It is also known as white-box or clear-box testing as thorough knowledge and access of the code is required. | It is also known as black-box testing as no knowledge of the internal code is required. |
| Finds errors in the internal code logic and data structure usage. | It ensures that the system is error-free. |
| It does not ensure that the user requirements are met. | It is a quality assurance testing process ensuring the business requirements are met. |
| Performed the entire software in accordance with the system requirements. | Functional testing checks that the output is given as per expected. |
| Testing teams usually require developers to perform structural testing. | A QA professional can simply perform this testing. |
| Perform on low-level modules/software components. | The functional testing tool works on event analysis methodology. |
| It provides information on improving the internal structure of the application. | It provides information that prevents business loss. |
| Structural testing tools follow data analysis methodology. | Functional testing tool works on event analysis methodology. |
| Writing a structural test case requires understanding the coding aspects of the application. | Before writing a functional test case, a tester is required to understand the application’s requirements. |
| It examines how well modules communicate with one another. | It examines how well a system satisfies the business needs or the SRS. |

**Q32. What is Cyclomatic Complexity Number?**

Cyclomatic Complexity is a software metric used to measure the complexity of a program's control flow. It is defined as the number of linearly independent paths through a program's source code. This metric is used to quantify the complexity of a program's logic and can help assess the ease of testing, maintenance, and understanding of the code.

## Formula:

Cyclomatic complexity (V) can be calculated using the formula:

**V = E - N + 2P**

Where:

- **E** is the number of edges in the control flow graph (CFG).
- **N** is the number of nodes in the control flow graph (CFG).
- **P** is the number of connected components (usually, P = 1 for a single program).

Alternatively, cyclomatic complexity can be computed using the number of decision points (e.g., if, while, for, case) in the program:

**V = D + 1**

Where:

- **D** is the number of decision points (conditional statements).

## Interpretation:

- **V = 1**: The program has no decision points and is simple.
- **V = 2-10**: The program has low to moderate complexity and can be easily understood and tested.
- **V > 10**: The program is complex and may be difficult to maintain, test, or understand. It may require refactoring.

Q33. What is Software Maintenance?

Software Maintenance is a very broad activity that includes error corrections, enhancements of capabilities, deletion of obsolete capabilities, and optimization.

Q34. What are Categories of Maintenance?

- **Corrective maintenance**

This refer to modifications initiated by defects in the software.

- **Adaptive maintenance**

It includes modifying the software to match changes in the ever changing environment.

- **Perfective maintenance**

It means improving processing efficiency or performance, or restructuring the software to improve changeability. This may include enhancement of existing system functionality, improvement in computational efficiency etc.

There are long term effects of corrective, adaptive and perfective changes. This leads to increase in the complexity of the software, which reflect deteriorating structure. The work is required to be done to maintain it or to reduce it, if possible. This work may be named as preventive maintenance.

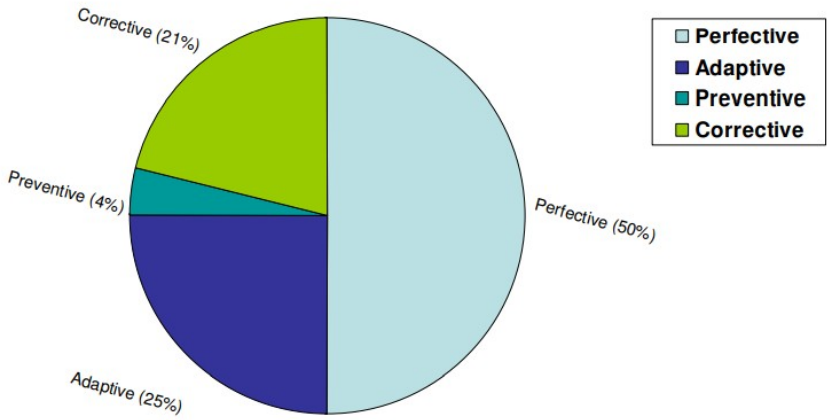



**Fig. 1:** Distribution of maintenance effort

Q35. What are the Problems During Maintenance?

- Often the program is written by another person or group of persons.
- Often the program is changed by person who did not understand it clearly.
- Program listings are not structured.
- High staff turnover.
- Information gap.
- Systems are not designed for change.

Q36. Explain the maintenance process.

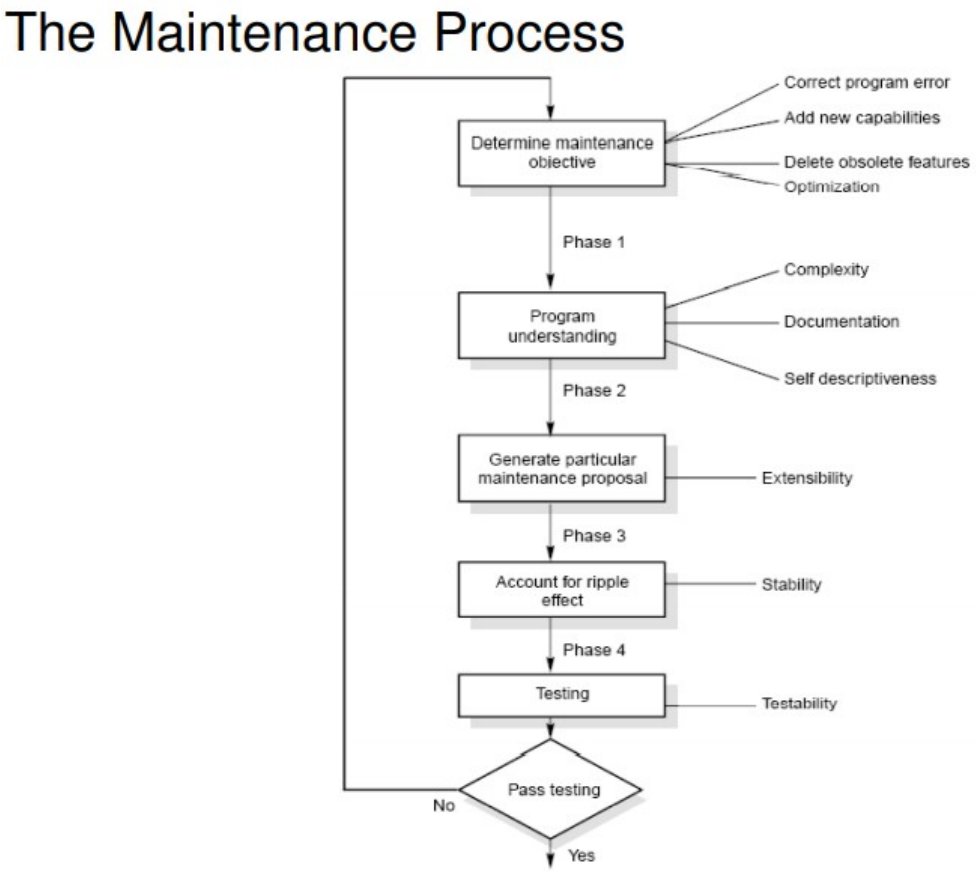



**Fig. 2:** The software maintenance process

Q37. Explain all the Software Maintenance Model.

- Quick-fix Model

This is basically an adhoc approach to maintaining software. It is a fire fighting approach, waiting for the problem to occur and then trying to fix it as quickly as possible.

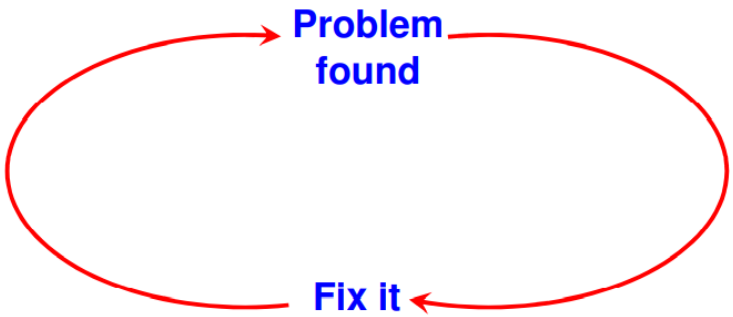



**Fig. 3:** The quick-fix model

- Iterative Enhancement Model

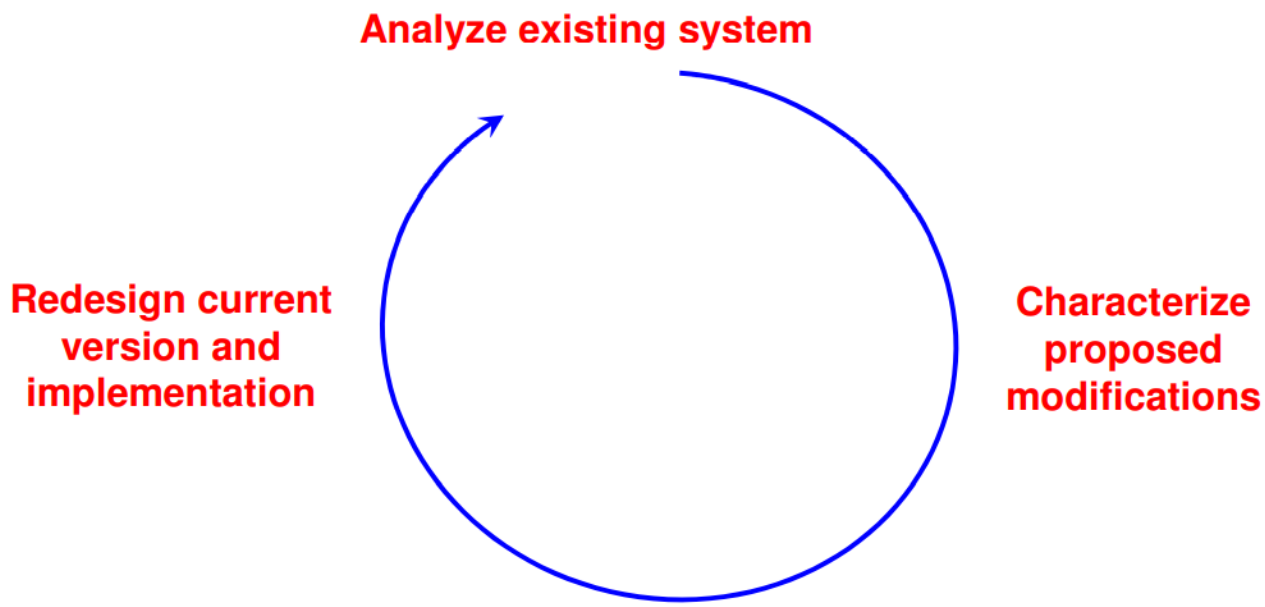



**Fig. 4:** The three stage cycle of iterative enhancement

- Reuse Oriented Model

The reuse model has four main steps:

1. Identification of the parts of the old system that are candidates for reuse.
2. Understanding these system parts.
3. Modification of the old system parts appropriate to the new requirements.
4. Integration of the modified parts into the new system.

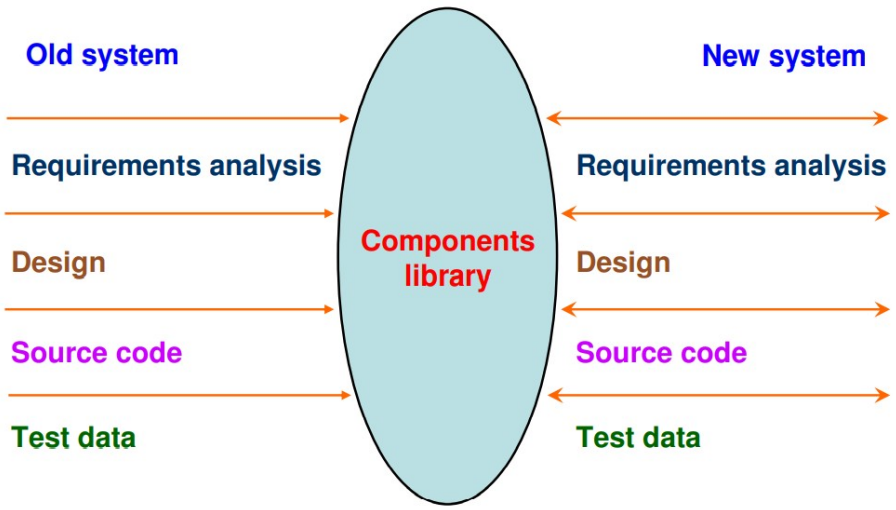



**Fig. 5:** The reuse model

- Boehm's Model

Boehm proposed a model for the maintenance process based upon the economic models and principles.

Boehm represent the maintenance process as a closed loop cycle.

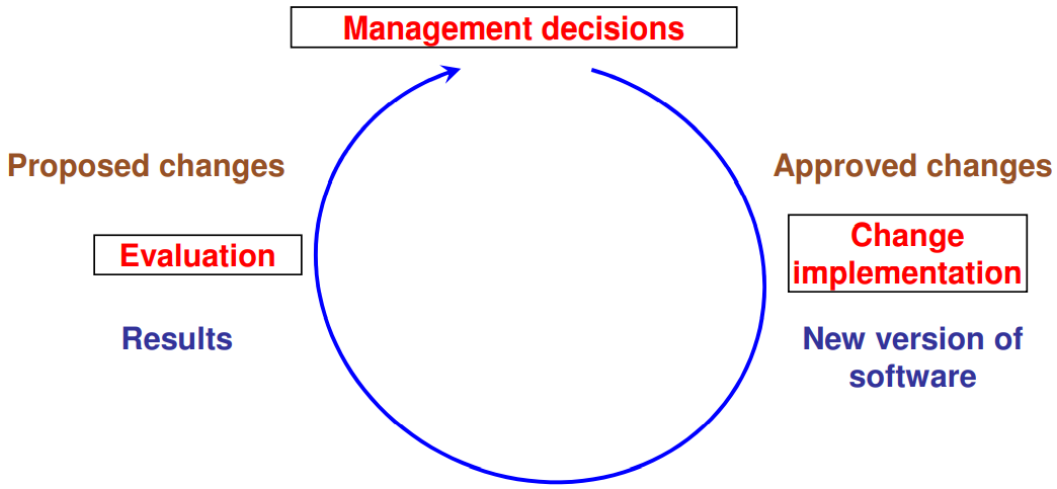



**Fig. 6:** Boehm's model

The Boehm Model, developed by Barry Boehm, focuses on software cost estimation, particularly maintenance costs. One of its key components is the **Annual Change Traffic (ACT)** metric, which helps estimate the effort required for maintenance based on the frequency and nature of changes. Important elements include:

- Boehm Model

Boehm used a quantity called Annual Change Traffic (ACT).

"The fraction of a software product's source instructions which undergo change during a year either through addition, deletion or modification".

$$ACT = \frac{KLOC_{added} + KLOC_{deleted}}{KLOC_{total}}$$

**AME = ACT x SDE**

Where, **SDE :** Software development effort in person months
**ACT :** Annual change Traffic
**EAF :** Effort Adjustment Factor

**AME = ACT * SDE * EAF**

## Taute Maintenance Model

It is a typical maintenance model and has eight phases in cycle fashion. The phases are shown in Fig. 7

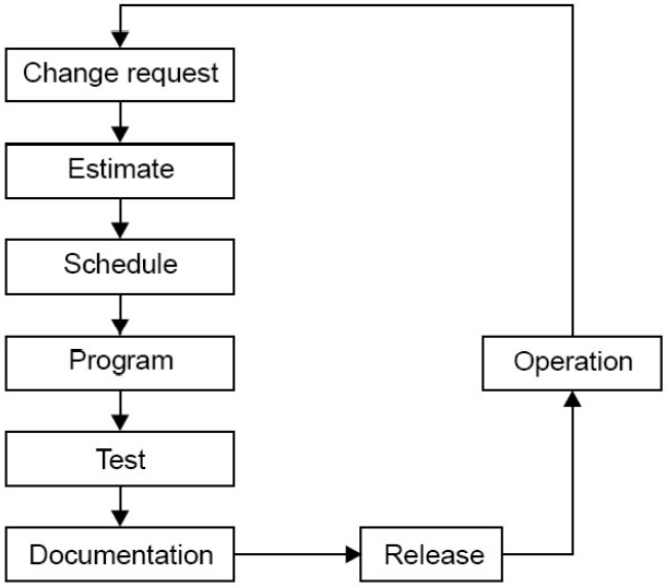



**Fig. 7:** Taute maintenance model

# Estimation of maintenance costs

| Phase | Ratio |
|---|---|
| Analysis | 1 |
| Design | 10 |
| Implementation | 100 |

**Table 3:** Defect repair ratio

**The Belady and Lehman Model**, proposed by Mehdi Belady and Mario M. Lehman, focuses on understanding software evolution and its impact on maintenance costs. This model emphasizes the relationship between software changes and various factors influencing these changes.

- Belady and Lehman Model

$$M = P + Ke^{(c-d)}$$

where

**M :** Total effort expended

**P :** Productive effort that involves analysis, design, coding, testing and evaluation.

**K :** An empirically determined constant.

**c :** Complexity measure due to lack of good design and documentation.

**d :** Degree to which maintenance team is familiar with the software.

Q37. What is selective retest technique?

- ## Selective Retest Techniques

Selective retest techniques may be more economical than the “retest-all” technique.

Selective retest techniques are broadly classified in three categories :

1. **Coverage techniques :** They are based on test coverage criteria. They locate coverable program components that have been modified, and select test cases that exercise these components.
2. **Minimization techniques:** They work like coverage techniques, except that they select minimal sets of test cases.
3. **Safe techniques:** They do not focus on coverage criteria; instead they select every test case that cause a modified program to produce different output than its original version.

**Inclusiveness** measures the extent to which a technique chooses test cases that will cause the modified program to produce different output than the original program, and thereby expose faults caused by modifications.

**Precision** measures the ability of a technique to avoid choosing test cases that will not cause the modified program to produce different output than the original program.

**Efficiency** measures the computational cost, and thus, practically, of a technique.

**Generality** measures the ability of a technique to handle realistic and diverse language constructs, arbitrarily complex modifications, and realistic testing applications.

**Q38. Discuss Reverse Engineering and Re-engineering in detail.**

**Reverse Engineering:**

Reverse engineering is the process followed in order to find difficult, unknown and hidden information about a software system

Reverse Engineering encompasses a wide array of tasks related to understanding and modifying software system. This array of tasks can be broken into a number of classes

Reverse engineering can extract design information from source code, but the abstraction level, the completeness of the documentation, the degree to which tools and a human analyst work together, and the directionality of the process are highly variable [CAS88]. The abstraction level of a reverse engineering process and the tools used to effect it refers to the sophistication of the design

information that can be extracted from source code. Ideally, the abstraction level should be as high as possible. That is, the reverse engineering

The reverse engineering process is represented in Figure 30.3. Before reverse engineering activities can commence, unstructured ("dirty") source code is restructured (Section 30.4.1) so that it contains only the structured programming constructs.2 This makes the source code easier to read and provides the basis for all the subsequent reverse engineering activities. The core of reverse engineering is an activity called extract abstractions. The engineer must evaluate the old program and from the (often undocumented) source code, extract a meaningful specification of the processing that is performed, the user interface that is applied, and the program data structures or database that is used.

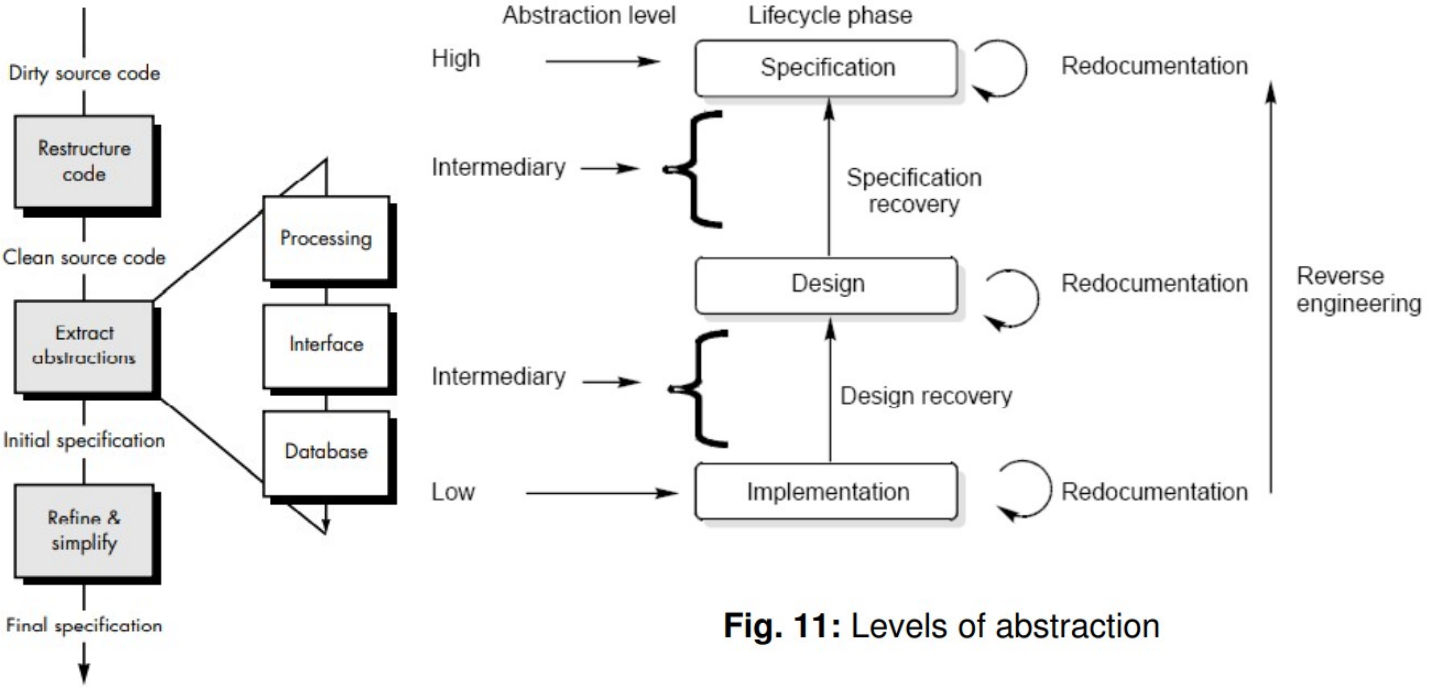



**Fig. 11:** Levels of abstraction

## Scope and Tasks

The areas there reverse engineering is applicable include (but not limited to):

1. Program comprehension
2. Redocumentation and/ or document generation
3. Recovery of design approach and design details at any level of abstraction
4. Identifying reusable components
5. Identifying components that need restructuring
6. Recovering business rules, and
7. Understanding high level system description

Levels of Reverse Engineering

Reverse Engineers detect low level implementation constructs and replace them with their high level counterparts. The process eventually results in an incremental formation of an overall architecture of the program.

- Mapping between application and program domains

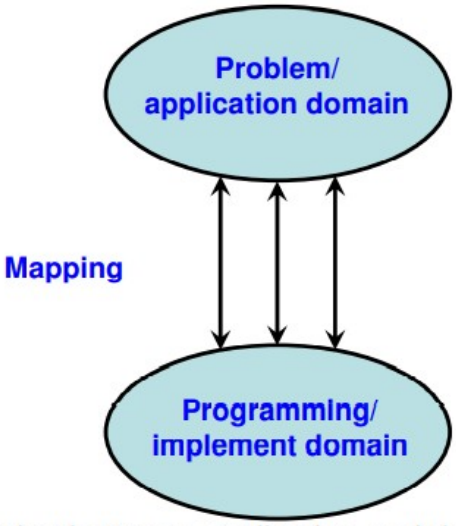



**Fig. 10:** Mapping between application and domains program

Redocumentation

Redocumentation is the recreation of a semantically equivalent representation within the same relative abstraction level.

Design recovery

Design recovery entails identifying and extracting meaningful higher level abstractions beyond those obtained directly from examination of the source code. This may be achieved from a combination of code, existing design documentation, personal experience, and knowledge of the problem and application domains.

## Software RE-Engineering

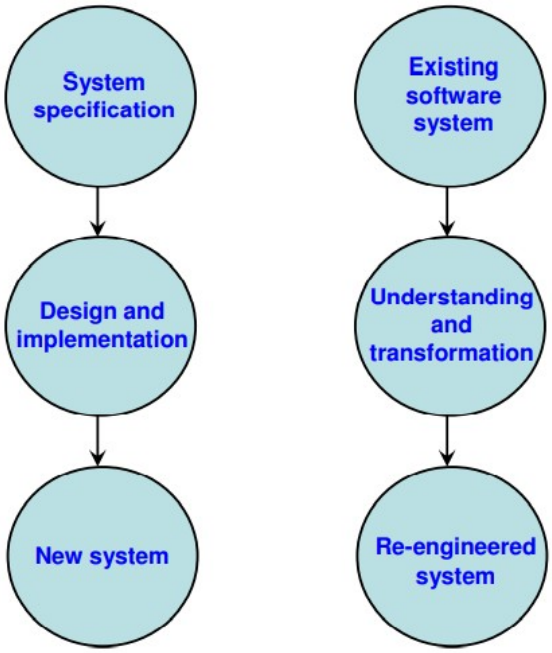



**Fig. 12:** Comparison of new software development with re-engineering

### Software Re-engineering

Software re-engineering is concerned with taking existing legacy systems and re-implementing them to make them more maintainable. The critical distinction between re-engineering and new software development is the starting point for the development, as shown in Figure 12.

### Best Practices for Software Maintenance

To effectively maintain and re-engineer legacy systems, the following suggestions may be useful:

1. **Study Code Well Before Attempting Changes**:
    - Thoroughly understanding the existing codebase helps avoid introducing new bugs during modifications.
2. **Concentrate on Overall Control Flow and Not Coding**:
    - Focus on the overall structure and logic of the software to ensure that changes align with the intended functionality.
3. **Heavily Comment Internal Code**:
    - Detailed comments enhance code readability and provide context, facilitating easier understanding for future maintainers.
4. **Create Cross References**:

- Cross-referencing related code segments helps developers understand dependencies and interactions within the system.

5. **Build Symbol Tables**:
    - Symbol tables assist in tracking variable definitions and scopes, making it easier to manage complex variable usage.
6. **Use Own Variables, Constants, and Declarations to Localize the Effect**:
    - Defining well-scoped variables reduces the risk of unintended side effects from changes.
7. **Keep Detailed Maintenance Documents**:
    - Comprehensive documentation of maintenance activities provides historical context and aids future maintenance efforts.

**Software Re-engineering Steps:-**

- **Source Code Translation**
    A. **Hardware platform update**: Organizations may need to change their standard hardware platform, and compilers for the original programming language might not be available on the new platform.
    B. **Staff skill shortages**: A lack of trained maintenance staff for the original language, especially for non-standard or outdated languages, can pose a significant challenge.
    C. **Organizational policy changes**: Organizations may standardize on a particular language to minimize support costs. Maintaining multiple versions of old compilers is often expensive.

- **Program Restructuring**
    1. **Control flow-driven restructuring**: This focuses on imposing a clear control structure within the source code, which can be inter-modular or intra-modular in nature.
    2. **Efficiency-driven restructuring**: This involves modifying a function or algorithm to improve efficiency, such as replacing an `IF-THEN-ELSE-IF-ELSE` construct with a `CASE` construct.

- **Adaptation-driven restructuring**:
    1. This involves altering the coding style to fit a new programming language or operating environment. For example, converting an imperative program in PASCAL into a functional program in LISP.

## Q39. What is Configuration Management?

The process of software development and maintenance is controlled is called configuration management. The configuration management is different in development and maintenance phases of life cycle due to different environments.

- **Configuration Management Activities**

The activities are divided into four broad categories.

1. The identification of the components and changes
2. The control of the way by which the changes are made
3. Auditing the changes
4. Status accounting recording and documenting all the activities that have take place

**Q40. What are Software Versions and Version Control?**

Two types of versions namely revisions (replace) and variations (variety).

**Version Control :**

A version control tool is the first stage towards being able to manage multiple versions. Once it is in place, a detailed record of every version of the software must be kept. This comprises the

- ✓ Name of each source code component, including the variations and revisions
- ✓ The versions of the various compilers and linkers used
- ✓ The name of the software staff who constructed the component
- ✓ The date and the time at which it was constructed

**Q41. What is Change Control Process?**

Change control process comes into effect when the software and associated documentation are delivered to configuration management change request form (as shown in fig. 14), which should record the recommendations regarding the change.

**CHANGE REQUEST FORM**

Project ID:

Change Requester with date:

Requested change with date:

Change analyzer:

Components affected:

Q42. What is Software Documentation? Explain User and System Documentation.

**Software Documentation** is the written record that provides information about a software system, aiming to communicate its purpose, content, and details with clarity.

### User Documentation

This type of documentation is intended for end-users. It provides instructions on how to use the software effectively, including user guides, tutorials, and troubleshooting steps. The goal is to make the software accessible and understandable for its intended audience.

- User Documentation

| S.No. | Document | Function |
|---|---|---|
| 1. | System Overview | Provides general description of system's functions. |
| 2. | Installation Guide | Describes how to set up the system, customize it to local hardware needs and configure it to particular hardware and other software systems. |
| 3. | Beginner's Guide | Provides simple explanations of how to start using the system. |
| 4. | Reference Guide | Provides in depth description of each system facility and how it can be used. |
| 5. | Enhancement | Booklet Contains a summary of new features. |
| 6. | Quick reference card | Serves as a factual lookup. |
| 7. | System administration | Provides information on services such as net-working, security and upgrading. |

**Table 5:** User Documentation

### System Documentation

This encompasses detailed records about the system's analysis, specifications, design, implementation, testing, security, error diagnosis, and recovery processes. It is primarily for developers and technical personnel involved in maintaining or enhancing the system.

| S.No. | Document | Function |
|---|---|---|
| 1. | System Rationale | Describes the objectives of the entire system. |
| 2. | SRS | Provides information on exact requirements of system as agreed between user and developers. |
| 3. | Specification/ Design | Provides description of:<br>(i) How system requirements are implemented.<br>(ii) How the system is decomposed into a set of interacting program units.<br>(iii) The function of each program unit. |
| 4. | Implementation | Provides description of:<br>(i) How the detailed system design is expressed in some formal programming language.<br>(ii) Program actions in the form of intra program comments. |